(총괄·1 세부) 지능형 디지털 트윈 연합 운용 및 예측 핵심기술 개발

연합트윈 기술해설서

동적 데이터/이벤트 트래킹

티오이십일콤즈

2026-08-25

문서 정보

구분	내용
문서명	연합트윈 기술해설서 동적 데이터/이벤트 트래킹
발행일	2026-08-25
작성 주체	티오이십일콤즈
대상 저장소	https://github.com/etri/FeDiT.git
대상 모듈	OPFW/msa_dynamic_event_tracker

작성 기준

- 본 문서는 GitHub 저장소 `https://github.com/etri/FeDiT.git` 를 기준으로 기술합니다.
- 프론트엔드 소스 및 운용 자원이 GitHub 저장소에 반영 완료된 상태를 전제로 합니다.
- 본문의 `{ }` 표기는 실제 운용 환경 값으로 바꾸어 입력하는 부분입니다.

목차

문서 정보 2

작성 기준 2

목차 3

1. 개요 9

가. 문서의 목적 및 활용 방법 9

○ 작업 상황별 수행 범위 9

나. 시스템 구성 9

다. 배포 프로세스 10

2. 개발 환경 구성 12

가. 개발 환경 정보 12

○ 백엔드 12

○ 프론트엔드 12

나. 사전 준비 사항 13

○ 작업 PC 에 설치가 필요한 도구 13

○ 설치 후 터미널에서 버전을 확인합니다. 13

○ 백엔드 빌드용 JDK 11 지정 (JDK 가 여러 개 설치된 경우) 13

○ PowerShell 스크립트 실행 정책 설정 (Windows) 14

○ Docker Desktop 설치 (Windows) 15

○ STS Lombok 설치 (최초 1 회) 16

○ Windows 환경 사전 설정 (필수) 16

○ 운용 서버에 설치가 필요한 도구 17

○ 사전에 확보해야 하는 정보 17

다. 로컬 실행 환경 준비 17

○ 1) 준비되는 구성 요소 17

○ 2) 인프라 기동 18
○ 3) 기동 확인 18
○ 4) 자동 생성 결과 확인 18
○ 5) 접속 정보 19
○ 6) 중지 및 초기화 19
○ 7) 문제 해결 19
라. 문서에서 사용하는 용어 20
마. 표기 규칙 20
3. 소스 코드 내려받기 22
가. 작업 폴더 생성 22
나. Git 저장소 Clone 22
○ 필요한 모듈만 내려받기 (권장) 23
다. 내려받은 폴더 구조 24
○ 대상 모듈의 주요 폴더 구성 24
○ 백엔드 소스 구성 (`BackEnd/src/DataEventTrackingService/`) 25
○ 프론트엔드 소스 구성 (`FrontEnd/src/DataEventTrackingDashboard/`) 26
○ 운용 자원 구성 (`initResource/`) 26
4. 백엔드 빌드 28
가. 이 장에서 수행하는 작업 28
나. 프로젝트 열기 28
○ Spring Tool Suite(STS) 사용 시 28
다. 데이터베이스 준비 확인 30
라. 빌드 전 설정 확인 31
마. 빌드 32
○ STS 에서 빌드하는 경우 32
바. 백엔드 로컬 구동 확인 33

○ 기동 순서 33
○ STS Boot Dashboard 사용 34
○ 정상 기동 판단 34
사. 빌드 산출물 확인 및 반영 34
아. 명령행(CLI)으로 수행하기 36
○ 1) 사전 확인 및 소스 내려받기 36
○ 2) 빌드 37
○ 3) 로컬 인프라 기동 (Docker) 37
○ 4) 로컬 로그 설정 파일 생성 38
○ 5) 로컬 접속 설정 (소스 수정 없이) 39
○ 5-1) 환경 변수를 사용하는 이유 39
○ 6) 백엔드 기동 40
○ 7) 기동 확인 41
○ 8) 종료 및 정리 42
○ 9) 서버 배포용 산출물 반영 (선택) 42
5. 프론트엔드 빌드 43
가. 프로젝트 경로 이동 43
나. 패키지 설치 43
다. 로컬 구동 확인 44
라. 서버 URL 설정 (빌드 전 필수) 45
1) 환경 설정 파일(`.env.production`) 수정 45
2) 데이터/이벤트 트래킹 서버 주소 수정 45
마. 배포 빌드 47
○ 빌드 산출물을 운용 폴더에 반영 48
바. 빌드 산출물 확인 48
사. 명령행(CLI)으로 수행하기 50

○ 1) 사전 확인 및 소스 내려받기 50
○ 2) 패키지 설치 51
○ 3) 로컬 구동 및 화면 확인 51
○ 4) 배포 빌드 52
○ 5) 산출물 확인 52
○ 6) 빌드 산출물 화면 확인 53
6. 서버 배포 53
가. 이 장에서 수행하는 작업 53
○ 서버에서 구동되는 구성 요소 54
나. 서버 접속 및 사전 준비 54
○ 1) 서버 접속 55
○ 2) 시스템 사전 점검 55
○ 3) 시각 동기화 56
○ 4) Docker 설치 56
○ 5) docker 그룹 등록 57
○ 6) SELinux 설정 57
○ 7) 포트 선점 확인 58
○ 8) 설치 검증 58
다. 운용 폴더 이관 58
라. 데이터 저장 폴더 생성 60
○ 생성되는 폴더와 용도 60
○ SELinux 컨텍스트 적용 (Rocky / RHEL 계열 필수) 61
마. 설정 파일 수정 61
바. Docker 네트워크 생성 62
사. 백엔드 이미지 생성 63
아. 백엔드 서비스 기동 64

○ 기동되는 컨테이너와 포트 (서버 포트 : 컨테이너 포트) 64
자. 웹 서버(Nginx) 기동 65
1) 화면 파일 확인 65
2) 웹 서버(Nginx) 기동 65
3) 요청 전달(프록시) 설정 66
차. 구동 상태 확인 66
○ 백엔드 컨테이너 확인 66
○ 웹 서버 컨테이너 확인 66
○ 백엔드·웹 서버를 한 번에 확인 67
○ 로그 확인 67
○ 서버 내부에서 응답 여부 확인 68
○ 웹 브라우저 확인 주소 68
○ 배포 확인 체크리스트 68
카. 컨테이너 중지 및 재기동 69
○ 대상별 실행 위치 69
타. 문제 해결 70
7. 부록 71
가. 신규 서버 구축 순서 요약 71
나. 백엔드 갱신 배포 순서 요약 72
다. 화면 갱신 배포 순서 요약 72
○ dist 업로드 절차 73
라. 자주 사용하는 명령어 75
마. 설정 항목 일람 75
바. 인터넷이 차단된 환경에서의 설치 (폐쇄망) 76
○ 1) Docker 설치 패키지 반입 76
○ 2) 컨테이너 이미지 반입 76

사. 프록시 환경에서의 Docker 설정 77
아. Docker 미사용 시 로컬 환경 직접 구성 78
○ 1) 데이터베이스 설치 및 계정 생성 78
○ 2) 스키마 및 테이블 생성 78
○ 3) 메시지 큐 설치 78
자. 로컬 빌드·구동 검증 결과 78
○ 1) 검증 환경 79
○ 2) 백엔드 빌드 결과 79
○ 3) 프론트엔드 빌드 결과 81
○ 4) 로컬 인프라 기동 결과 81
○ 5) 로컬 로그 설정 파일 생성 (백엔드 구동 전 필수) 82
○ 6) 백엔드 로컬 구동 결과 84
○ 7) 프론트엔드 로컬 구동 결과 85
○ 8) 로컬 구동 시 서버 주소(URL) 설정 87
○ 9) 검증 결과 요약 91
○ 10) 검증 중 확인된 유의 사항 91

1. 개요

가. 문서의 목적 및 활용 방법

본 문서는 연합트윈 동적 데이터/이벤트 트래킹 시스템을 소스 코드 내려받기 단계부터 운용 서버 배포 및 구동 확인 단계까지 수행하기 위한 절차를 기술한 기술해설서입니다. 백엔드(트래킹 서비스)와 프론트엔드(모니터링 대시보드)를 모두 포함하며, 실제 입력해야 하는 명령어와 정상 여부 확인 방법을 순서대로 기술하였으므로 앞에서부터 차례대로 따라 수행하시면 됩니다.

별도의 사전 지식이 없더라도 수행할 수 있도록 각 단계마다 수행 목적, 입력 명령어, 정상 여부 확인 방법을 함께 기술하였습니다. 본문에 등장하는 용어는 2 장 「라. 문서에서 사용하는 용어」에, 명령어 표기 방식은 2 장 「마. 표기 규칙」에 정리되어 있습니다.

ㅇ 작업 상황별 수행 범위

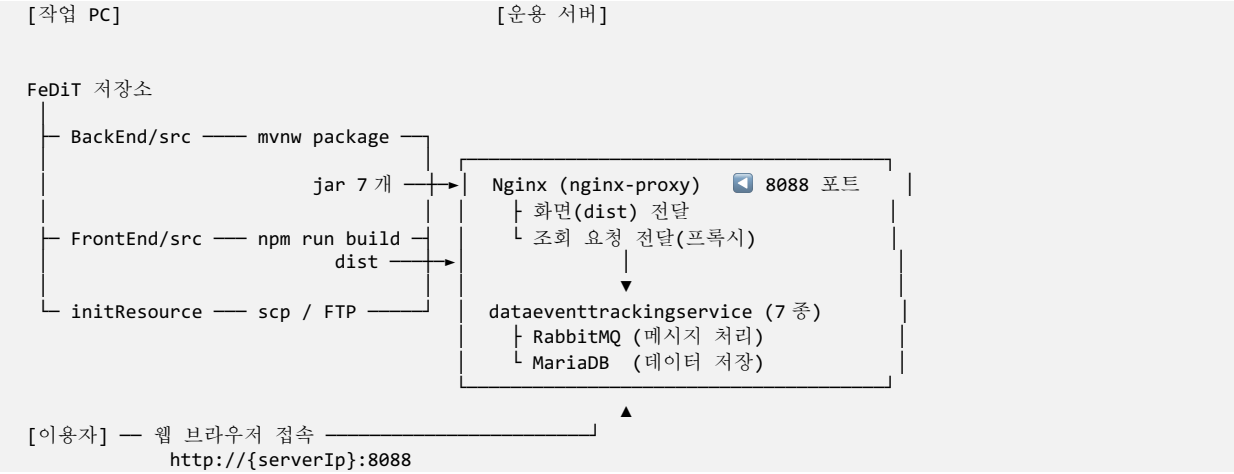
배포 상황에 따라 수행해야 하는 장이 다릅니다. 아래 표를 먼저 확인하여 본인의 작업 범위를 파악하시기 바랍니다.

작업 상황	설명	수행 범위
신규 서버 구축	서버에 아무것도 구성되어 있지 않은 상태에서 처음 배포하는 경우	2 장 ~ 6 장 (전체 수행)
백엔드 갱신 배포	백엔드 소스를 수정하여 서비스를 교체하는 경우	3 장, 4 장, 6 장 「사」·「아」 (jar 가 이미지에 포함되므로 이미지 재생성 필요)
화면 갱신 배포	대시보드 화면만 최신 버전으로 교체하는 경우	3 장, 5 장, 7 장 「다」
서비스 재기동	서버의 백엔드 또는 웹 서버를 중지 / 재기동해야 하는 경우	6 장 「카」

나. 시스템 구성

동적 데이터/이벤트 트래킹 시스템은 **백엔드 서비스 7 종**과 **모니터링 대시보드(프론트엔드)** 로 구성됩니다.

백엔드는 Spring Boot Maven 멀티 모듈 구조이며, 빌드 산출물(jar) 7 개를 단일 Docker 이미지(`msa-detrck`)로 묶어 하나의 컨테이너에서 동시 구동합니다. 프론트엔드는 React 기반 SPA 로서, 작업 PC 에서 빌드하여 생성한 정적 파일(`dist`)을 서버의 웹 서버(Nginx)가 이용자에게 전달합니다.



이용자의 브라우저는 서버의 8088 포트로 접속합니다. 화면 파일뿐만 아니라 조회 요청도 이 포트를 통해 전달되므로, 이용자 PC 에서 서버의 8088 포트로 접근할 수 있어야 합니다.

서버에서 동작하는 구성 요소는 모두 Docker 컨테이너로 구동됩니다. 운용에 필요한 설정 파일과 실행 파일은 저장소의 `initResource/` 폴더에 모여 있으며, 이 폴더를 서버의 운용 경로로 옮긴 후 그 안에서 명령을 실행하는 것이 서버 운용의 기본 구조입니다.

다. 배포 프로세스

배포는 아래 7 단계로 진행됩니다. 대시보드가 연계하는 서버 주소는 **빌드 시점에 정적 파일 안에 포함**되므로, 빌드 이전에 반드시 운용 환경에 맞게 주소를 변경해야 합니다.

단계	작업 구분	주요 내용	사용 도구 / 명령어	해당 장
1 단계	사전 준비	필요 도구 설치 및 접속 정보 확보	-	2 장
2 단계	소스 코드 내려받기	GitHub 저장소로부터 프로젝트 소스 Clone	`git clone`	3 장
3 단계	백엔드 빌드	DB 스키마 생성 및 jar 7 개 생성	`mvnw clean install`	4 장
4 단계	프론트엔드 빌드	서버 URL 설정 후 배포용 정적 파일 생성	`npm install` / `npm run build`	5 장
5 단계	운용 폴더 이관	`initResource` 폴더를 서버로 전송	`scp` / FTP	6 장「다」
6 단계	서버 기동	네트워크 생성, 이미지 생성, 컨테이너 기동	`docker compose`	6 장「바」~「자」
7 단계	구동 확인	컨테이너 정상 동작 확인	브라우저 / `curl`	6 장「차」

- ※ 5 단계 ~ 6 단계는 **신규 서버를 구축하는 경우에만** 수행합니다.
- ※ 이미 구동 중인 서버에 화면만 갱신하는 경우에는 4 단계와 7 장「다」만 수행하면 됩니다.
- ※ 백엔드만 갱신하는 경우에는 3 단계와 6 장「사」·「아」를 수행합니다. jar 는 Docker 이미지 안에 포함되므로, jar 만 교체하고 컨테이너를 재기동해서는 반영되지 않습니다.

2. 개발 환경 구성

가. 개발 환경 정보

○ 백엔드

구분	내용
언어	Java 11
프레임워크	Spring Boot 2.7.13
빌드 도구	Maven (프로젝트에 `mvnw` 래퍼 포함)
데이터베이스	MariaDB 10 (개발 시 MySQL 5.7.30 호환)
프로젝트 구조	Spring Boot Maven Multi Module
메시지 큐	RabbitMQ 3.7.14
개발 도구	Spring Tool Suite(STS) 또는 IntelliJ IDEA

○ 프론트엔드

구분	내용
언어	TypeScript 5.8
런타임	Node.js 20.19 이상 또는 22.12 이상 (npm 포함)
프레임워크	React 19.1
빌드 도구	Vite 7.0
UI 라이브러리	Tailwind CSS 4.1, daisyUI 5.0
주요 라이브러리	TanStack Query 5(서버 상태 관리), Jotai 2(전역 상태 관리), React Router 7(라우팅), Axios 1.10(HTTP 통신), RJSF 6(동적 폼)
운용 웹 서버	Nginx (정적 파일 서빙 및 API 프록시)
형상 관리	Git

> ※ Vite 7.0 은 Node.js 20.19 미만 버전을 지원하지 않습니다. Node.js 18 에서는 구동 및 빌드가 정상 동작하지 않으므로 반드시 버전을 확인하시기 바랍니다.

나. 사전 준비 사항

○ 작업 PC 에 설치가 필요한 도구

구분	도구	확인 / 설치 경로
필수	Git	https://git-scm.com
필수 (백엔드)	JDK 11 (프로젝트 빌드용)	https://adoptium.net
필수 (백엔드)	Lombok (STS 연동용)	https://projectlombok.org/download
필수 (백엔드)	Spring Tool Suite (STS)	https://spring.io/tools
필수 (프론트)	Node.js (20.19 이상)	https://nodejs.org
필수 (프론트)	Visual Studio Code	https://code.visualstudio.com
필수	FTP 클라이언트	FileZilla (https://filezilla-project.org)
필수	SSH 접속 도구	터미널(macOS / Linux), PuTTY 또는 MobaXterm(Windows)
필수 (백엔드)	JDK 17 (STS 구동용)	https://adoptium.net
필수 (백엔드)	Docker Desktop (로컬 실행 환경 구성용)	https://www.docker.com/products/docker-desktop
권장	압축 도구 (7-Zip)	https://www.7-zip.org
선택	MySQL Workbench	개발용 DB 스키마 확인

○ 설치 후 터미널에서 버전을 확인합니다.

```
git --version
java -version    # 11.x
node -v          # v20.19.0 이상 또는 v22.12.0 이상
npm -v
```

- 명령어를 입력했을 때 버전이 출력되면 정상적으로 설치된 것입니다. `command not found` 가 출력되면 설치가 되지 않았거나 재부팅이 필요한 상태입니다.

○ 백엔드 빌드용 JDK 11 지정 (JDK 가 여러 개 설치된 경우)

위 표와 같이 STS 구동용 JDK 17 과 빌드용 JDK 11 을 함께 설치하면, 터미널의 기본 JDK 가 17 로 잡혀 백엔드 빌드가 실패하거나 클래스 파일 버전이 맞지 않는 오류가 발생할 수 있습니다.

명령행에서 빌드하기 전에 아래와 같이 JDK 11 을 지정합니다.

```
# macOS
/usr/libexec/java_home -V                              # 설치된 JDK 목록 확인
export JAVA_HOME=$(/usr/libexec/java_home -v 11)
java -version                                          # 11.x 로 표시되는지 확인
```

```
# Linux
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk
export PATH=$JAVA_HOME/bin:$PATH
java -version


# Windows (PowerShell)
$env:JAVA_HOME = "C:\Program Files\Eclipse Adoptium\jdk-11.0.21.9-hotspot"
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
java -version
```

> ※ 위 설정은 명령을 입력한 터미널 창에서만 유지됩니다. 창을 새로 열면 다시 지정해야 하며, 영구적으로 적용하려면 Windows 는 [시스템 환경 변수 편집], macOS · Linux 는 셸 설정 파일(`~/.zshrc` 등)에 등록합니다.
>
> ※ JDK 설치 경로는 환경마다 다릅니다. Windows 에서는 `Get-ChildItem "C:\Program Files\Eclipse Adoptium"` 로 실제 폴더명을 확인한 뒤 입력하십시오.
>
> ※ STS 에서 빌드하는 경우에는 이 설정이 필요하지 않습니다. STS 는 [Window] – [Preferences] – [Java] – [Installed JREs] 에서 지정한 JDK 를 사용합니다.

○ PowerShell 스크립트 실행 정책 설정 (Windows)

Windows PowerShell 에서 `npm` 명령은 `npm.ps1` 스크립트를 통해 실행됩니다. Windows 의 기본 실행 정책은 `Restricted` 이므로, 별도로 설정하지 않으면 `node -v` 는 되는데 `npm` 만 아래 오류로 실행되지 않습니다. 프론트엔드 작업 전에 반드시 확인하십시오.

```
npm : 이 시스템에서 스크립트를 실행할 수 없으므로
C:\Program Files\nodejs\npm.ps1 파일을 로드할 수 없습니다.
```

1) 현재 상태를 확인합니다.

```
Get-ExecutionPolicy          # 실제 적용되는 값
Get-ExecutionPolicy -List    # 범위별 설정값
```

- `Get-ExecutionPolicy` 가 `RemoteSigned` 또는 `Bypass` 이면 이미 허용된 상태이므로 조치가 필요 없습니다.
- `Restricted` 이면 아래 2)를 수행합니다.
- `Get-ExecutionPolicy -List` 의 모든 범위가 `Undefined` 로 표시되는 것은 정상입니다. 아무 설정도 하지 않은 기본 상태를 뜻하며, 이때 적용되는 값이 `Restricted` 입니다.

2) 실행 정책을 허용합니다. 관리자 권한은 필요하지 않습니다.

```
# 권장 – 사용자 계정에 적용. 창을 새로 열어도 유지됩니다.
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned -Force


# 임시 – 현재 창에서만 적용. 창을 닫으면 원래대로 돌아갑니다.
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
```

3) 적용 여부를 확인합니다.

```
Get-ExecutionPolicy          # RemoteSigned 로 표시되면 완료
npm -v                       # 버전이 출력되면 정상
```

※ `RemoteSigned` 는 인터넷에서 내려받은 서명 없는 스크립트만 차단하고, 로컬 스크립트와 npm 등 설치 도구의 스크립트는 허용하는 표준 개발 설정입니다.

※ 정책을 변경할 수 없는 환경(그룹 정책이 적용된 PC 등)에서는 `npm` 대신 `npm.cmd` 를 사용하면 됩니다. (예: `npm.cmd install`, `npm.cmd run build`) 동작 결과는 동일합니다. `Get-ExecutionPolicy -List` 의 `MachinePolicy` · `UserPolicy` 행에 값이 지정되어 있으면 이 경우에 해당합니다.

※ `-Scope Process` 로 설정한 경우 그 창에서만 유효합니다. 다른 창에서 `npm` 을 실행하면 다시 오류가 발생하므로, 반복 작업에는 `-Scope CurrentUser` 를 사용하십시오.

※ Windows PowerShell 5.1 과 PowerShell 7(pwsh)은 실행 정책이 각각 따로 저장됩니다. 한쪽에서 설정해도 다른 쪽에는 적용되지 않습니다.

※ `node -v` 는 실행 파일(node.exe)을 직접 호출하므로 이 정책의 영향을 받지 않습니다. `node` 는 되는데 `npm` 만 동작하지 않는 경우 본 항목을 확인하십시오.

○ Docker Desktop 설치 (Windows)

2 장「다. 로컬 실행 환경 준비」의 데이터베이스와 메시지 큐는 Docker 컨테이너로 기동합니다. **작업 PC 에 Docker Desktop 이 설치되어 있어야 합니다.**

1. 관리자 권한 PowerShell 에서 WSL2 를 준비한 뒤 PC 를 재부팅합니다.

```
wsl --install
# 이미 설치되어 있다면
wsl --update
wsl --set-default-version 2
```

2. 공식 사이트(https://www.docker.com/products/docker-desktop/)에서 "Download for Windows (AMD64)" 설치 파일을 내려받습니다.
3. 설치 마법사에서 "Use WSL 2 instead of Hyper-V (recommended)" 옵션을 유지한 채 설치하고, 완료 후 재부팅 또는 재로그인합니다.
4. 설치가 끝나면 Docker Desktop 을 실행한 뒤 아래 명령어로 확인합니다.

```
docker --version
docker compose version
docker run hello-world     # "Hello from Docker!" 출력 시 정상
```

※ 컨테이너를 기동하려면 Docker Desktop 이 실행 중이어야 합니다. 실행되지 않은 상태에서는 `Cannot connect to the Docker daemon` 오류가 발생합니다.

※ 본 모듈의 로컬 인프라는 컨테이너 2 개(MariaDB · RabbitMQ)이므로 Docker Desktop 의 기본 메모리 설정으로 충분합니다.

> ※ 리눅스 PC 에서 작업하는 경우 Docker Desktop 대신 Docker Engine 을 설치합니다. 설치 절차는 6 장「나. 서버 접속 및 사전 준비」의「○ 4) Docker 설치」와 동일합니다.

○ STS Lombok 설치 (최초 1 회)

백엔드 소스는 Lombok 을 사용합니다. Lombok 라이브러리 자체는 빌드 시 자동으로 내려받아지므로 별도로 설치할 필요가 없으나, **STS 가 소스를 정상적으로 인식하려면 Lombok 을 STS 에 설치해야 합니다.** 설치하지 않으면 프로젝트를 Import 한 직후 The method getXxx() is undefined 형태의 오류가 다수 표시됩니다.

1. https://projectlombok.org/download 에서 `lombok.jar` 를 내려받습니다.
2. 내려받은 폴더에서 아래 명령어를 실행합니다.

```
java -jar lombok.jar
```

3. 설치 화면의 목록에서 STS 설치 경로가 선택되어 있는지 확인한 후 `[Install / Update]` 를 클릭합니다. 목록에 표시되지 않으면 `[Specify location…]` 으로 STS 설치 폴더를 직접 지정합니다.
4. STS 를 재시작합니다.
5. `[Help]` – `[About Spring Tool Suite 4]` 화면 하단에 `Lombok v1.18.30` 이 표시되면 정상입니다.

> ※ 명령행(`mvnw`)으로만 빌드하는 경우에는 이 절차가 필요하지 않습니다. Lombok 은 각 모듈의 `pom.xml` 에 `1.18.30` 버전으로 선언되어 있어 빌드 시 자동으로 내려받아집니다.
>
> ※ IntelliJ IDEA 를 사용하는 경우 Lombok 플러그인이 기본 포함되어 있으며, `[Settings]` – `[Build, Execution, Deployment]` – `[Compiler]` – `[Annotation Processors]` 에서 "Enable annotation processing" 만 체크하면 됩니다.

○ Windows 환경 사전 설정 (필수)

본 저장소에는 경로 길이가 긴 소스 파일이 포함되어 있어, Windows 의 기본 경로 길이 제한(260 자)에 걸려 Clone 이 실패할 수 있습니다. **Clone 이전에** 아래 설정을 먼저 수행하십시오.

```
git config --global core.longpaths true
```

> ※ 작업 폴더를 `C:\FeDiT` 와 같이 짧은 경로에 두면 더 안전합니다. 바탕화면이나 문서 폴더 하위의 깊은 경로는 피하는 것을 권장합니다.

○ 운용 서버에 설치가 필요한 도구

서버의 서비스는 모두 Docker 컨테이너로 구동되므로, 서버에 Docker 가 설치되어 있어야 합니다.

서버에 접속하여 아래 명령어로 설치 여부를 확인합니다.

```
docker -v                # 예) Docker version 24.0.7
docker compose version   # 예) Docker Compose version v2.21.0
```

`command not found` 로 출력되면 설치되지 않은 상태입니다. 이 경우 별도의 준비 없이 6 장 「나」의 절차를 그대로 수행하면 되므로, 지금 조치할 필요는 없습니다.

> ※ 본 문서의 서버 측 절차는 Rocky Linux 9 에 SSH 포트만 열려 있고 그 외에는 아무것도 설치되어 있지 않은 상태를 기준으로 작성되었습니다. Docker 설치, 계정 권한, SELinux 설정까지 6 장 「나」·「마」에 모두 포함되어 있습니다.
>
> ※ `docker compose` 명령이 동작하지 않고 `docker-compose` 명령만 동작하는 구버전 환경에서는, 본 문서의 `docker compose` 를 `docker-compose` 로 바꾸어 입력하시면 됩니다.

○ 사전에 확보해야 하는 정보

구분	필요 정보	용도
Git	저장소 주소, (공개 전환 전) GitHub 계정 및 접근 권한	3 장 소스 코드 내려받기
서버	서버 IP, SSH 계정 / 비밀번호, SSH 포트	6 장 서버 접속
서버	FTP 계정 / 비밀번호, FTP 포트	6 장 파일 업로드
서버	배포 경로에 대한 쓰기 권한	6 장
연계 서버	데이터/이벤트 트래킹 서버 주소, 메타 정보 서버 주소	5 장 서버 URL 설정

다. 로컬 실행 환경 준비

로컬에서 소스를 빌드하고 구동하려면 **데이터베이스와 메시지 큐가 먼저 준비되어 있어야 합니다.** 저장소에 포함된 로컬 전용 설정 파일로 두 가지를 한 번에 기동합니다.

○ 1) 준비되는 구성 요소

구분	역할	준비 방식
MariaDB	스키마 · 계정 · 테이블 저장	`docker-compose.local.yml` 로 기동

RabbitMQ	서비스 간 메시지 연계	`docker-compose.local.yml` 로 기동

※ 이 파일은 **로컬 개발 전용**입니다. 서버 배포에는 `docker-compose.yml` 을 사용합니다. (6 장 참고)

※ 백엔드 서비스는 포함되어 있지 않습니다. 백엔드는 4 장에서 빌드한 뒤 개발 도구에서 기동합니다.

○ 2) 인프라 기동

작업 PC 에서 Docker Desktop 을 실행한 후, 아래 명령어를 입력합니다.

```
cd {작업폴더}/OPFW/msa_dynamic_event_tracker/initResource/DataEventTrackingService
docker compose -f docker-compose.local.yml up -d
```

최초 실행 시 이미지(MariaDB · RabbitMQ)를 내려받으므로 수 분이 소요될 수 있습니다.

○ 3) 기동 확인

```
docker compose -f docker-compose.local.yml ps
```

컨테이너 2 개가 모두 `Up` 으로 표시되면 정상입니다.

○ 4) 자동 생성 결과 확인

MariaDB 가 최초 기동될 때 **스키마 · 서비스 계정 · 테이블이 모두 자동으로 생성**됩니다. 별도로 SQL 을 실행하거나 계정을 만들 필요가 없습니다.

자동 생성 항목	내용
스키마	`dataEventTrackings`
서비스 계정	`fdtTwin` (해당 스키마에 대한 권한 포함)
테이블	`db/initdb.d/create_table.sql` 이 실행되어 **11 개** 생성

아래 명령어로 확인합니다.

```
docker exec detrck-mariadb-local mariadb -ufdtTwin -pFdt1234!@#$ -e "show tables from
dataEventTrackings;"
```

※ 자동 생성은 **최초 기동 시 1 회만** 수행됩니다. 이미 데이터가 있는 상태에서는 실행되지 않습니다.

처음부터 다시 만들려면 6) 의 초기화 절차를 수행하십시오.

○ 5) 접속 정보

4 장「라. 빌드 전 설정 확인」에서 각 모듈의 `application.properties` 에 아래 값을 입력합니다.

항목	로컬 기동 시 값
`spring.datasource.hikari.jdbc-url`	`jdbc:mariadb://localhost:30068/dataEventTrackings` (이하 옵션은 기존 값 유지)
`spring.datasource.hikari.username` / `password`	`fdtTwin` / `Fdt1234!@#$`
`spring.rabbitmq.host` / `port`	`localhost` / `30071`
`spring.rabbitmq.username` / `password`	`guest` / `guest`

RabbitMQ 관리 콘솔은 브라우저에서 `http://localhost:30072` 로 접속합니다.

※ 컨테이너 이름(`mariadb`, `rabbitmq`)은 도커 네트워크 안에서만 통합니다. 개발 도구에서 접속할 때는 반드시 **`localhost` 와 위 표의 포트**를 사용하십시오.

※ 위 포트는 다른 모듈과 겹치지 않도록 지정되어 있습니다. 연합트윈의 세 모듈을 한 PC 에서 동시에 기동해도 충돌하지 않습니다.

○ 6) 중지 및 초기화

```
docker compose -f docker-compose.local.yml stop     # 중지 (데이터 유지)
docker compose -f docker-compose.local.yml start    # 재개
docker compose -f docker-compose.local.yml down -v  # 초기화 (데이터까지 삭제)
```

※ `down -v` 는 저장된 데이터를 모두 삭제합니다. 다시 기동하면 스키마 · 계정 · 테이블이 처음부터 새로 생성됩니다.

○ 7) 문제 해결

증상	원인	조치
`port is already allocated`	해당 포트를 다른 프로그램이 사용 중	`docker-compose.local.yml` 의 `ports:` 왼쪽(호스트) 번호만 변경
`Cannot connect to the Docker daemon`	Docker Desktop 미실행	Docker Desktop 실행 후 재시도
테이블이 생성되지 않음	이미 데이터가 있어 초기화 SQL 이 실행되지 않음	`down -v` 수행 후 다시 기동
백엔드가 DB · 메시지 큐 접속 실패	`application.properties` 의 포트 ·	5) 표 기준으로 수정

	계정이 5) 표와 다름	

라. 문서에서 사용하는 용어

용어	설명
빌드(build)	소스 코드를 실행 가능한 파일로 변환하는 작업. 백엔드는 `mvnw package`, 프론트엔드는 `npm run build` 로 수행합니다.
jar	백엔드 빌드 시 생성되는 Java 실행 파일. 서버의 Docker 이미지에 포함됩니다.
dist	프론트엔드 빌드 시 생성되는 폴더. 서버에 올리는 실제 배포 대상입니다.
Docker	프로그램을 실행 환경까지 통째로 묶어 구동하는 도구. 서버에 프로그램을 직접 설치하지 않고 실행할 수 있습니다.
이미지(image)	프로그램과 실행 환경을 묶어 놓은 설치 파일에 해당합니다.
컨테이너(container)	이미지를 실행한 상태. 실제로 동작 중인 프로그램에 해당합니다.
Docker Compose	여러 개의 컨테이너를 설정 파일(.yml) 하나로 함께 기동하는 도구.
볼륨(volume) / 마운트(mount)	서버의 특정 폴더를 컨테이너 안에서 그대로 사용하도록 연결하는 것. 서버 폴더의 파일을 바꾸면 컨테이너에도 즉시 반영됩니다.
프록시(proxy)	이용자의 요청을 대신 받아 실제 처리 서버로 전달하는 역할. 본 문서에서는 Nginx 가 이 역할을 수행합니다.
Nginx	화면 파일을 이용자에게 전달하고, 조회 요청을 백엔드로 전달하는 웹 서버.
Eureka	백엔드 서비스들이 서로를 찾을 수 있도록 등록/조회를 담당하는 서비스 레지스트리.
SSH	서버에 원격으로 접속하여 명령어를 입력하기 위한 접속 방식.
FTP	작업 PC 의 파일을 서버로 전송하기 위한 파일 전송 방식.
운용 폴더	서버에서 서비스를 구동하는 기준 폴더. 저장소의 `initResource/` 를 서버로 옮겨 `/usr/local/dataeventtrackingservice` 로 이름을 바꾼 것입니다. 6 장의 명령어는 모두 이 폴더에서 실행합니다.

마. 표기 규칙

표기	의미	예시
{ }	실제 환경에 맞는 값으로 바꾸어 입력하는 부분 (중괄호는 입력하지 않음)	`{serverIp}` → 운용 서버 주소
#	명령어 뒤에 붙는 설명(주석). 입력하지 않아도 됩니다.	`node -v  # 버전 확인`

표기	의미	예시
\	명령어가 다음 줄로 이어짐을 의미합니다. 한 줄로 이어서 입력해도 됩니다.	
회색 배경 영역	터미널에 그대로 입력하는 명령어 또는 파일 내용	
※ 표기	반드시 확인해야 하는 주의 사항	

명령어를 입력하는 위치는 아래 두 가지로 구분되므로 혼동하지 않도록 주의합니다.

구분	입력 위치	본 문서의 해당 장
작업 PC	VS Code / STS 의 터미널 또는 PC 의 터미널	3 장, 4 장, 5 장
운용 서버	SSH 로 서버에 접속한 상태의 터미널	6 장

> ※ 서버에 접속한 상태인지 확인하려면 `hostname` 또는 `pwd` 명령어를 입력하여 현재 위치를 확인합니다.

3. 소스 코드 내려받기

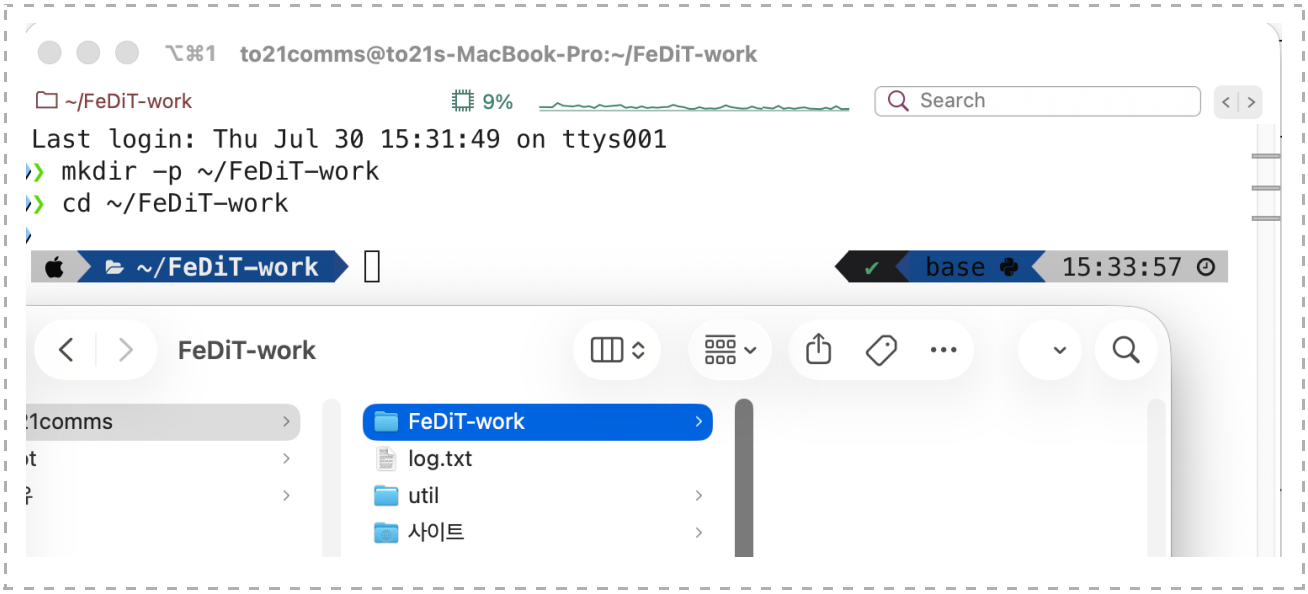
가. 작업 폴더 생성

소스 코드를 내려받을 로컬 작업 폴더를 생성합니다. 본 문서에서는 아래 경로를 기준으로 기술합니다.

```
# macOS / Linux
mkdir -p ~/FeDiT-work
cd ~/FeDiT-work


# Windows (PowerShell)
mkdir C:\FeDiT-work
cd C:\FeDiT-work
```

> ※ Windows 에서는 경로 길이 제한을 피하기 위해 드라이브 최상위에 가까운 짧은 경로를 사용하시기 바랍니다. (2 장「나」참고)



[그림 6] 작업 폴더 생성

나. Git 저장소 Clone

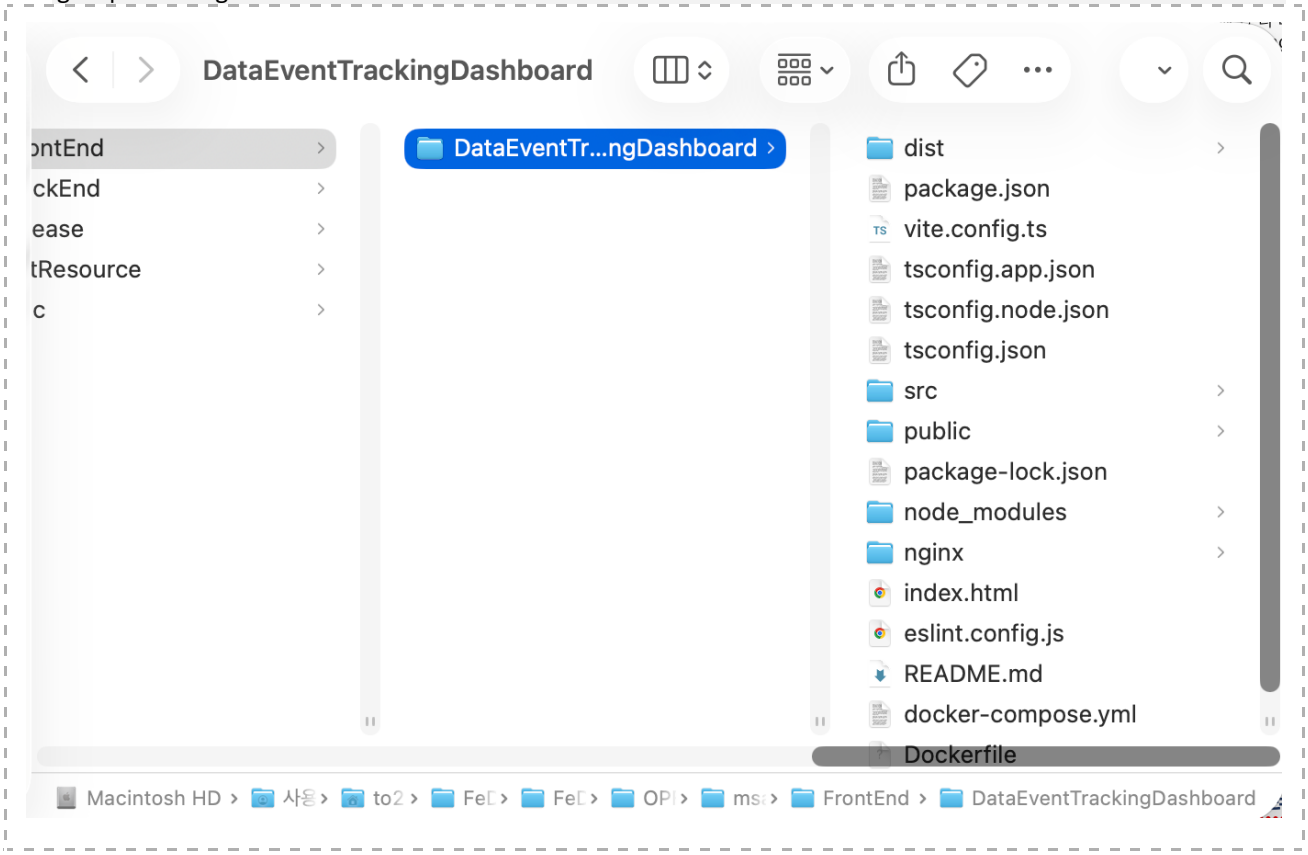
```
git clone https://github.com/etri/FeDiT.git
cd FeDiT
```

- `etri/FeDiT` 가 공개 저장소로 전환된 이후에는 별도의 계정 정보나 토큰(PAT) 입력 없이 내려받을 수 있습니다.

> ※ 공개 전환 이전에는 접근 권한이 필요합니다. 저장소 관리자에게 GitHub 계정을 협업자(Collaborator)로 등록 요청한 뒤, PAT(repo 권한) 또는 SSH 키로 인증하십시오. 등록되지 않은 계정은 저장소가 조회되지 않으며 `"repository not found"` 오류가 표시됩니다.

- Clone 이 완료되면 작업 폴더 하위에 `FeDiT` 폴더가 생성됩니다.
- 이미 내려받은 저장소가 있는 경우에는 Clone 대신 아래 명령어로 최신 소스를 반영합니다.

```
cd {작업폴더}/FeDiT
git pull origin main
```



[그림 7] GitHub 저장소 Clone

○ 필요한 모듈만 내려받기 (권장)

저장소 전체를 내려받으면 다른 모듈까지 포함되어 용량이 커집니다. 본 문서의 대상 모듈만 필요한 경우 아래와 같이 부분 내려받기를 수행할 수 있습니다.

```
git clone --filter=blob:none --no-checkout --branch main \
    https://github.com/etri/FeDiT.git
cd FeDiT
git sparse-checkout set --cone OPFW/msa_dynamic_event_tracker
git checkout
```

※ 이후 본 문서에서 `{작업폴더}` 는 Clone 으로 생성된 `FeDiT` 폴더를 가리킵니다. (예: `C:\FeDiT-work\FeDiT`)

※ 저장소 전체를 내려받으면 용량이 크므로, 본 문서의 대상 모듈만 내려받는 위 방식을 권장합니다.

다. 내려받은 폴더 구조

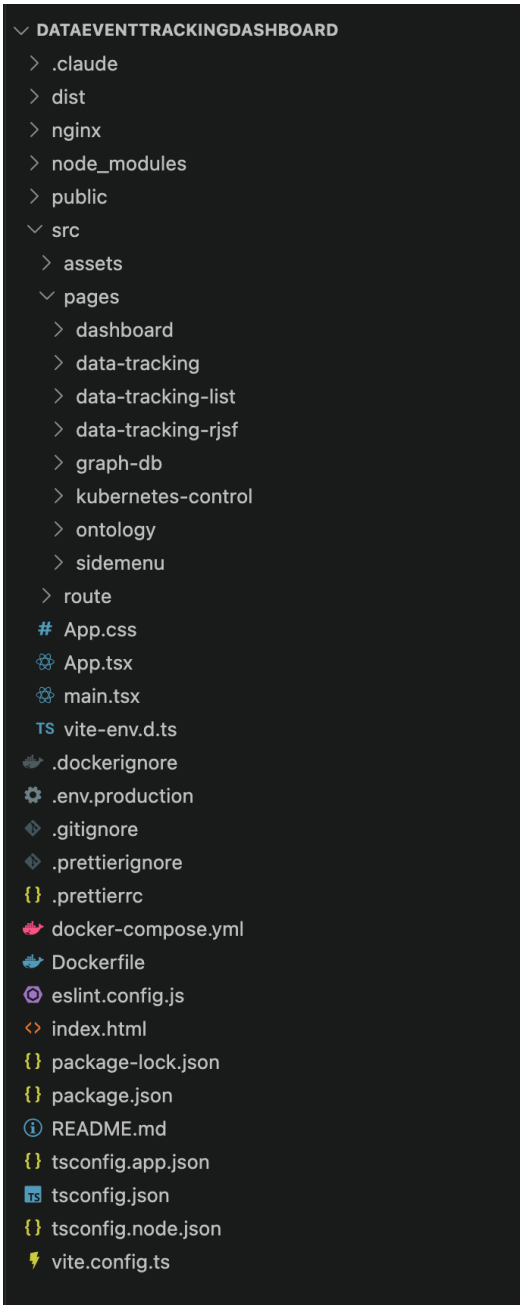
Clone 이 완료되면 아래와 같은 구조가 생성됩니다. 본 문서의 대상은

`OPFW/msa_dynamic_event_tracker` 입니다.

```
FeDiT/
├── CDFW/                                    연합 데이터 프레임워크
├── Doc/                                     전체 문서
├── OSFW/                                    개방형 서비스 프레임워크
├── OPFW/                                    연합 운용 프레임워크
│   ├── msa_device_api_gateway/
│   ├── msa_device_orchestrator/
│   ├── ontology_digital_twin_search/
│   └── msa_dynamic_event_tracker/           ◀ 본 문서의 대상 모듈
│       ├── doc/                             기술 문서
│       ├── initResource/                    서버 운용 자원 (6 장에서 서버로 이관)
│       ├── release/                         배포 산출물 보관
│       ├── BackEnd/
│       │   └── src/
│       │       └── DataEventTrackingService/   ◀ 백엔드 소스 (4 장)
│       └── FrontEnd/
│           └── src/
│               └── DataEventTrackingDashboard/   ◀ 프론트엔드 소스 (5 장)
├── AUTHORS.md
├── LICENSE
└── README.md
```

○ 대상 모듈의 주요 폴더 구성

폴더 / 파일	설명
doc/	기술해설서 및 설계서
initResource/	서버 운용에 필요한 설정 파일 및 실행 자원 (6 장에서 서버로 이관)
release/	배포 산출물 보관 폴더
BackEnd/src/DataEventTrackingService/	백엔드 프로젝트 루트 (Maven 멀티 모듈)
FrontEnd/src/DataEventTrackingDashboard/	프론트엔드 프로젝트 루트



[그림 8] Clone 완료 후 폴더 구조 (VS Code 탐색기)

○ 백엔드 소스 구성 (`BackEnd/src/DataEventTrackingService/`)

모듈	설명
`pom.xml`	상위(부모) 프로젝트 정의
`mvnw`, `mvnw.cmd`, `.mvn/`	Maven 래퍼 (별도 Maven 설치 불필요)
`common/`	공통 모듈
`DETRCKDiscoveryService/`	서비스 등록/조회 (Eureka)
`DETRCKGatewayService/`	API 게이트웨이

모듈	설명
TrackingCommandReceiver/	트래킹 명령 수신
TrackingCommandPublisher/	트래킹 명령 발행
TrackingDataSubscriber/	트래킹 데이터 수신
TrackingEventTrigger/	이벤트 트리거
TrackingCommandStatusChecker/	명령 상태 점검
create_table.sql	DB 스키마 및 테이블 생성 스크립트

○ 프론트엔드 소스 구성 (`FrontEnd/src/DataEventTrackingDashboard/`)

폴더 / 파일	설명
src/pages	화면(페이지) 단위 소스
src/config	화면 공통 설정 — 트래킹 서버 주소(trackingApi.ts) 수정 대상
src/route	화면 라우팅 정의 (CoreRouter.tsx)
public	빌드 시 그대로 복사되는 정적 리소스
dist	빌드 산출물 폴더 (빌드 수행 시 생성, 서버 배포 대상)
package.json	의존 패키지 및 실행 스크립트(dev / build) 정의
.env.production	운용 환경 변수 설정 파일 (배포 빌드에만 적용)
vite.config.ts	빌드 및 개발 서버(프록시) 설정

○ 운용 자원 구성 (`initResource/`)

이 폴더는 6 장에서 서버의 운용 경로로 이관됩니다.

```
initResource/
 ├ create_directories.sh          ← 서버에 데이터 저장 폴더를 만드는 스크립트
 ├ network-dash-create.yml        ← 네트워크(monitoring) 생성용 설정
 ├ docker-compose_front.yml       ← 웹 서버(Nginx) 기동용 설정
 ├ nginx/
 │   ├ nginx.conf                 ← 현재 적용 중인 웹 서버 설정
 │   └ backup/                    ← 이전 설정 백업본 (운용에 사용되지 않음)
 ├ frontend/
 │   └ dist/                      ← 대시보드 화면 파일 (5 장 「마」에서 복사 → 6 장 「다」로
이관)
 └ DataEventTrackingService/
     ├ docker-compose.yml         ← 백엔드 / RabbitMQ / MariaDB 기동용 설정
     ├ Dockerfile                 ← 백엔드 이미지(msa-detrck)를 만드는 설정
     ├ extStartUp.sh              ← 컨테이너 내부에서 7 개 서비스를 기동하는 스크립트
     ├ .env                       ← 데이터베이스 계정 설정
     ├ .env.prod                  ← 백엔드 서비스 접속 설정
```

```
    ├ *.jar                          ← 백엔드 실행 파일 7 개 (4 장에서 생성한 산출물로 교체)
    └ db/
        ├ conf.d/my.cnf              ← 데이터베이스 설정
        └ initdb.d/                  ← 최초 기동 시 실행되는 SQL
```

※ frontend/dist.zip, nginx/backup/, DataEventTrackingService/*.jar.bak 은 이전 이력의 백업본으로 운용에 직접 사용되지 않습니다. 배포 시 혼동하지 않도록 주의하시기 바랍니다.

※ frontend/dist 폴더에는 저장소에 포함된 시점의 화면 파일이 이미 들어 있습니다. 이는 이전 빌드본이므로, 5 장「마」○ 빌드 산출물을 운용 폴더에 반영 을 수행하여 최신 빌드본으로 교체한 뒤 서버로 이관해야 합니다. 이 단계를 건너뛰면 화면은 표시되나 이전 서버 주소로 동작합니다.

4. 백엔드 빌드

> ⚠ **이 장에서 고치는 값은 「로컬 구동용」 입니다**
>
> 「라. 빌드 전 설정 확인」에서 DB · RabbitMQ 주소를 로컬 값으로 바꾸어야 로컬에서 기동됩니다. 기본값은 운용 서버 주소로 되어 있어 그대로 두면 접속에 실패합니다.
>
> 이 값은 서버에서 `.env.prod` 환경변수가 덮어쓰므로, 배포 전에 되돌리지 않아도 서버 동작에는 영향이 없습니다. 즉 백엔드는 로컬용으로 고친 상태 그대로 빌드해서 배포해도 됩니다.

가. 이 장에서 수행하는 작업

백엔드 소스를 빌드하여 서버 배포에 사용할 **실행 파일(jar) 7 개**를 생성합니다. 본 장을 완료하면 `initResource/DataEventTrackingService/` 폴더에 최신 jar 가 준비된 상태가 됩니다.

순서	작업	해당 절
1	개발 도구에서 프로젝트를 엽니다.	나
2	데이터베이스가 준비되어 있는지 확인합니다.	다
3	접속 설정(DB · RabbitMQ)을 확인합니다.	라
4	빌드를 수행합니다.	마
5	서비스를 기동하여 정상 동작을 확인합니다.	바
6	산출물(jar 7 개)을 확인하고 배포 폴더에 반영합니다.	사

나. 프로젝트 열기

○ Spring Tool Suite(STS) 사용 시

1. STS 를 실행하고 원하는 경로에 Workspace 를 지정합니다.
2. `[File]` – `[Import…]` – `[Maven]` – `[Existing Maven Projects]` 를 선택합니다.
3. Root Directory 에 아래 경로를 지정합니다.

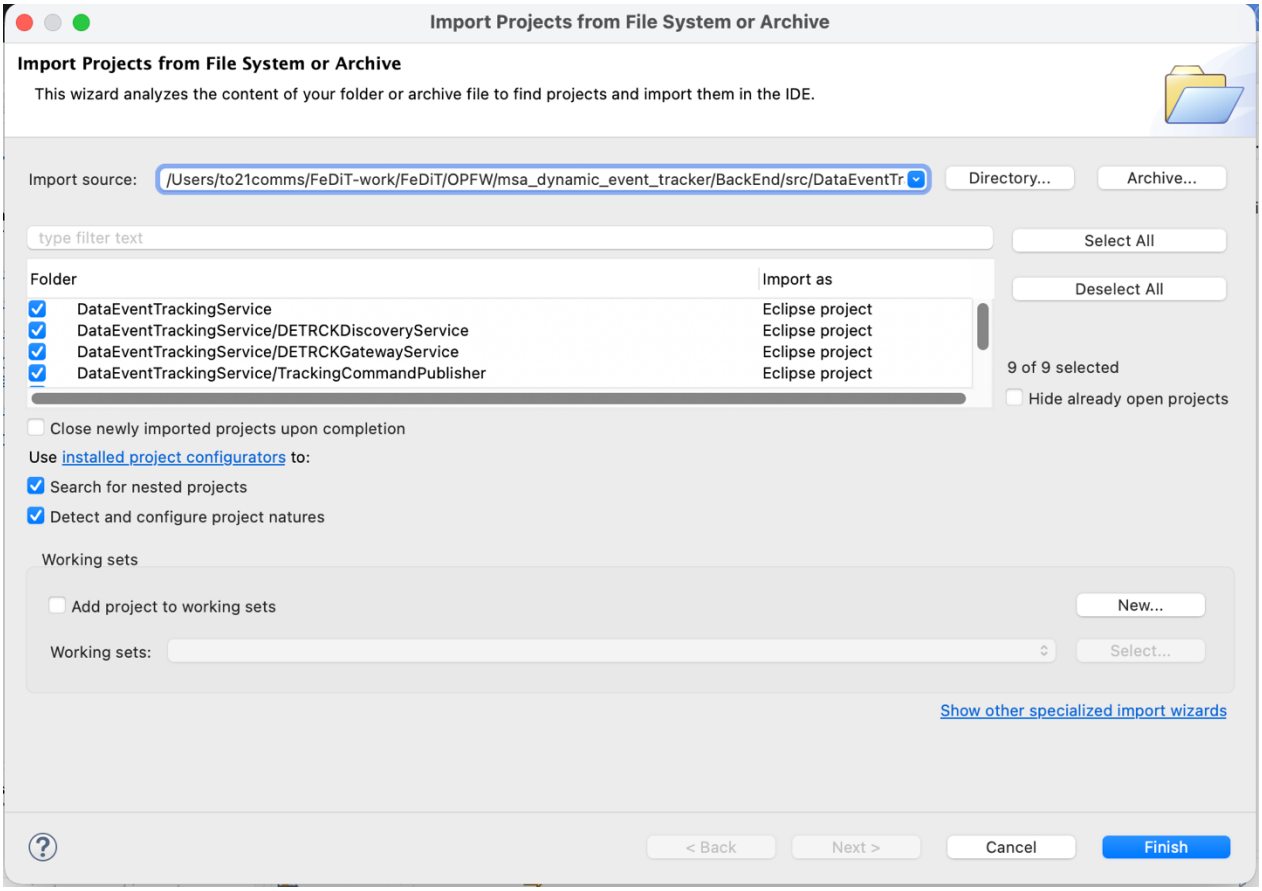
```
{작업폴더}/OPFW/msa_dynamic_event_tracker/BackEnd/src/DataEventTrackingService
```

4. 하위 모듈(common 및 서비스 7 종)이 모두 체크되어 있는지 확인한 후 `[Finish]` 를 클릭합니다.
5. `[Window]` – `[Show View]` – `[Project Explorer]` 에서 모듈이 모두 표시되는지 확인합니다.

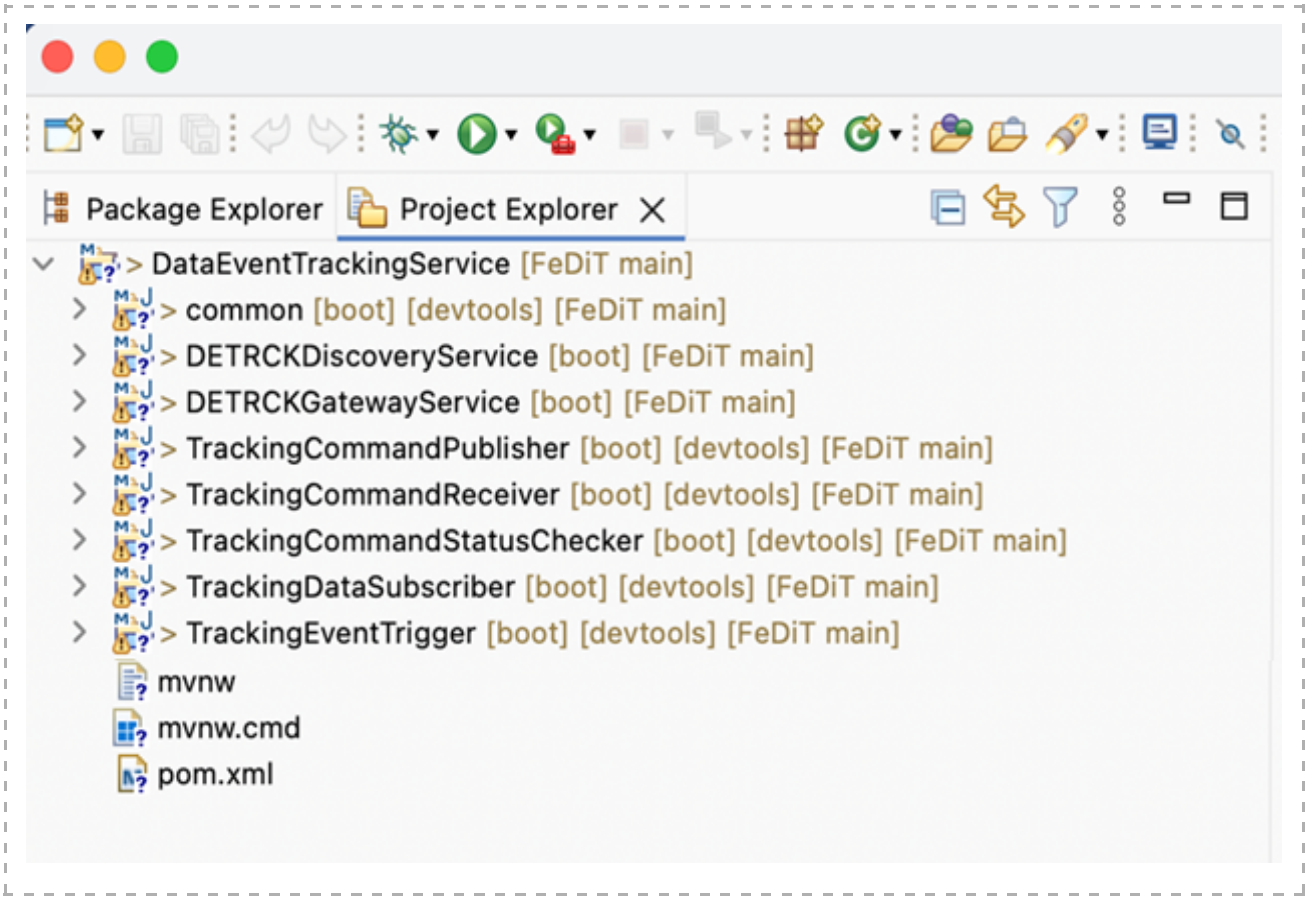
※ 상위 `pom.xml` 이 포함된 폴더를 Root Directory 로 지정해야 하위 모듈이 함께 인식됩니다.

※ Import 직후 소스 전체에 오류 표시가 나타나는 경우, 2 장 「나」의 **STS Lombok 설치**가 누락된 것입니다.

Lombok 설치 후 STS 를 재시작하고 `[Maven] – [Update Project]` 를 실행하십시오.



[그림 9] STS – Maven 프로젝트 Import



[그림 10] Project Explorer – 모듈 8 개 표시 확인

> ※ 개발 도구 없이 명령행만으로 수행하는 절차는 본 장「아. 명령행(CLI)으로 수행하기」에 한 곳으로 정리되어 있습니다.

다. 데이터베이스 준비 확인

데이터베이스는 2 장「다. 로컬 실행 환경 준비」에서 이미 준비되어 있습니다. 스키마 · 서비스 계정 · 테이블이 모두 생성된 상태이므로 본 장에서 별도로 수행할 작업은 없습니다.

준비 상태가 의심되면 아래 명령어로 확인합니다.

```
docker exec detrck-mariadb-local mariadb -ufdtTwin -pFdt1234!@#$ -e "show tables from dataEventTrackings;"
```

테이블 11 개가 출력되면 정상입니다.

> ※ 운용 서버에서는 MariaDB 컨테이너가 최초 기동될 때 `initResource/DataEventTrackingService/db/initdb.d/` 하위의 SQL 이 자동 실행됩니다. (6 장 참고)
>
> ※ 계정 정보는 반드시 운용 환경에 맞게 변경하십시오. 소스에 포함된 기본 계정을 그대로 사용하지 마시기 바랍니다.

※ Docker 를 사용할 수 없는 환경에서는 7 장「Docker 미사용 시 로컬 환경 직접 구성」을 참고하십시오.

라. 빌드 전 설정 확인

각 모듈의 `src/main/resources/application.properties` 에 DB · RabbitMQ 접속 정보가 정의되어 있습니다. 기본값이 운용 서버 주소로 되어 있으므로, 로컬에서 구동하려면 아래 표의「로컬 구동 시 값」으로 바꾸어야 합니다. 바꾸지 않으면 서비스는 기동되지만 데이터베이스 · 메시지 큐 접속에 실패합니다.

항목	로컬 구동 시 값 (application.properties 를 직접 수정)	서버 배포 시 값 (.env.prod 로 자동 주입 — 수정 불필요)
`spring.datasource.hikari.jdbc-url`	`jdbc:mariadb://localhost:30068/dataEventTrackings…`	`jdbc:mariadb://mariadb:3306/dataEventTrackings…`
`spring.datasource.hikari.username`	`fdtTwin`	`dtEtTrck`
`spring.datasource.hikari.password`	`Fdt1234!@#$`	`Fdt1234!@#$` (동일)
`spring.rabbitmq.host` / `port`	`localhost` / `30071`	`rabbitmq` / `5672`
`cors.allowed-origin-patterns` (TrackingCommandReceiver 만)	`http://localhost:[*]`	`http://{serverIp}:8088`
`eureka.client.service-url.defaultZone`	`localhost:8761` (수정 불필요)	`dataeventtrackingservice:8761`

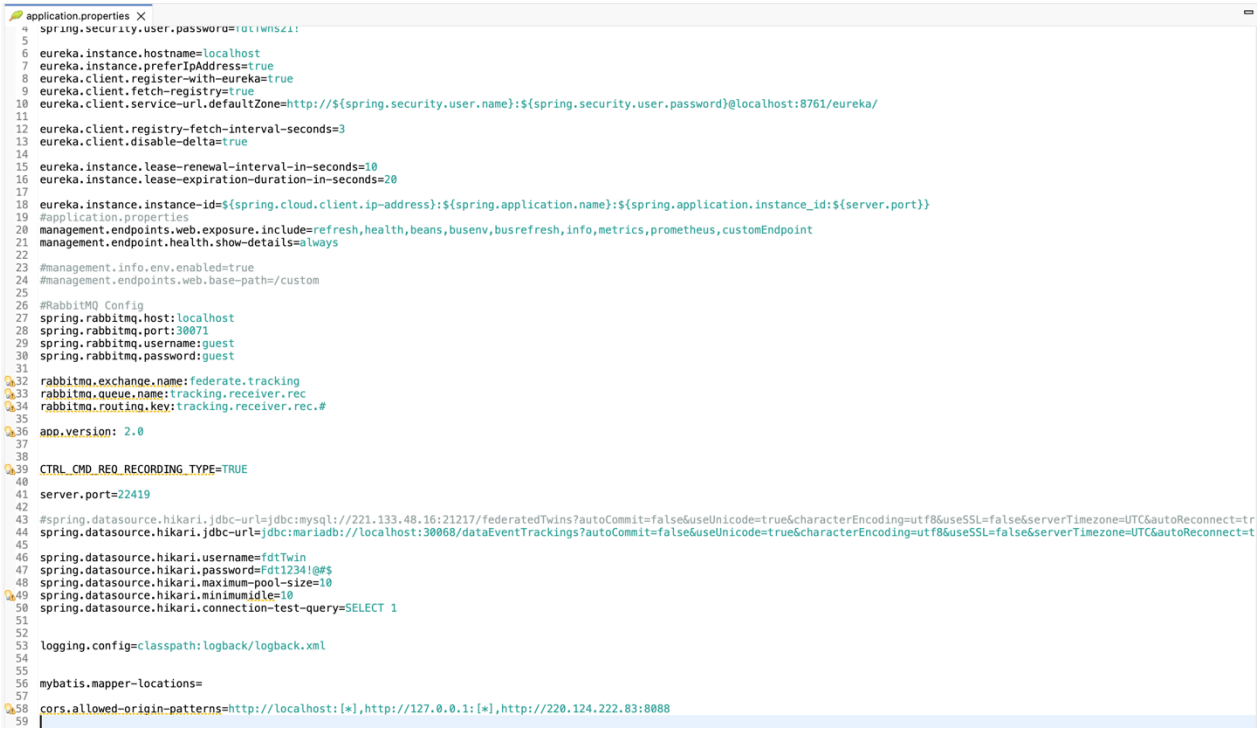
수정해야 하는 모듈은 아래와 같습니다. 같은 항목이 모듈마다 반복되므로 빠뜨리지 않도록 확인하십시오.

수정 대상 모듈	DB	RabbitMQ	CORS
TrackingCommandReceiver	○	○	○
TrackingCommandPublisher	○	○	—
TrackingDataSubscriber	○	○	—
TrackingEventTrigger	○	○	—
TrackingCommandStatusChecker	○	○	—
DETRCKGatewayService	—	○	—
DETRCKDiscoveryService	—	—	— (수정할 항목 없음)

※ 서버 배포 시에는 위 값을 되돌리지 않아도 됩니다. 운용 서버에서는 `initResource/DataEventTrackingService/.env.prod` 의 환경 변수가 `application.properties` 보다 우선 적용되기 때문입니다. 즉 로컬용으로 고친 상태 그대로 빌드하여 배포해도 서버는 서버 값으로 동작합니다.

※ 단, 컨테이너가 아닌 방식으로 `java -jar` 를 직접 실행하는 경우에는 환경 변수가 없어 `application.properties` 의 로컬 값이 그대로 적용되므로 주의하십시오.

※ 서버 쪽 값을 바꾸는 방법은 6 장「마. 설정 파일 수정」에 있습니다. 특히 `cors_allowed_origin_patterns` 는 서버 IP 가 바뀔 때마다 반드시 함께 바꾸어야 하며, 빠뜨리면 조회는 되는데 트래킹 등록·취소만 403 으로 실패합니다.



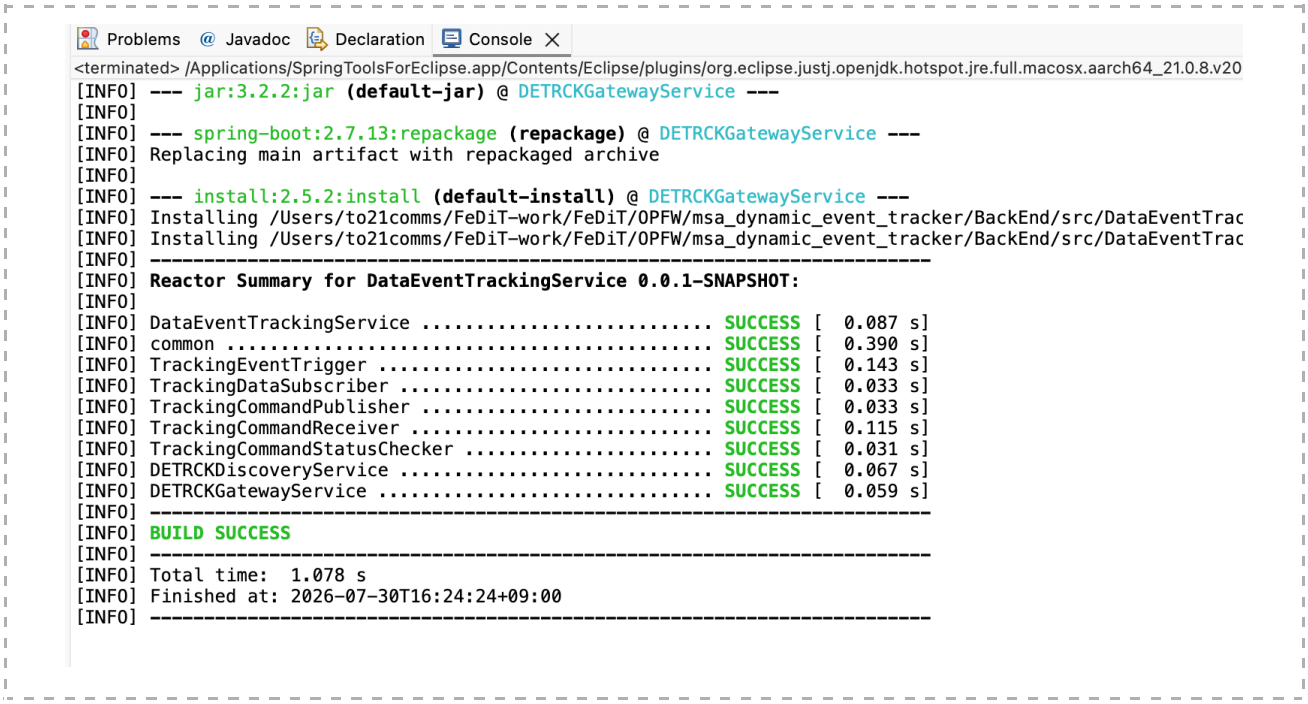
[그림 11] application.properties – 로컬 접속 정보 및 CORS 허용 출처

마. 빌드

○ STS 에서 빌드하는 경우

1. 상위 프로젝트에서 마우스 우클릭 – `[Maven]` – `[Update Project]` 를 실행합니다.
2. 마우스 우클릭 – `[Run As]` – `[Maven Clean]` 을 실행합니다.
3. 마우스 우클릭 – `[Run As]` – `[Maven Install]` 을 실행합니다.
4. Console 에 `BUILD SUCCESS` 가 출력되면 정상 완료된 것입니다.

※ 개발 도구 없이 명령행만으로 수행하는 절차는 본 장「아. 명령행(CLI)으로 수행하기」에 한 곳으로 정리되어 있습니다.



[그림 12] 빌드 수행 및 BUILD SUCCESS 확인

바. 백엔드 로컬 구동 확인

빌드가 끝나면 개발 도구에서 서비스를 기동하여 동작을 확인합니다.

○ 기동 순서

서비스 사이에 의존 관계가 있어 아래 순서로 기동해야 합니다.

순서	서비스	포트	비고
1	DETRCKDiscoveryService	8761	다른 서비스가 등록할 대상이므로 가장 먼저 기동합니다.
2	DETRCKGatewayService	8000	Eureka 에 등록됩니다.
3	TrackingCommandReceiver	22419	대시보드가 직접 호출하는 서비스입니다. Eureka 에 등록됩니다.
4	TrackingCommandPublisher	22429	이하 4 종은 Eureka 가 아니라 RabbitMQ 로 연계되므로 서로 간의 순서는 무관합니다.
5	TrackingDataSubscriber	22439	
6	TrackingEventTrigger	22449	
7	TrackingCommandStatusChecker	22459	

※ 각 서비스는 앞 단계에서 `Started {서비스명}Application in {n} seconds` 가 출력된 것을 확인한 뒤 다음 서비스를 기동합니다. 한꺼번에 올리면 어느 서비스에서 실패했는지 구분하기 어렵습니다.

※ 서버에서는 `extStartUp.sh` 가 7 종을 한 번에 기동하며, 이 스크립트는 `DETRCKDiscoveryService` 를 마지막에 실행합니다. Eureka 등록은 클라이언트가 재시도하여 이루어지므로 동작에는 문제가 없습니다. 로컬에서 하나씩 확인할 때는 위 순서를 따르는 것이 원인 파악에 유리합니다.

○ STS Boot Dashboard 사용

`[Window]` – `[Show View]` – `[Other…]` – `[Spring]` – `[Boot Dashboard]` 를 엽니다. 목록에서 서비스를 선택하고 `Start` 를 누릅니다.

○ 정상 기동 판단

Console 에 아래 로그가 출력되면 기동이 완료된 것입니다.

```
Started {서비스명}Application in {n} seconds
```

※ `RestartClassLoader` 관련 로그까지만 출력되고 위 메시지가 나오지 않으면, 아직 기동 중이거나 DB · 메시지 큐 접속에서 대기 중인 상태입니다. Console 의 오류 메시지를 확인하십시오.

※ 2 장「다」의 로컬 인프라가 기동되어 있지 않으면 접속에 실패합니다. `docker compose -f docker-compose.local.yml ps` 로 먼저 확인하십시오.

※ 개발 도구 없이 jar 를 직접 기동하는 방법은 본 장「아」○ 5) 기동 에 있습니다.

사. 빌드 산출물 확인 및 반영

빌드가 완료되면 각 모듈의 `target/` 폴더에 jar 파일이 생성됩니다.

```
BackEnd/src/DataEventTrackingService/
  ├─ DETRCKDiscoveryService/target/DETRCKDiscoveryService-0.0.1-SNAPSHOT.jar
  ├─ DETRCKGatewayService/target/DETRCKGatewayService-0.0.1-SNAPSHOT.jar
  ├─ TrackingCommandReceiver/target/TrackingCommandReceiver-0.0.1-SNAPSHOT.jar
  ├─ TrackingCommandPublisher/target/TrackingCommandPublisher-0.0.1-SNAPSHOT.jar
  ├─ TrackingDataSubscriber/target/TrackingDataSubscriber-0.0.1-SNAPSHOT.jar
  ├─ TrackingEventTrigger/target/TrackingEventTrigger-0.0.1-SNAPSHOT.jar
  └─ TrackingCommandStatusChecker/target/TrackingCommandStatusChecker-0.0.1-SNAPSHOT.jar
```

생성된 jar 7 개를 **배포 폴더로 복사**합니다. 이 폴더가 6 장에서 서버로 이관됩니다.

※ 복사 명령은 본 장「아」○ 3) 산출물(jar 7 개) 반영 및 확인 에 있습니다.

```
❯ ls -l initResource/Tracking/*.jar
-rw-r--r--  1 to21comms  staff     48078  7월  30 16:44 initResource/Tracking/com
mon-0.0.1-SNAPSHOT.jar
-rw-r--r--  1 to21comms  staff  56468133  7월  30 16:44 initResource/Tracking/DET
RCKDiscoveryService-0.0.1-SNAPSHOT.jar
-rw-r--r--  1 to21comms  staff  47586408  7월  30 16:44 initResource/Tracking/DET
RCKGatewayService-0.0.1-SNAPSHOT.jar
-rw-r--r--  1 to21comms  staff  43536928  7월  30 16:44 initResource/Tracking/Tra
ckingCommandPublisher-0.0.1-SNAPSHOT.jar
-rw-r--r--  1 to21comms  staff  69612347  7월  30 16:44 initResource/Tracking/Tra
ckingCommandReceiver-0.0.1-SNAPSHOT.jar
-rw-r--r--  1 to21comms  staff  43521098  7월  30 16:44 initResource/Tracking/Tra
ckingCommandStatusChecker-0.0.1-SNAPSHOT.jar
-rw-r--r--  1 to21comms  staff  44113260  7월  30 16:44 initResource/Tracking/Tra
ckingDataSubscriber-0.0.1-SNAPSHOT.jar
-rw-r--r--  1 to21comms  staff  28360621  7월  30 16:44 initResource/Tracking/Tra
ckingEventTrigger-0.0.1-SNAPSHOT.jar

 ~/F/FeDiT/OP/msa_dynamic_event_tracker  main !146 ?3
```

[그림 13] 빌드 산출물(jar 7 개) 반영 확인

No	실행 파일	역할	포트
1	DETRCKDiscoveryService	서비스 등록/조회(Eureka)	8761
2	DETRCKGatewayService	API 게이트웨이	8000
3	TrackingCommandReceiver	트래킹 명령 수신	22419
4	TrackingCommandPublisher	트래킹 명령 발행	22429
5	TrackingDataSubscriber	트래킹 데이터 수신	22439
6	TrackingEventTrigger	이벤트 트리거	22449
7	TrackingCommandStatusChecker	명령 상태 점검	-

> ※ 위 포트는 **컨테이너 내부 포트**입니다. 서버에서 외부로 공개되는 포트는 6 장「아」의 표를 참고하십시오.
>
> ※ `TrackingCommandStatusChecker` 는 로컬에서 개발 도구로 기동하면 `application.properties` 의 `server.port=22459` 가 적용되지만, 서버의 기동 스크립트(`extStartUp.sh`)에는 포트가 지정되어 있지 않아 기본 포트로 구동됩니다. 4 장「바」의 표에 적힌 22459 는 로컬 기준입니다.

아. 명령행(CLI)으로 수행하기

본 절은 소스 내려받기부터 빌드 · 기동 · 동작 확인까지를 개발 도구(STS) 없이 명령행만으로 수행하는 절차입니다. 소스를 수정하지 않고 복사 · 붙여넣기만으로 로컬 빌드와 테스트를 마칠 수 있도록 구성하였습니다. 각 단계의 배경 설명은 앞의 「나」 ~ 「사」에 있습니다.

```
1) 사전 확인 · 소스 내려받기
2) 빌드                          → target/ 에 jar 7 개 생성
3) 로컬 인프라 기동 (Docker)  → MariaDB · RabbitMQ
4) 로컬 로그 설정 파일 생성  → 로그 경로 오류 회피 (필수)
5) 로컬 접속 설정               → 소스 수정 없이 환경 변수로 지정
6) 백엔드 기동                   → jar 7 개를 java -jar 로 실행
7) 기동 확인
8) 종료 및 정리
9) 서버 배포용 산출물 반영 (선택)
```

> ※ 2) 빌드와 3) 로컬 인프라 기동은 서로 무관하므로 순서를 바꾸어도 됩니다. 다만 6) 백엔드 기동 시점에는 3) 이 기동되어 있어야 합니다.
>
> ※ 본 절의 명령은 하나의 터미널 창에서 1)부터 순서대로 이어서 실행하는 것을 기준으로 작성되었습니다. 각 항목의 블록에는 작업 경로 지정 줄이 포함되어 있어, 창을 새로 열었거나 중간 항목부터 시작하는 경우에도 해당 블록만 붙여넣으면 동작합니다.
>
> ※ 아래 명령은 한 줄이 하나의 명령입니다. 복사해 붙여넣을 때 한 줄이 여러 줄로 나뉘면 오류가 발생하므로, 줄이 나뉘지 않도록 붙여넣으십시오.

○ 1) 사전 확인 및 소스 내려받기

```
# Windows (PowerShell)
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force


git --version
java -version
docker version


git config --global core.longpaths true


New-Item -ItemType Directory -Force -Path C:\FeDiT-work | Out-Null
Set-Location C:\FeDiT-work
git clone --filter=blob:none --no-checkout --branch main https://github.com/etri/FeDiT.git
Set-Location C:\FeDiT-work\FeDiT
git sparse-checkout set --cone OPFW/msa_dynamic_event_tracker
git checkout


$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
Get-ChildItem $Work | Format-Table Mode, Name
# macOS / Linux
git --version
java -version
docker version
```

```
mkdir -p ~/FeDiT-work
cd ~/FeDiT-work
git clone --filter=blob:none --no-checkout --branch main https://github.com/etri/FeDiT.git
cd FeDiT
git sparse-checkout set --cone OPFW/msa_dynamic_event_tracker
git checkout


WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
ls -l $WORK
```

- `java` 는 11.x 로 표시되어야 합니다. 다른 버전이면 2 장「나」의「○ 백엔드 빌드용 JDK 11 지정」을 먼저 수행하십시오.
- `docker version` 이 Server 정보까지 출력되어야 합니다. `Cannot connect to the Docker daemon` 이 출력되면 Docker Desktop 을 실행하십시오.
- `BackEnd` · `FrontEnd` · `initResource` 폴더가 표시되면 정상입니다. (3 장「다」)

> ※ 이미 소스를 내려받은 상태라면 Clone 줄만 건너뛰고 나머지를 실행하십시오. 소스를 새로 내려받으려면 `Remove-Item -Recurse -Force C:\FeDiT-work\FeDiT` 로 기존 폴더를 삭제한 뒤 실행합니다.

○ 2) 빌드

```
# Windows (PowerShell)
java -version


$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
Set-Location "$Work\BackEnd\src\DataEventTrackingService"
.\mvnw.cmd clean package -DskipTests
# macOS / Linux
export JAVA_HOME=$(/usr/libexec/java_home -v 11)      # macOS
WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
cd $WORK/BackEnd/src/DataEventTrackingService
./mvnw clean package -DskipTests
```

- Reactor Summary 의 9 개 프로젝트가 모두 `SUCCESS` 이고 마지막에 `BUILD SUCCESS` 가 출력되면 완료입니다.
- 각 모듈의 `target/` 폴더에 jar 가 생성됩니다. 로컬 기동은 이 jar 를 그대로 사용합니다.
- 최초 실행 시에는 의존 라이브러리를 모두 내려받으므로 수 분이 소요됩니다.

> ※ PowerShell 에서는 현재 폴더의 실행 파일임을 나타내는 `.\` 를 붙여 `.\mvnw.cmd` 로 입력해야 합니다. 명령 프롬프트(cmd)에서는 `mvnw.cmd` 만 입력해도 됩니다.

○ 3) 로컬 인프라 기동 (Docker)

백엔드가 사용할 데이터베이스(MariaDB)와 메시지 큐(RabbitMQ)를 기동합니다. 저장소에 포함된 로컬 전용 파일 `docker-compose.local.yml` 을 사용합니다.

> ※ 로컬에서는 반드시 `docker-compose.local.yml` 을 사용합니다. 같은 폴더의 `docker-compose.yml` 은 운용 서버 배포용으로, 백엔드 이미지(`msa-detrck`) · 외부 네트워크(`monitoring`) · `/hdd` 볼륨을 전제로 하므로 작업 PC 에서 기동하면 안 됩니다. (6 장 참고)

```
# Windows (PowerShell)
$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
Set-Location "$Work\initResource\DataEventTrackingService"
docker compose -f docker-compose.local.yml up -d
docker compose -f docker-compose.local.yml ps
# macOS / Linux
WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
cd $WORK/initResource/DataEventTrackingService
docker compose -f docker-compose.local.yml up -d
docker compose -f docker-compose.local.yml ps
```

- `detrck-mariadb-local`(30068) 과 `detrck-rabbitmq-local`(30071 · 30072) 2 개가 `Up` 으로 표시되면 정상입니다.
- 최초 기동 시 `db/initdb.d/create_table.sql` 이 자동 실행되어 스키마 · 계정 · 테이블이 모두 생성됩니다. 별도로 SQL 을 실행할 필요가 없습니다.

```
# 데이터베이스 확인 — 테이블 11 개가 출력되면 정상
docker exec detrck-mariadb-local mariadb -ufdtTwin '-pFdt1234!@#$' -e "show tables from
dataEventTrackings;"


# 메시지 큐 확인 — 브라우저에서 관리 콘솔 접속 (guest / guest)
#   http://localhost:30072
```

○ 4) 로컬 로그 설정 파일 생성

트래킹 서비스 5 종은 로그 설정 파일(`src/main/resources/logback/logback.xml`)에 로그 경로가 `\hdd\DtEvtTracking\{서비스}` 로 지정되어 있습니다. logback 은 이 값의 역슬래시를 이스케이프 문자로 처리하므로, 앞의 역슬래시는 사라지고 `\t` 는 탭 문자로 치환됩니다. 트래킹 서비스 5 종은 마지막 폴더명이 모두 `t` 로 시작하여(`trackCmdReceiver` · `trackCmdPublisher` · `trackDataSub` · `trackEntTrigger` · `trackCmdStsChecker`) 동일하게 영향을 받습니다.

그 결과 로그 경로가 아래와 같이 깨지므로, `C:\hdd\DtEvtTracking\...` 폴더를 미리 만들어 두어도 참조되지 않습니다. 그대로 기동하면 트래킹 서비스 5 종이 모두 `APPLICATION FAILED TO START` 로 종료됩니다.

```
ERROR in ch.qos.logback.core.rolling.RollingFileAppender[ROLLING-FILE]
  - Failed to create parent directories for
    [...\DataEventTrackingService\hddDtEvtTracking→rackEntTrigger\TrackingEventTrigger.log]
      ( \hdd · \DtEvtTracking 의 역슬래시는 소실되고, \trackEntTrigger 의 \t 는 탭(→)으로
치환됨 )
```

소스를 수정하지 않고 회피하려면, 콘솔로만 출력하는 로컬 전용 로그 설정 파일을 만들어 ○ 6) 에서 `--logging.config` 옵션으로 지정합니다.

```
# Windows (PowerShell) — Set-Content 는 한 줄입니다. 그대로 붙여넣으십시오.
```

```
$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
Set-Content -Path "$Work\logback-local.xml" -Encoding utf8 -Value '<configuration><appender
name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender"><encoder><pattern>%-
5level %d{HH:mm:ss.SSS} [%thread] %logger{36} - %msg%n</pattern></encoder></appender><root
level="INFO"><appender-ref ref="CONSOLE"/></root></configuration>'
Get-Content "$Work\logback-local.xml"
# macOS / Linux — echo 는 한 줄입니다. 그대로 붙여넣으십시오.
WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
echo '<configuration><appender name="CONSOLE"
class="ch.qos.logback.core.ConsoleAppender"><encoder><pattern>%-5level %d{HH:mm:ss.SSS}
[%thread] %logger{36} - %msg%n</pattern></encoder></appender><root level="INFO"><appender-
ref ref="CONSOLE"/></root></configuration>' > $WORK/logback-local.xml
cat $WORK/logback-local.xml
```

`logback-local.xml` 의 내용이 그대로 출력되면 정상입니다. 이 파일은 ○ 6) 에서 7 종 모두에 지정합니다.

> ※ 이 파일은 로컬 확인용으로 콘솔에만 로그를 출력합니다. 소스의 `logback.xml` 은 그대로 두므로 서버 배포용 빌드에는 영향을 주지 않습니다.
>
> ※ 소스의 `logback.xml` 에서 경로를 `/hdd/DtEvtTracking/{서비스}` 형태의 슬래시로 바꾸면 폴더를 만드는 방식으로도 동작합니다. 다만 수정 내용이 서버 배포용 jar 에 그대로 반영되므로 로컬 확인 목적으로는 권장하지 않습니다.
>
> ※ `DETRCKDiscoveryService` 와 `DETRCKGatewayService` 는 별도의 로그 설정 파일이 없어 이 항목의 영향을 받지 않습니다. ○ 6) 에서 7 종에 같은 옵션을 함께 적용해도 문제되지 않습니다.

○ 5) 로컬 접속 설정 (소스 수정 없이)

각 모듈의 `application.properties` 는 운용 서버 주소를 기본값으로 가지고 있습니다. 그대로 실행하면 ○ 3) 에서 기동한 로컬 인프라가 아니라 운용 서버로 접속을 시도하여 기동에 실패합니다. 아래와 같이 주소 · 포트 · 계정이 모두 다르기 때문입니다.

```
덮어쓰는 항목                          로컬 값                          기본값(운용 서버)
spring.datasource.hikari.jdbc-url     localhost:30068                  221.133.48.18:21119
spring.datasource.hikari.username     fdtTwin                          dtEtTrck
spring.rabbitmq.host / port           localhost / 30071                221.133.48.18 / 5674
cors.allowed-origin-patterns          http://localhost:[*]
http://220.124.222.83:8088
```

○ 5-1) 환경 변수를 사용하는 이유

- Spring Boot 는 `SPRING_APPLICATION_JSON` 환경 변수를 `application.properties` 보다 우선 적용합니다. 따라서 소스를 수정하지 않고 실행 시점에만 로컬 값을 적용할 수 있습니다.
- 같은 항목이 트래킹 서비스 5 종에 반복 선언되어 있어 직접 수정하면 일부를 빠뜨리기 쉽습니다. 환경 변수는 한 번 지정으로 모든 모듈에 함께 적용됩니다.

- 소스가 그대로이므로 로컬 확인이 끝난 뒤 값을 되돌릴 필요가 없고, 서버 배포용 빌드에도 영향을 주지 않습니다. 환경 변수는 실행 시점에만 적용되며 jar 에 포함되지 않습니다.
- 운용 서버도 같은 원리로 동작합니다. `docker-compose.yml` 의 `env_file: .env.prod` 가 환경 변수로 `application.properties` 를 덮어씁니다. 본 항목은 그 방식을 로컬에 적용한 것입니다. (앞의「라」참고)

아래 명령을 그대로 입력합니다.

```
# Windows (PowerShell) — 한 줄입니다. 그대로 붙여넣으십시오.
$env:SPRING_APPLICATION_JSON = '{"spring":{"datasource":{"hikari":{"jdbc-
url":"jdbc:mariadb://localhost:30068/dataEventTrackings?autoCommit=false&useUnicode=true&cha
racterEncoding=utf8&useSSL=false&serverTimezone=UTC&autoReconnect=true","username":"fdtTwin"
,"password":"Fdt1234!@#$"}},"rabbitmq":{"host":"localhost","port":30071}},"cors":{"allowed-
origin-patterns":"http://localhost:[*]"}}'
# macOS / Linux — 한 줄입니다. 그대로 붙여넣으십시오.
export SPRING_APPLICATION_JSON='{"spring":{"datasource":{"hikari":{"jdbc-
url":"jdbc:mariadb://localhost:30068/dataEventTrackings?autoCommit=false&useUnicode=true&cha
racterEncoding=utf8&useSSL=false&serverTimezone=UTC&autoReconnect=true","username":"fdtTwin"
,"password":"Fdt1234!@#$"}},"rabbitmq":{"host":"localhost","port":30071}},"cors":{"allowed-
origin-patterns":"http://localhost:[*]"}}'
```

> ※ 이 환경 변수는 명령을 입력한 터미널 창에서만 유지되며, 그 창에서 기동한 서비스 창에 그대로 전달됩니다. 창을 새로 열었다면 이 줄을 다시 입력한 뒤 기동하십시오.
>
> ※ 소스를 직접 고치는 방식으로 진행하려면 앞의「라. 빌드 전 설정 확인」의 표를 따르십시오. 두 방식 중 하나만 수행하면 되며, 함께 수행할 필요는 없습니다.
>
> ※ `DETRCKDiscoveryService` 와 `DETRCKGatewayService` 는 데이터베이스를 사용하지 않으므로 위 값 중 일부만 적용됩니다. 모든 서비스에 같은 환경 변수를 적용해도 문제되지 않습니다.

○ 6) 백엔드 기동

앞의「바」의 기동 순서(Discovery → Gateway → 나머지)에 따라 7 종을 순서대로 기동합니다.

서비스마다 창이 따로 열려 로그를 개별적으로 확인할 수 있습니다.

```
# Windows (PowerShell)
$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
$Src = "$Work\BackEnd\src\DataEventTrackingService"
$LogUrl = "file:///" + ("$Work\logback-local.xml" -replace '\\','/')
$services =
'DETRCKDiscoveryService','DETRCKGatewayService','TrackingCommandReceiver','TrackingCommandPu
blisher','TrackingDataSubscriber','TrackingEventTrigger','TrackingCommandStatusChecker'
foreach ($s in $services) { Start-Process powershell -WorkingDirectory $Src -ArgumentList "-
NoExit","-Command","java -jar $s\target\$s-0.0.1-SNAPSHOT.jar --logging.config=$LogUrl";
Start-Sleep -Seconds 5 }
# macOS / Linux — 서비스마다 터미널 창을 따로 열어 실행합니다.
WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
cd $WORK/BackEnd/src/DataEventTrackingService
LOGOPT=--logging.config=file://$WORK/logback-local.xml


java -jar DETRCKDiscoveryService/target/DETRCKDiscoveryService-0.0.1-SNAPSHOT.jar $LOGOPT
java -jar DETRCKGatewayService/target/DETRCKGatewayService-0.0.1-SNAPSHOT.jar $LOGOPT
java -jar TrackingCommandReceiver/target/TrackingCommandReceiver-0.0.1-SNAPSHOT.jar $LOGOPT
```

```
java -jar TrackingCommandPublisher/target/TrackingCommandPublisher-0.0.1-SNAPSHOT.jar
$LOGOPT
java -jar TrackingDataSubscriber/target/TrackingDataSubscriber-0.0.1-SNAPSHOT.jar $LOGOPT
java -jar TrackingEventTrigger/target/TrackingEventTrigger-0.0.1-SNAPSHOT.jar $LOGOPT
java -jar TrackingCommandStatusChecker/target/TrackingCommandStatusChecker-0.0.1-
SNAPSHOT.jar $LOGOPT
```

※ 새로 열리는 창은 ○ 5) 에서 지정한 환경 변수를 그대로 물려받습니다. 따라서 ○ 5) 를 수행한 창에서 본 항목을 실행해야 합니다.

※ `Start-Process` 는 `Set-Location` 으로 옮긴 위치를 따라가지 않는 경우가 있어 `-WorkingDirectory` 로 경로를 명시합니다.

※ `--logging.config` 는 ○ 4) 에서 만든 로컬 로그 설정 파일을 지정하는 옵션입니다. 이 옵션을 빼면 트래킹 서비스 5 종이 로그 경로 오류로 기동되지 않습니다.

○ 7) 기동 확인

```
# 각 창에 아래 로그가 출력되면 기동 완료입니다.
Started {서비스명}Application in {n} seconds


# 포트 확인 — Windows (PowerShell)
Get-NetTCPConnection -State Listen -LocalPort 8761,8000,22419,22429,22439,22449,22459


# 포트 확인 — macOS / Linux
lsof -nP -iTCP -sTCP:LISTEN | grep -E '8761|8000|2241|2242|2243|2244'


# 트래킹 API 응답 확인 — Windows (PowerShell)
Invoke-WebRequest http://localhost:22419/fdt/detrck/rcv/tracking-request-list -
UseBasicParsing | Select-Object StatusCode


# 트래킹 API 응답 확인 — macOS / Linux
curl http://localhost:22419/fdt/detrck/rcv/tracking-request-list
```

- 7 개 창 모두에서 `Started ...Application in {n} seconds` 가 출력되면 정상입니다.
- 포트 8761 · 8000 · 22419 · 22429 · 22439 · 22449 · 22459 가 열려 있으면 7 종이 모두 기동된 것입니다.
- 트래킹 이력이 없는 상태에서는 조회 결과가 비어 있거나 500 으로 응답할 수 있습니다. 응답이 돌아오면 연동 자체는 정상입니다.

※ Windows PowerShell 5.1 에서 `curl` 은 `Invoke-WebRequest` 의 별칭이어서 macOS · Linux 의 `curl` 과 출력 형태가 다릅니다. 위와 같이 운영체제별 명령을 구분하여 사용하십시오. `curl.exe` 로 입력하면 Windows 에서도 동일한 형태로 출력됩니다.

※ 특정 창에서 `APPLICATION FAILED TO START` 가 출력되면 ○ 4) 의 로컬 로그 설정 파일 생성과 ○ 6) 의 `--logging.config` 지정 여부, ○ 3) 의 인프라 기동 상태를 확인하십시오. 데이터베이스 · 메시지 큐 접속 실패인 경우 ○ 5) 의 환경 변수가 해당 창에 적용되었는지 확인합니다.

○ 8) 종료 및 정리

```
# 백엔드 종료 — 각 창에서 Ctrl + C, 또는 아래 명령


# Windows (PowerShell)
Get-Process java -ErrorAction SilentlyContinue | Stop-Process


# macOS / Linux
pkill -f "DataEventTrackingService/.*target/.*SNAPSHOT.jar"


# 로컬 인프라 정리 — initResource/DataEventTrackingService 에서
docker compose -f docker-compose.local.yml stop     # 컨테이너 정지 (데이터 유지)
docker compose -f docker-compose.local.yml down     # 컨테이너 삭제 (데이터는 볼륨에 유지)
docker compose -f docker-compose.local.yml down -v  # 데이터까지 초기화
```

> ※ Windows 의 `Get-Process java | Stop-Process` 는 작업 PC 의 다른 Java 프로그램까지 종료시킵니다.
>
> 다른 Java 프로그램을 사용 중이라면 각 창에서 Ctrl + C 로 개별 종료하십시오.
>
> ※ `down -v` 는 볼륨까지 삭제하여 데이터베이스 내용이 모두 사라집니다. 다시 기동하면 `create_table.sql` 이 재실행되어 초기 상태가 됩니다.

○ 9) 서버 배포용 산출물 반영 (선택)

로컬 테스트에는 필요하지 않습니다. 6 장의 서버 배포를 이어서 수행하는 경우에만, 빌드한 jar 7 개를 배포 폴더로 복사합니다. 이 폴더가 6 장에서 서버로 이관되며 백엔드 이미지 생성에 사용됩니다.

```
# Windows (PowerShell)
$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
Set-Location $Work
$jars = "BackEnd\src\DataEventTrackingService\*\target\*-0.0.1-SNAPSHOT.jar"
Get-ChildItem $jars | Copy-Item -Destination "initResource\DataEventTrackingService\"
Get-ChildItem "initResource\DataEventTrackingService\*.jar" | Format-Table Name, Length,
LastWriteTime
# macOS / Linux
WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
cd $WORK
cp BackEnd/src/DataEventTrackingService/*/target/*-0.0.1-SNAPSHOT.jar
initResource/DataEventTrackingService/
ls -l initResource/DataEventTrackingService/*.jar
```

- jar 7 개의 수정 시각이 방금 빌드한 시각으로 갱신되었는지 확인합니다. 저장소에 포함되어 있던 이전 jar 의 시각이 그대로면 복사가 수행되지 않은 것입니다.

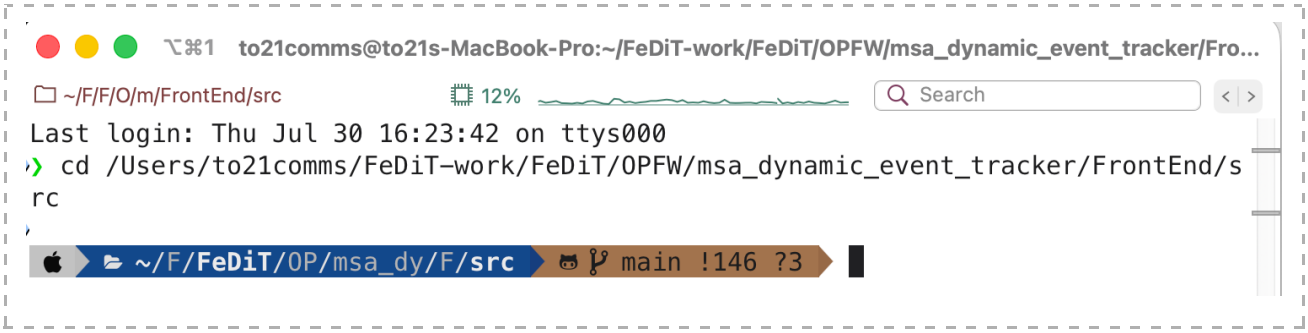
5. 프론트엔드 빌드

> ⚠ **서버에 배포할 빌드라면 「라」를 먼저 수행하십시오**
>
> 대시보드가 호출할 서버 주소는 빌드 시점에 정적 파일 안에 박힙니다. 「라. 서버 URL 설정」을 건너뛰고 「마. 배포 빌드」를 수행하면, 화면은 정상적으로 뜨지만 이전 서버로 요청이 나가 데이터가 조회되지 않습니다.
>
> 수정 위치는 `FrontEnd/src/DataEventTrackingDashboard/src/config/trackingApi.ts` 한 곳입니다.
>
> 로컬에서 화면만 확인하는 경우에는 「가」 ~ 「다」 만 수행하면 됩니다.

가. 프로젝트 경로 이동

Visual Studio Code 를 실행한 후 [파일(File)] – [폴더 열기(Open Folder…)] 로 Clone 한 폴더를 엽니다. 이후 터미널([터미널] – [새 터미널])에서 프론트엔드 소스 경로로 이동합니다.

> ※ 개발 도구 없이 명령행만으로 수행하는 절차는 본 장 「사. 명령행(CLI)으로 수행하기」에 한 곳으로 정리되어 있습니다.



[그림 14] 프론트엔드 프로젝트 경로 이동

나. 패키지 설치

`package.json` 에 정의된 의존 패키지를 설치합니다.

> ※ 설치 명령은 본 장 「사」 ○ 2) 패키지 설치 에 있습니다.

- 설치가 완료되면 프로젝트 하위에 `node_modules` 폴더가 생성됩니다.
- 설치 완료 후 출력되는 취약점(vulnerabilities) 경고는 빌드 및 구동에 영향을 주지 않으므로, 필요 시 별도로 조치합니다.



[그림 15] 패키지 설치(npm install)

다. 로컬 구동 확인

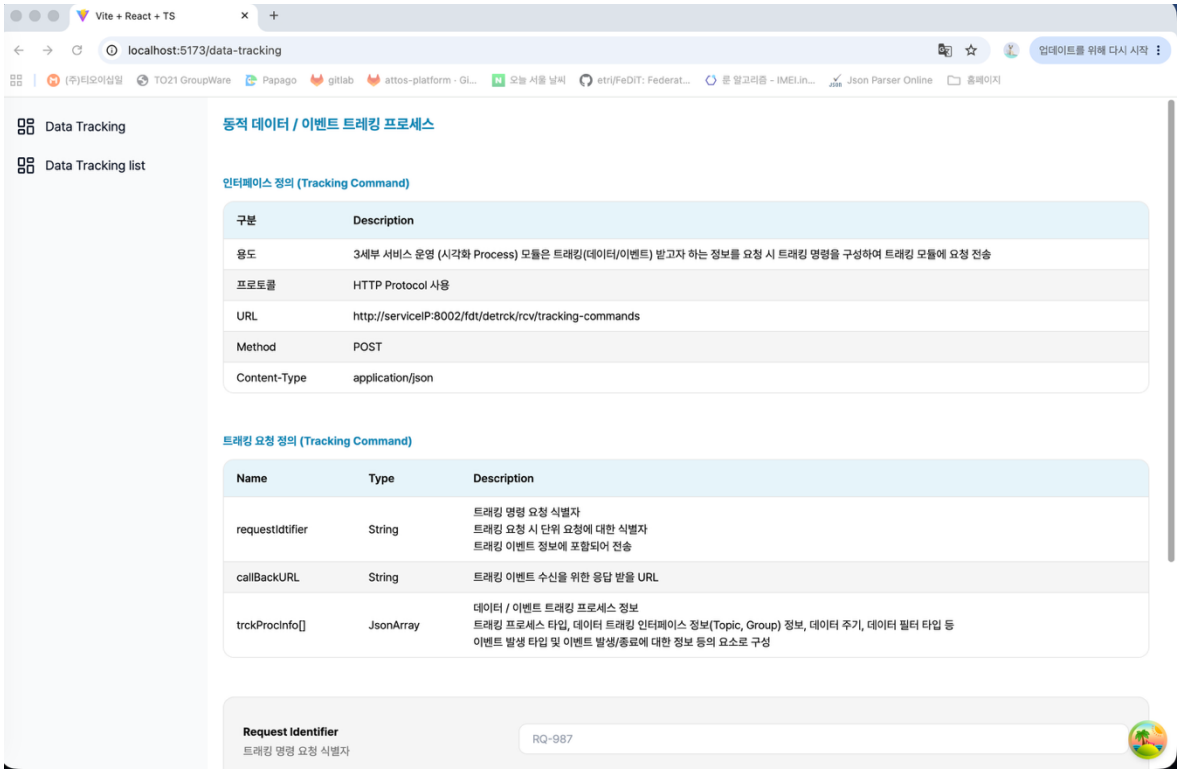
서버에 배포하기 전 로컬 환경에서 화면이 정상 동작하는지 확인합니다.

> ※ 구동 명령은 본 장「사」○ 3) 로컬 구동 확인 에 있습니다.

- 구동이 완료되면 접속 주소(`Local : http://localhost:5173/`)가 출력됩니다.
- 브라우저로 접속한 후 좌측 메뉴에서 `[Data Tracking]` 을 선택하면 트래킹 화면으로 이동합니다.
- 구동을 중지할 경우 터미널에서 `Ctrl + C` 를 입력합니다.



[그림 16] 로컬 구동/실행(npm run dev)



[그림 17] 로컬 구동 화면 (http://localhost:5173/data-tracking)

> ※ 로컬 환경에서는 연계 백엔드 서버가 구동되어 있지 않으면 화면만 표시되고 데이터가 조회되지 않을 수 있습니다. 이는 정상 동작이며, 화면 구성과 메뉴 이동이 정상적으로 동작하는지만 확인하면 됩니다.

라. 서버 URL 설정 (빌드 전 필수)

대시보드가 연계하는 서버 주소는 **빌드 시점에 정적 파일 안에 포함**됩니다. 따라서 배포 대상 서버가 변경되는 경우, 반드시 아래 두 가지를 먼저 수정한 후 빌드를 수행해야 합니다. 수정 없이 빌드하면 화면은 정상적으로 표시되지만 이전 서버로 요청이 전송되어 데이터가 조회되지 않습니다.

1) 환경 설정 파일(`.env.production`) 수정

```
FrontEnd/src/DataEventTrackingDashboard/.env.production
```

변수명	설명	기본값
VITE_API_BASE_URL	현재 소스에서 사용하지 않습니다. (변경 불필요)	`/api`
VITE_GRAFANA_URL	Grafana 대시보드 화면 주소. 트래킹 기능과는 무관합니다.	`http://221.133.48.18/grafana`
VITE_NEO4J_URI	Neo4j 그래프 화면 접속 주소. 트래킹 기능과는 무관합니다.	`bolt://221.133.48.18:7687`
VITE_NEO4J_USER / PASSWORD	Neo4j 접속 계정	`neo4j` / `test1234`

> ※ 이 파일은 배포 빌드(`npm run build`) 에만 적용됩니다. `npm run dev` 는 `.env.production` 을 읽지 않으므로 로컬 구동에는 영향이 없습니다. (7 장「자」8) 참고)
>
> ※ 트래킹 서버 주소는 이 파일이 아니라 아래 2)의 `src/config/trackingApi.ts` 에서 지정합니다. 트래킹 기능만 사용하는 경우 이 표의 항목은 수정하지 않아도 됩니다.

2) 데이터/이벤트 트래킹 서버 주소 수정

트래킹 서버 주소는 `src/config/trackingApi.ts` 한 곳에 모여 있습니다. 배포 대상 서버가 변경되면 이 파일의 주소 한 줄만 수정하면 트래킹 화면 전체에 반영됩니다.

사용할 주소의 주석(`//`)을 해제하고, 사용하지 않는 주소는 주석 처리합니다.

```
// FrontEnd/src/DataEventTrackingDashboard/src/config/trackingApi.ts
```

```
// 운용 서버 (Nginx 경유)
const TARGET: Target = { baseUrl: 'http://{serverIp}:8088', contextPath:
'/dataeventtrackingservice' }

// 로컬 – 개발 서버 경유 (npm run dev)
// const TARGET: Target = { baseUrl: 'http://localhost:5173', contextPath:
'/dataeventtrackingservice' }

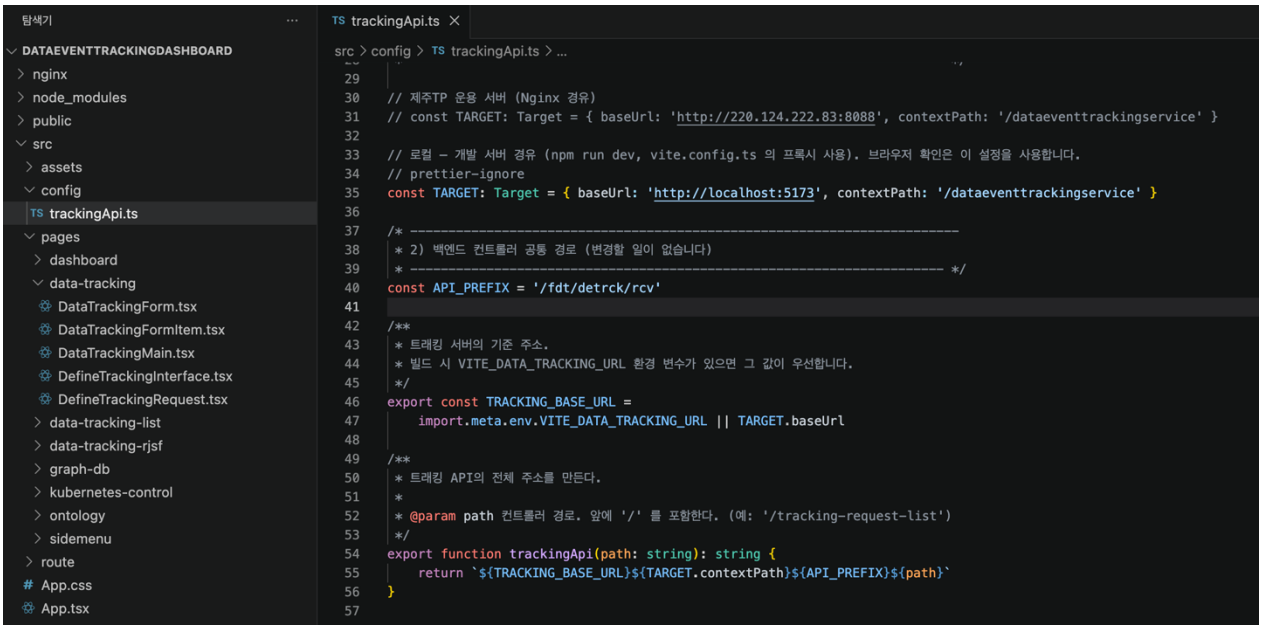
// 로컬 – 백엔드 직접 호출 (내부 테스트용, 브라우저에서는 CORS 차단)
// const TARGET: Target = { baseUrl: 'http://localhost:22419', contextPath: '' }
```

이 파일이 만들어 주는 최종 호출 주소는 아래와 같습니다.

```
http://{serverIp}:8088/dataeventtrackingservice/fdt/detrck/rcv/tracking-request-list
```

아래 화면들이 모두 이 설정을 함께 사용하므로, 개별 화면 소스를 수정할 필요가 없습니다.

No	이 설정을 사용하는 화면 (src/pages 하위)	호출 기능
1	data-tracking/DataTrackingForm.tsx	트래킹 등록
2	data-tracking-list/DataTrackingList.tsx	트래킹 목록 조회 · 전체 취소
3	data-tracking-list/DataTrackingDetailModal.tsx	상세 조회 · 실행 내역 조회
4	data-tracking-list/DataTrackingResultModal.tsx	개별 취소 · 전체 취소



[그림 18] trackingApi.ts – 트래킹 서버 주소 설정

> ※ 주소에는 포트까지 함께 입력합니다. 대시보드 화면과 트래킹 API 는 모두 Nginx 가 서비스하는 같은 포트(8088)로 제공되며, Nginx 가 `/dataeventtrackingservice/` 요청을 백엔드(22419)로 전달합니다. 따라서 이 주소에는 22419 가 아니라 8088 을 입력해야 합니다. (6 장「자」3) 참고)

※ 브라우저는 등록·취소(POST·DELETE)에는 같은 출처라도 `Origin` 헤더를 붙입니다. 백엔드에 허용 출처로 등록되어 있지 않으면 조회(GET)는 되는데 등록·취소만 `403` 으로 막히므로 원인을 찾기 어렵습니다. 따라서 대시보드를 제공하는 주소(`http://{serverIp}:8088`)를 허용 출처에 반드시 등록해야 합니다.

※ 허용 출처는 `initResource/DataEventTrackingService/.env.prod` 의 `cors_allowed_origin_patterns` 로 지정합니다. 서버 IP 가 바뀌면 이 값도 반드시 함께 바꾸어야 합니다. (6 장「마」·7 장「마」 참고)

※ 이전 버전에서는 화면 소스 4 개에 주소가 각각 선언되어 있어 모두 수정해야 했으나, 현재는 `src/config/trackingApi.ts` 한 곳만 수정하면 됩니다. 수정 대상을 확인하려면 VS Code 에서 전체 검색(macOS `Command + Shift + F` / Windows `Ctrl + Shift + F`)으로 `const TARGET` 을 검색합니다.

※ 로컬에서 구동하는 경우의 설정은 7 장「자」 8)을 참고하십시오.

※ 연합트윈 **메타 정보 조회 주소**는 환경 변수를 사용하지 않고 소스에 직접 입력되어 있습니다. 해당 서버가 변경되는 경우 `src/pages/data-tracking/DataTrackingFormItem.tsx` 의 **100, 106, 130 번 라인**에 있는 메타 정보 서버 주소를 함께 수정해야 합니다.

마. 배포 빌드

서버 URL 설정이 완료되면 배포용 정적 산출물을 생성합니다.

※ 빌드 명령은 본 장「사」 ○ 4) 배포 빌드 에 있습니다.

- 타입 검사(`tsc -b`) 수행 후 `vite build` 가 실행되며, 정상 완료 시 프로젝트 하위에 `dist` 폴더가 생성됩니다.
- 빌드가 정상 완료되면 생성된 파일 목록과 `built in ooos` 메시지가 출력됩니다.
- `Some chunks are larger than 500 kB` 메시지는 번들 크기에 대한 안내 경고로서 배포에는 영향을 주지 않습니다.
- 타입 오류로 빌드가 중단된 경우 오류가 표시된 소스를 수정한 후 다시 빌드를 수행합니다.



[그림 19] 배포 빌드(npm run build)

※ 서버 URL 을 수정한 뒤에는 **반드시 다시 빌드해야** 변경 사항이 반영됩니다. 빌드하지 않고 기존 `dist` 를 업로드하면 이전 주소가 그대로 적용됩니다.

○ 빌드 산출물을 운용 폴더에 반영

빌드한 dist 를 initResource/frontend/ 로 복사합니다. 이 폴더가 6 장「다」에서 서버로 이관되어 그대로 서비스되므로, 이 단계를 수행해야 최신 화면이 배포됩니다.

```
# Windows (PowerShell) — 저장소 최상위에서
Remove-Item -Recurse -Force .\initResource\frontend\dist
Copy-Item -Recurse .\FrontEnd\src\DataEventTrackingDashboard\dist .\initResource\frontend\


# macOS / Linux — 저장소 최상위에서
rm -rf initResource/frontend/dist
cp -r FrontEnd/src/DataEventTrackingDashboard/dist initResource/frontend/
```

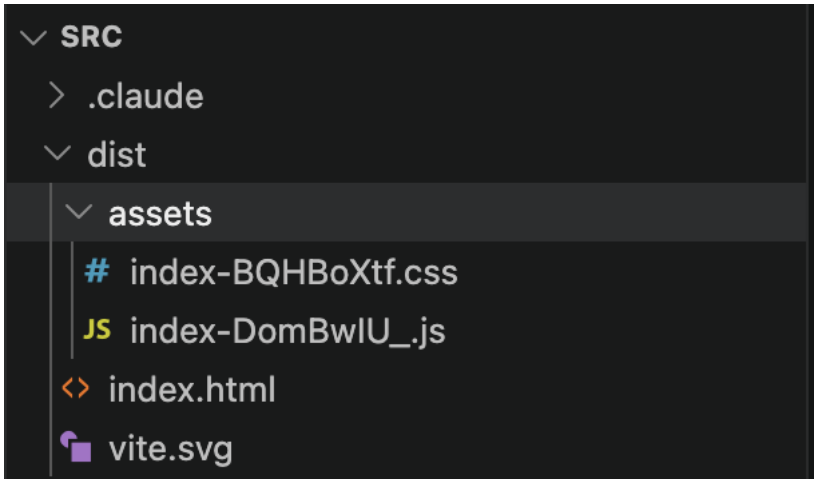
- 복사 후 initResource/frontend/dist 에 index.html 과 assets 폴더가 있으면 정상입니다.

> ※ 기존 dist 를 삭제한 후 복사합니다. assets 파일명에는 빌드할 때마다 바뀌는 해시가 붙으므로, 덮어쓰기만 하면 이전 빌드의 파일이 지워지지 않고 누적됩니다.
>
> ※ 백엔드에서 jar 를 initResource/DataEventTrackingService/ 로 복사하는 것과 같은 성격의 단계입니다. 이 복사를 누락하면 저장소에 포함된 이전 빌드본이 그대로 서버로 이관되어, 화면은 표시되지만 이전 서버 주소로 데이터를 조회하게 됩니다.

바. 빌드 산출물 확인

```
FrontEnd/src/DataEventTrackingDashboard/dist/
 ├─ index.html                    ← 진입점 HTML
 ├─ vite.svg                      ← public 폴더의 정적 리소스
 └─ assets/
     ├─ index-{hash}.css          ← 스타일시트 번들
     └─ index-{hash}.js           ← 애플리케이션 번들
```

구분	경로	설명
배포 단위	FrontEnd/src/DataEventTrackingDashboard/dist	서버에 업로드하는 폴더 (폴더 전체가 배포 대상)
HTML	dist/index.html	브라우저가 최초로 내려받는 진입점 파일
CSS	dist/assets/index-{hash}.css	Tailwind CSS / daisyUI 스타일 번들
JS	dist/assets/index-{hash}.js	React 애플리케이션 번들
정적 파일	dist/vite.svg	public 폴더의 파일이 그대로 복사됨



[그림 20] 빌드 산출물(dist) 확인

> ※ {hash} 값은 빌드할 때마다 변경되므로, 서버에 기존 assets 파일이 남지 않도록 기존 dist 폴더를 백업(이름 변경)한 후 업로드하는 것을 권장합니다. (화면 갱신 배포 — 7 장「다」참고)
>
> ※ 산출물 확인 및 화면 확인 명령은 본 장「사」○ 5)·○ 6) 에 있습니다.

사. 명령행(CLI)으로 수행하기

본 절은 소스 내려받기부터 패키지 설치 · 구동 · 빌드 · 화면 확인까지를 개발 도구(Visual Studio Code) 없이 명령행만으로 수행하는 절차입니다. 소스를 수정하지 않고 복사 · 붙여넣기만으로 로컬 테스트를 마칠 수 있도록 구성하였습니다. 각 단계의 배경 설명은 앞의「가」~「바」에 있습니다.

```
1) 사전 확인 · 소스 내려받기
2) 패키지 설치
3) 로컬 구동 및 화면 확인       → 백엔드 연동까지 확인
4) 배포 빌드
5) 산출물 확인
6) 빌드 산출물 화면 확인
```

> ※ 본 절의 명령은 하나의 터미널 창에서 1)부터 순서대로 이어서 실행하는 것을 기준으로 작성되었습니다. 각 항목의 블록에는 작업 경로 지정 줄이 포함되어 있어, 창을 새로 열었거나 중간 항목부터 시작하는 경우에도 해당 블록만 붙여넣으면 동작합니다.
>
> ※ 아래 명령은 한 줄이 하나의 명령입니다. 복사해 붙여넣을 때 한 줄이 여러 줄로 나뉘면 오류가 발생하므로, 줄이 나뉘지 않도록 붙여넣으십시오.

○ 1) 사전 확인 및 소스 내려받기

```
# Windows (PowerShell)
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force


git --version
node -v
npm -v


git config --global core.longpaths true


New-Item -ItemType Directory -Force -Path C:\FeDiT-work | Out-Null
Set-Location C:\FeDiT-work
git clone --filter=blob:none --no-checkout --branch main https://github.com/etri/FeDiT.git
Set-Location C:\FeDiT-work\FeDiT
git sparse-checkout set --cone OPFW/msa_dynamic_event_tracker
git checkout


$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
Get-ChildItem $Work | Format-Table Mode, Name
# macOS / Linux
git --version
node -v
npm -v


mkdir -p ~/FeDiT-work
cd ~/FeDiT-work
git clone --filter=blob:none --no-checkout --branch main https://github.com/etri/FeDiT.git
cd FeDiT
git sparse-checkout set --cone OPFW/msa_dynamic_event_tracker
git checkout


WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
```

```
ls -l $WORK
```

- node 는 v20.19.0 이상 또는 v22.12.0 이상이어야 합니다. Node.js 18 에서는 빌드가 실패합니다. (2 장「가」)
- BackEnd · FrontEnd · initResource 폴더가 표시되면 정상입니다. (3 장「다」)

> ※ 이미 소스를 내려받은 상태라면 Clone 줄만 건너뛰고 나머지를 실행하십시오. 소스를 새로 내려받으려면 `Remove-Item -Recurse -Force C:\FeDiT-work\FeDiT` 로 기존 폴더를 삭제한 뒤 실행합니다.

○ 2) 패키지 설치

```
# Windows (PowerShell)
$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
Set-Location "$Work\FrontEnd\src\DataEventTrackingDashboard"
npm install
# macOS / Linux
WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
cd $WORK/FrontEnd/src/DataEventTrackingDashboard
npm install
```

- node_modules 폴더가 생성되고 added ∞ packages 가 출력되면 완료입니다.

> ※ npm 실행 시「이 시스템에서 스크립트를 실행할 수 없으므로 ...npm.ps1 파일을 로드할 수 없습니다」 오류가 발생하면 2 장「나」의「○ PowerShell 스크립트 실행 정책 설정」을 수행한 뒤 다시 실행하십시오. 정책을 변경할 수 없는 환경에서는 npm 대신 npm.cmd 를 사용하면 됩니다.
>
> ※ 설치 완료 후 출력되는 취약점(vulnerabilities) 경고는 빌드 및 구동에 영향을 주지 않습니다.

○ 3) 로컬 구동 및 화면 확인

개발 서버를 기동하여 브라우저에서 화면을 확인합니다. 트래킹 서버 주소의 기본값은 src/config/trackingApi.ts 에 운용 서버로 지정되어 있으나, 같은 파일에 환경 변수를 우선 적용하는 처리가 이미 들어 있어 소스를 수정하지 않고 로컬 주소로 바꿀 수 있습니다.

```
// src/config/trackingApi.ts — 환경 변수가 있으면 그 값이 우선합니다.
export const TRACKING_BASE_URL =
    import.meta.env.VITE_DATA_TRACKING_URL || TARGET.baseUrl
```

- VITE_DATA_TRACKING_URL 을 지정하면 TARGET.baseUrl 대신 그 값이 사용됩니다.「라」2)처럼 주석을 바꿔 가며 소스를 고칠 필요가 없습니다.
- 값을 http://localhost:5173 으로 두는 이유는, 화면과 API 요청의 출처를 같게 만들어 CORS 없이 개발 서버의 프록시를 거치도록 하기 위한 것입니다. vite.config.ts 의 프록시가 /dataeventtrackingservice 요청을 로컬 백엔드(22419)로 전달합니다.
- Vite 는 VITE_ 로 시작하는 환경 변수만 화면 소스에 전달합니다.

아래 명령을 그대로 입력합니다.

```
# Windows (PowerShell)
$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
Set-Location "$Work\FrontEnd\src\DataEventTrackingDashboard"
$env:VITE_DATA_TRACKING_URL = "http://localhost:5173"
npm run dev        # http://localhost:5173/ — 종료는 Ctrl + C
# macOS / Linux
WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
cd $WORK/FrontEnd/src/DataEventTrackingDashboard
export VITE_DATA_TRACKING_URL=http://localhost:5173
npm run dev        # http://localhost:5173/ — 종료는 Ctrl + C
```

- 브라우저에서 `http://localhost:5173/` 에 접속한 뒤 좌측 메뉴 [Data Tracking] 으로 이동되면 화면 구동은 정상입니다.
- 4 장「아」로 백엔드를 기동해 둔 상태라면 트래킹 등록 · 목록 조회 · 취소까지 동작합니다.

> ※ 백엔드를 기동하지 않은 상태에서는 화면만 표시되고 데이터가 조회되지 않습니다. 이는 정상이며, 화면 구성과 메뉴 이동만 확인하면 됩니다. (앞의「다」참고)
>
> ※ 데이터 조회까지 확인하려면 4 장「아」○ 3) 로컬 인프라 기동과 ○ 6) 백엔드 기동을 먼저 수행하십시오.
>
> ※ 환경 변수를 지정하지 않으면 `trackingApi.ts` 의 기본값(운용 서버)으로 요청이 전송됩니다. 소스를 직접 고치는 방식은 앞의「라」2)를 참고하십시오.

○ 4) 배포 빌드

```
# Windows (PowerShell)
$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
Set-Location "$Work\FrontEnd\src\DataEventTrackingDashboard"
Remove-Item Env:VITE_DATA_TRACKING_URL -ErrorAction SilentlyContinue
npm run build
# macOS / Linux
WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
cd $WORK/FrontEnd/src/DataEventTrackingDashboard
unset VITE_DATA_TRACKING_URL
npm run build
```

- 타입 검사(`tsc -b`) 통과 후 `vite build` 가 수행되며, `built in oos` 가 출력되면 완료입니다.
- `Some chunks are larger than 500 kB` 경고는 배포에 영향을 주지 않습니다.

> ※ 첫 줄에서 `VITE_DATA_TRACKING_URL` 을 해제하는 이유는, ○ 3) 에서 지정한 로컬 주소가 배포 빌드에 그대로 포함되는 것을 막기 위한 것입니다. 이 값이 남아 있으면 서버에 올린 화면이 `localhost` 로 요청을 보내게 됩니다.

○ 5) 산출물 확인

```
# Windows (PowerShell)
$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
Set-Location "$Work\FrontEnd\src\DataEventTrackingDashboard"
Get-ChildItem -Recurse dist | Format-Table Name, Length, LastWriteTime
# macOS / Linux
```

```
WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
cd $WORK/FrontEnd/src/DataEventTrackingDashboard
ls -lR dist
```

- `dist/index.html`, `dist/vite.svg`, `dist/assets/index-{hash}.js`, `dist/assets/index-{hash}.css`

  가 생성되어 있으면 완료입니다.

○ 6) 빌드 산출물 화면 확인

빌드된 `dist` 를 그대로 서비스하여 배포 전에 실제 화면을 확인합니다.

```
# Windows (PowerShell)
$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
Set-Location "$Work\FrontEnd\src\DataEventTrackingDashboard"
npm run preview    # http://localhost:4173/ — 종료는 Ctrl + C
# macOS / Linux
WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
cd $WORK/FrontEnd/src/DataEventTrackingDashboard
npm run preview    # http://localhost:4173/ — 종료는 Ctrl + C
```

> ※ `dist/index.html` 을 브라우저로 직접 여는(file:// 경로) 방식으로는 화면이 표시되지 않습니다. 자원 경로가 절대 경로로 생성되기 때문이며, 반드시 위와 같이 웹 서버를 통해 확인해야 합니다.
>
> ※ `npm run preview` 에는 개발 서버의 프록시가 적용되지 않습니다. 따라서 화면 구성만 확인할 수 있으며, 데이터 조회까지 확인하려면 ○ 3) 의 개발 서버를 사용하십시오.
>
> ※ `npm run preview` 는 확인용 서버입니다. 운용 환경에서는 6 장「자」와 같이 Nginx 가 `dist` 를 서비스합니다.

6. 서버 배포

가. 이 장에서 수행하는 작업

본 장은 4 장·5 장에서 생성한 산출물을 운용 서버에 배포하고 서비스를 기동하는 절차입니다.

백엔드와 프론트엔드의 **공통 배포 작업**을 다룹니다.

순서	작업	해당 절
1	서버에 접속하고 Docker 등 실행 환경을 준비합니다.	나
2	운용 폴더를 서버로 옮깁니다.	다
3	데이터가 저장될 폴더를 서버에 만듭니다.	라
4	설정 파일을 운용 환경에 맞게 확인하고 수정합니다.	마
5	컨테이너 간 통신을 위한 네트워크를 만듭니다.	바

순서	작업	해당 절
6	백엔드 이미지를 생성합니다.	사
7	백엔드 서비스를 기동합니다.	아
8	웹 서버(Nginx)를 기동합니다. (화면 파일은 2 번 운용 폴더 이관에 포함되어 있습니다.)	자
9	정상 기동 여부를 확인합니다.	차

> ※ 각 절의 순서는 서로 의존 관계가 있으므로 반드시 위 순서대로 수행해야 합니다. 예를 들어 네트워크를 만들지 않고 컨테이너를 기동하면 오류가 발생합니다.

> ⚠ **서버 IP 가 바뀌었다면 아래 두 곳을 먼저 확인하십시오**
>
> ① `initResource/DataEventTrackingService/.env.prod` 의 `cors_allowed_origin_patterns` → `http://{serverIp}:8088` (6 장「마」). 빠뜨리면 조회는 되는데 트래킹 등록·취소만 403 으로 실패합니다.
>
> ② 대시보드 `dist` 가 새 서버 주소로 빌드된 것인지 (5 장「라」). 이전 주소로 빌드된 `dist` 를 올리면 화면만 뜨고 데이터가 조회되지 않습니다.

○ 서버에서 구동되는 구성 요소

구분	컨테이너 명	이미지	역할
대시보드	`nginx-proxy`	`nginx:latest`	화면(dist) 전달 및 요청 프록시
백엔드	`dataeventtrackingservice`	`msa-detrck`	데이터/이벤트 트래킹 서비스 7 종 (단일 컨테이너에서 동시 구동)
메시지 큐	`rabbitmq_detrck`	`rabbitmq:3.7.14-management-alpine`	트래킹 명령 / 이벤트 메시지 중계
데이터베이스	`mariadb`	`mariadb:10`	트래킹 이력 및 결과 데이터 저장

나. 서버 접속 및 사전 준비

본 절은 **아무것도 설치되어 있지 않고 SSH 포트만 열려 있는 Rocky Linux 9 서버**를 기준으로 합니다. 아래 명령어를 순서대로 복사하여 붙여 넣으면 배포에 필요한 실행 환경이 갖추어집니다.

> ※ 이 시점부터 6 장의 모든 명령어는 **서버에 접속한 상태**에서 입력합니다. 접속을 종료하려면 `exit` 를 입력합니다.

> ※ Ubuntu / Debian 계열 서버를 사용하는 경우 「4) Docker 설치」와 「6) SELinux 설정」만 해당 배포판의 절차로 대체하고 나머지는 동일합니다. (Ubuntu 는 SELinux 를 사용하지 않으므로 6)은 생략합니다)

○ 1) 서버 접속

1. 작업 PC 의 터미널 또는 SSH 접속 도구를 실행합니다.

- macOS / Linux : 터미널
- Windows : PuTTY, MobaXterm 등

2. 아래 명령어를 입력하여 운용 서버에 접속합니다.

```
ssh {계정}@{serverIp} -p {sshPort}
```

- 최초 접속 시 접속을 계속할지 묻는 메시지가 표시되면 yes 를 입력합니다.
- 이어서 비밀번호 입력을 요구하면 계정 비밀번호를 입력합니다. (입력하는 문자는 화면에 표시되지 않습니다)

3. 접속이 완료되면 아래 명령어로 정상 접속 여부와 운영체제를 확인합니다.

```
hostname                  # 서버의 이름이 출력됩니다.
whoami                    # 현재 접속한 계정이 출력됩니다.
pwd                       # 현재 위치한 경로가 출력됩니다.
cat /etc/os-release | grep -E '^(NAME|VERSION)='
```

- NAME="Rocky Linux", VERSION="9.x" 로 출력되면 본 절의 명령어를 그대로 사용합니다.

○ 2) 시스템 사전 점검

설치를 시작하기 전에 서버의 자원과 디스크 구성을 확인합니다.

```
uname -m                                  # x86_64 로 출력되어야 합니다
free -h                                   # 메모리
df -h /                                   # 루트 디스크 여유 공간
df -h /hdd 2>/dev/null || echo "※ /hdd 별도 파티션 없음 – 루트 디스크를 사용하게 됩니다"
lsblk                                     # 디스크 및 마운트 구성
```

아래 값은 **본 시스템만 단독으로 구동할 때의 참고 기준**입니다. 서버의 실제 사양과 함께 구동되는 다른 시스템에 따라 달라지므로, 절대적인 요구 사항이 아니라 도입 검토 시의 가늠자로 활용하시기 바랍니다.

항목	참고 기준	산정 근거
CPU	2 코어 이상	서비스 7 종 + MariaDB + RabbitMQ 동시 구동
메모리	4GB 이상	백엔드 컨테이너 하나에서 **Java 프로세스 7 개가 동시 구동**(extStartUp.sh, 각 최대 128MB) + MariaDB·RabbitMQ

항목	참고 기준	산정 근거
루트 디스크 여유	20GB 이상	컨테이너 이미지 4 종 + Docker 작업 공간
데이터 경로(/hdd)	운용 기간에 따라 증설	트래킹 이력·로그가 계속 누적됨

> ※ 위 값보다 낮은 사양에서도 기동은 되지만, 메모리가 부족하면 기동 도중 일부 서비스가 종료될 수 있습니다.
>
> ※ 실제 소요량은 서비스를 기동한 뒤 docker stats --no-stream 으로 확인하여 조정하는 것이 가장 정확합니다. (6 장「차」이후 수행)
>
> ※ 6 장「라」의 `create_directories.sh` 는 `/hdd` 아래에 데이터 폴더를 만듭니다. `/hdd` 가 **별도 디스크로 마운트되어 있지 않으면 루트 디스크에 그대로 쌓이므로**, 트래킹 이력이 늘어나면 루트 파티션이 가득 찰 수 있습니다. 별도 디스크를 사용하는 경우 이 시점에 마운트를 완료해 두십시오.

○ 3) 시각 동기화

```
sudo dnf install -y chrony
sudo systemctl enable --now chronyd
sudo timedatectl set-timezone Asia/Seoul


timedatectl                               # Time zone: Asia/Seoul, System clock synchronized:
yes
chronyc sources -v | head -5
```

> ※ MariaDB 컨테이너는 `TZ: Asia/Seoul` 로, 백엔드 서비스 7 종은 `-Duser.timezone=Asia/Seoul` 로 기동됩니다. 서버 시각이 어긋나면 트래킹 이력에 기록되는 시각이 실제와 달라집니다.

○ 4) Docker 설치

Rocky 9 의 기본 저장소에는 `docker-ce` 패키지가 없으므로 공식 저장소를 먼저 등록해야 합니다.

```
# (1) 충돌 패키지 확인 — podman-docker 가 있으면 docker 명령이 podman 으로 연결됩니다
rpm -q podman-docker && sudo dnf remove -y podman-docker


# (2) 구버전 docker 패키지 제거
sudo dnf remove -y docker docker-client docker-common docker-latest \
                   docker-logrotate docker-engine runc 2>/dev/null


# (3) Docker 공식 저장소 등록
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo


# (4) 설치
sudo dnf install -y docker-ce docker-ce-cli containerd.io \
                    docker-buildx-plugin docker-compose-plugin


# (5) 기동 및 부팅 시 자동 시작 등록
sudo systemctl enable --now docker
sudo systemctl status docker --no-pager | head -5
```

```
# (6) 버전 확인
docker --version                          # Docker version 2x.x.x
docker compose version                    # Docker Compose version v2.x.x
```

※ `docker-compose` (하이픈)가 아니라 `docker compose` (공백)로 출력되어야 합니다. 본 문서의 모든 명령어는 플러그인 방식(`docker compose`)을 기준으로 합니다.

※ (5)의 `enable` 을 누락하면 **서버를 재부팅했을 때 Docker 가 자동으로 시작되지 않아** 모든 컨테이너가 기동되지 않습니다.

※ (3)·(4)에는 인터넷 접속이 필요합니다. 폐쇄망 서버는 7 장 「바」를 참고하십시오.

○ 5) docker 그룹 등록

`docker` 명령을 `sudo` 없이 사용할 수 있도록 현재 계정을 `docker` 그룹에 추가합니다.

```
sudo usermod -aG docker $USER
exit                                      # 그룹 변경은 재접속해야 적용됩니다
```

재접속한 뒤 적용 여부를 확인합니다.

```
ssh {계정}@{serverIp} -p {sshPort}

id -nG | tr ' ' '\n' | grep -x docker     # docker 가 출력되면 정상
docker ps                                 # 오류 없이 표가 출력되면 정상
```

※ 이 단계를 건너뛰면 이후 절의 `docker` 명령이 `permission denied while trying to connect to the Docker daemon socket` 오류로 실패합니다. 본 문서의 명령어는 모두 `sudo` 없이 실행하는 것을 기준으로 작성되어 있습니다.

○ 6) SELinux 설정

Rocky 9 는 SELinux 가 **Enforcing** 상태로 기본 설정되어 있습니다.

```
getenforce                                # Enforcing · Permissive 이면 아래를 수행합니다

sudo dnf install -y policycoreutils-python-utils
sudo semanage fcontext -a -t container_file_t '/hdd(/.*)?'
```

※ Enforcing 이면 필수입니다. Disabled 는 SELinux 가 동작하지 않으므로 수행할 필요가 없습니다. Permissive 는 지금은 차단되지 않지만 이후 Enforcing 으로 전환하면 컨테이너가 /hdd 에 데이터를 쓰지 못하므로, 수행해 두는 것을 권장합니다. Ubuntu / Debian 계열은 SELinux 를 사용하지 않으므로 해당하지 않습니다.

※ 백엔드·RabbitMQ·MariaDB 컨테이너는 `/hdd` 아래 폴더를 컨테이너 내부에 연결(bind mount)하여 로그와 데이터를 저장합니다. SELinux 컨텍스트가 맞지 않으면 **컨테이너는 기동되지만 파일 쓰기가 거부**됩니다. RabbitMQ 와 MariaDB 가 데이터를 저장하지 못해 재시작을 반복하게 되며, 로그에는 권한 오류만 표시되어 원인을 찾기 어렵습니다.

※ 위 `semanage` 명령은 **규칙만 등록**합니다. 실제 적용은 폴더를 만든 뒤인 6 장「라」에서 `restorecon` 으로 수행합니다.

※ SELinux 를 끄는 방법(`setenforce 0`)은 서버 전체의 보안 설정을 낮추므로 권장하지 않습니다.

○ 7) 포트 선점 확인

이 시스템이 사용하는 포트를 다른 서비스가 이미 쓰고 있는지 확인합니다.

```
sudo ss -lntp | grep -E
':(8088|8763|8002|22419|22429|22439|22449|22459|5674|15674|25674|4371|5669|15669|1885|8885|6
1617|61618|21119) ' \
  || echo "선점된 포트 없음 – 계속 진행합니다"
```

※ 출력되는 포트가 있으면 그대로 진행할 경우 컨테이너 기동 시 `port is already allocated` 오류가 발생합니다. 기존 서비스를 중지하거나, 운용 폴더의 `docker-compose.yml` 에서 해당 포트의 **왼쪽(호스트) 번호만** 변경하십시오.

※ 연합트윈의 다른 모듈(Device API Gateway, 마이크로서비스 오케스트레이션)과 **같은 서버에 함께 설치해도 포트가 겹치지 않도록 설계**되어 있습니다. 다만 `monitoring` 네트워크 대역(172.20.0.0/16)이 다른 네트워크와 겹치지 않는지는 별도로 확인해야 합니다. 세 모듈을 함께 올리는 경우 메모리 소요도 합산되므로 2)의 참고 기준을 합산하여 보시기 바랍니다.

○ 8) 설치 검증

```
docker run --rm hello-world
```

- `Hello from Docker!` 가 출력되면 Docker 가 정상 동작하는 상태입니다.

※ 이 단계에서 이미지 내려받기가 실패하면 서버가 Docker Hub 에 접속하지 못하는 상태입니다. 폐쇄망 환경은 7 장「바」, 프록시 환경은 7 장「사」를 참고하십시오.

다. 운용 폴더 이관

1. `initResource.zip` 을 서버의 홈 디렉터리로 업로드합니다. (작업 PC 에서 입력)

```
cd C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker


scp -P {sshPort} ./initResource.zip {계정}@{serverIp}:~/
```

2. 서버에서 압축을 해제합니다. (서버에서 입력)

```
sudo dnf install -y unzip          # unzip 이 설치되어 있지 않은 경우에만
cd ~
unzip -q initResource.zip
rm -rf __MACOSX                    # macOS 에서 압축한 경우에만 생성됩니다
ls -al ~/initResource
```

- 압축을 풀면 홈 디렉터리에 `initResource` 폴더가 생성됩니다.

3. 서버의 운용 경로로 복사하고 폴더명을 변경합니다. (서버에서 입력)

```
sudo cp -r /home/jejutp/initResource /usr/local/dataeventtrackingservice
sudo chown -R $USER:$USER /usr/local/dataeventtrackingservice
sudo chmod -R 755 /usr/local/dataeventtrackingservice
```

※ /usr/local 은 root 소유이므로 일반 계정으로 직접 업로드하면 Permission denied 가 발생합니다. 반드시 홈 디렉터리로 업로드한 뒤 위와 같이 sudo cp -r 로 복사하십시오.

※ mv 가 아니라 cp -r 를 사용하므로 홈 디렉터리의 initResource 원본이 그대로 남습니다. 재이관이나 설정 파일 비교에 사용할 수 있으며, 필요 없으면 rm -rf ~/initResource 로 정리하십시오(약 350 MB). 또한 /usr/local/dataeventtrackingservice 가 이미 존재하면 그 안쪽으로 복사되므로, 재설치 시에는 기존 폴더를 먼저 이름 변경하거나 삭제하십시오.

※ zip 으로 전송하면 .env, .env.prod 처럼 이름이 점(.)으로 시작하는 숨김 파일도 압축 안에 포함되므로 누락될 염려가 없습니다. 폴더를 직접 전송할 때 발생하던 숨김 파일 누락 문제가 해소됩니다.

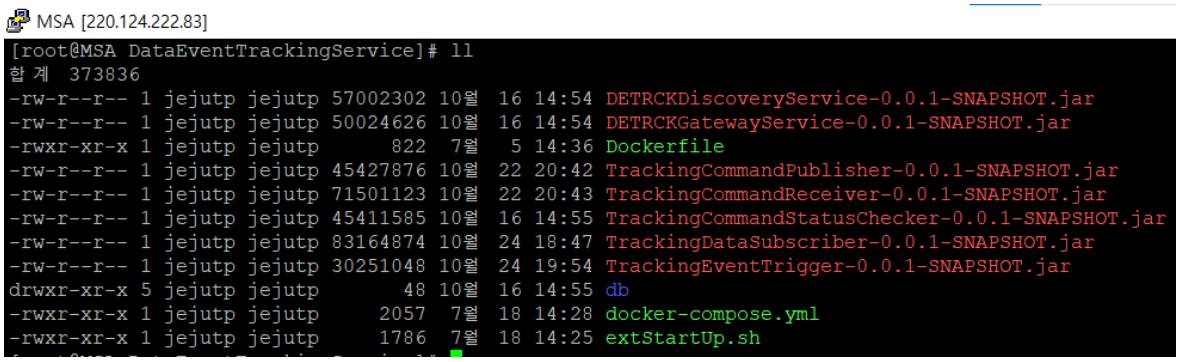
※ FileZilla 등 FTP 클라이언트를 사용하는 경우에도 initResource.zip 파일 하나만 홈 디렉터리로 업로드한 뒤, 위와 같이 서버에서 압축을 해제하십시오.

※ initResource.zip 은 약 356 MB 입니다. 회선 속도에 따라 업로드에 수 분이 소요될 수 있습니다.

4. 업로드 결과를 확인합니다. **(서버에서 입력)**

```
cd /usr/local/dataeventtrackingservice
ls -al
ls -al DataEventTrackingService/.env DataEventTrackingService/.env.prod    # 숨김 파일 존재 확인
```

- create_directories.sh, network-dash-create.yml, docker-compose_front.yml, nginx, frontend, DataEventTrackingService 가 모두 보이면 정상입니다.



[그림 21] 운용 폴더 이관 결과 확인 (ls -al)

5. 스크립트 파일에 실행 권한을 부여합니다.

```
chmod +x create_directories.sh
chmod +x DataEventTrackingService/extStartUp.sh
```

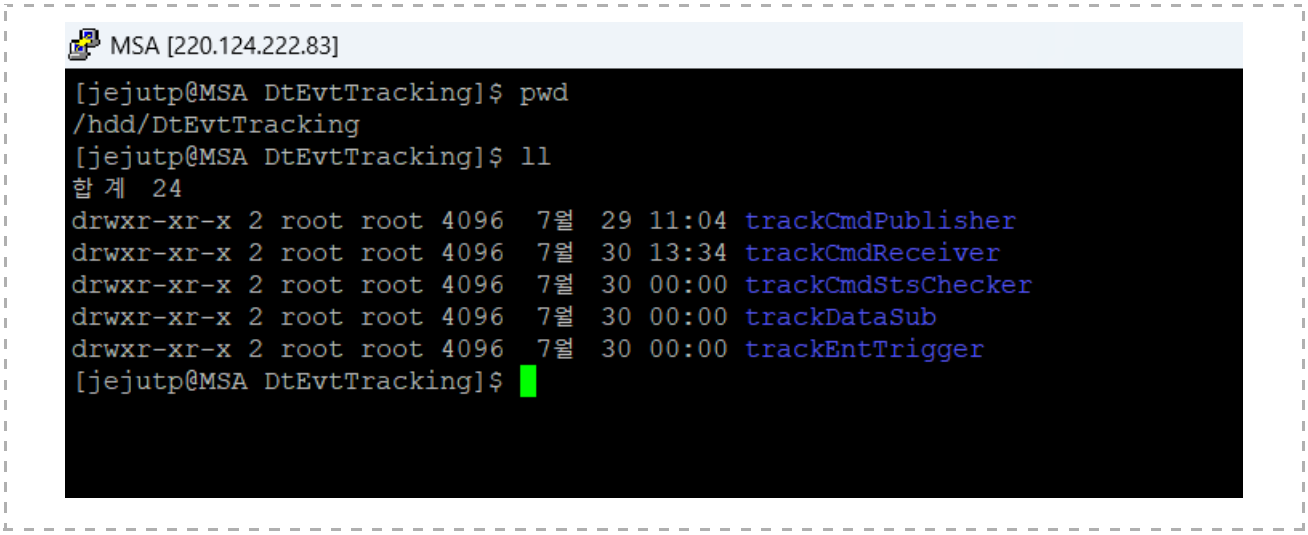
※ .env, .env.prod 및 db/initdb.d 하위 SQL 파일에는 데이터베이스 계정 정보가 포함되어 있습니다. 업로드 이후 권한을 제한하고 외부에 공유되지 않도록 관리하시기 바랍니다.

라. 데이터 저장 폴더 생성

컨테이너가 로그와 데이터를 저장할 서버의 폴더를 미리 만들어야 합니다. 폴더에 포함된 `create_directories.sh` 스크립트를 실행하면 한 번에 생성됩니다.

```
cd /usr/local/dataeventtrackingservice
./create_directories.sh
```

- 실행 시 sudo 권한이 필요하므로 비밀번호 입력을 요구할 수 있습니다.
- 정상 실행되면 생성되는 폴더가 한 줄씩 출력된 후 완료 메시지가 표시됩니다.



[그림 22] create_directories.sh 실행 결과

○ 생성되는 폴더와 용도

폴더	용도
/hdd/DtEvtTracking/trackCmdReceiver	트래킹 명령 수신 서비스 로그
/hdd/DtEvtTracking/trackCmdPublisher	트래킹 명령 발행 서비스 로그
/hdd/DtEvtTracking/trackDataSub	트래킹 데이터 수신 서비스 로그
/hdd/DtEvtTracking/trackEntTrigger	이벤트 트리거 서비스 로그
/hdd/DtEvtTracking/trackCmdStsChecker	명령 상태 점검 서비스 로그
/hdd/rabbitmq/data/detrck	RabbitMQ 데이터 저장 경로
/hdd/rabbitmq/logs/detrck	RabbitMQ 로그 저장 경로
/hdd/dataTest/mariadb3	MariaDB 데이터 저장 경로

```
ls -al /hdd/DtEvtTracking
ls -al /hdd/rabbitmq
ls -al /hdd/dataTest
```

> ※ `/hdd` 는 운용 서버의 데이터 저장용 디스크 경로입니다. 서버의 디스크 구성이 다른 경우 `create_directories.sh` 와 `DataEventTrackingService/docker-compose.yml` 의 경로를 함께 수정해야 합니다.

○ SELinux 컨텍스트 적용 (Rocky / RHEL 계열 필수)

6 장「나」6)에서 등록한 규칙을 지금 생성한 폴더에 실제로 적용합니다.

```
getenforce                                  # Disabled 이면 아래 restorecon 은 생략
sudo restorecon -Rv /hdd


ls -Z /hdd/DtEvtTracking                    # container_file_t 로 표시되면 정상
ls -Z /hdd/rabbitmq
ls -Z /hdd/dataTest
```

※ 이 단계를 건너뛰면 컨테이너는 기동되지만 RabbitMQ 와 MariaDB 가 데이터를 쓰지 못해 재시작을 반복합니다. 6 장「차」의 docker compose ps 에서 Restarting 상태가 반복되면 이 설정을 먼저 확인하십시오.

※ Enforcing 이면 restorecon 을 반드시 수행해야 합니다. Disabled 는 수행할 필요가 없습니다. Permissive 는 지금 수행하지 않아도 동작하지만, 이후 Enforcing 으로 전환하면 컨테이너가 데이터를 쓰지 못하므로 수행해 두는 것을 권장합니다. Ubuntu / Debian 계열은 해당하지 않습니다. SELinux 상태 확인과 규칙 등록 절차는 6 장「나」○ 6) SELinux 설정 을 참고하십시오.

마. 설정 파일 수정

파일	설정 항목	설명
DataEventTrackingService/.env	MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD, MYSQL_ROOT_PASSWORD	데이터베이스 컨테이너가 최초 기동될 때 생성하는 계정 정보
DataEventTrackingService/.env.prod	spring_datasource_hikari_jdbc_url, spring_rabbitmq_host, eureka_client_serviceUrl_defaultZone, cors_allowed_origin_patterns 등	백엔드 서비스가 접속하는 데이터베이스 / RabbitMQ / 서비스 등록 정보와, 대시보드 요청을 허용할 출처(CORS) 설정
DataEventTrackingService/db/conf.d/my.cnf	character-set-server, datadir	데이터베이스 문자셋(utf8) 및 데이터 저장 경로
DataEventTrackingService/db/initdb.d/	create_table.sql 등	최초 기동 시 1 회 실행 (테이블 생성)

nginx/nginx.conf	location	화면 파일 경로 및 요청 전달(프록시) 설정
docker-compose_front.yml	ports	대시보드 공개 포트 (8088:80)

> ※ cors_allowed_origin_patterns 는 대시보드 화면이 제공되는 주소를 그대로 적습니다. (예: http://{serverIp}:8088) 서버 IP 가 바뀌었는데 이 값을 그대로 두면, 화면과 조회는 정상인데 트래킹 등록·취소만 403 으로 실패합니다. 여러 주소를 허용할 때는 쉼표로 구분하고, 포트 전체를 허용할 때는 대괄호로 감쌉니다. (예: http://localhost:[*])

설정 파일을 수정한 후에는 해당 컨테이너를 재기동해야 변경 사항이 반영됩니다.

```
# 예) nginx.conf 수정 후
docker compose -f docker-compose_front.yml restart
```

> ※ .env 의 데이터베이스 계정 정보를 변경하는 경우, 백엔드가 접속에 사용하는 .env.prod 의 계정 정보도 **동일하게** 맞추어야 합니다. 또한 이미 생성된 데이터베이스의 계정은 .env 수정만으로는 변경되지 않으므로 데이터베이스에서 직접 변경해야 합니다.
>
> ※ **운용 배포 전 자격 증명을 반드시 교체하십시오.** .env, .env.prod, db/initdb.d 하위 SQL, DataEventTrackingService/docker-compose.yml 에 초기 계정 정보가 포함되어 있습니다.

바. Docker 네트워크 생성

서버에서 기동되는 컨테이너들은 monitoring 이라는 이름의 네트워크를 통해 서로 통신합니다.

따라서 컨테이너를 기동하기 전에 이 네트워크를 먼저 만들어야 합니다.

```
cd /usr/local/dataeventtrackingservice
docker compose -f network-dash-create.yml up -d
```

- 이 설정 파일에는 1 초 후 스스로 종료되는 임시 컨테이너만 정의되어 있습니다. 실행 목적은 컨테이너 구동이 아니라 네트워크를 만드는 것이므로, 컨테이너가 곧바로 종료되는 것은 정상입니다.
- 아래 명령어로 직접 생성하여도 결과는 동일합니다.

```
docker network create --driver bridge \
    --subnet 172.20.0.0/16 --gateway 172.20.0.1 monitoring
```

네트워크가 생성되었는지 확인합니다.

```
docker network ls | grep monitoring
```

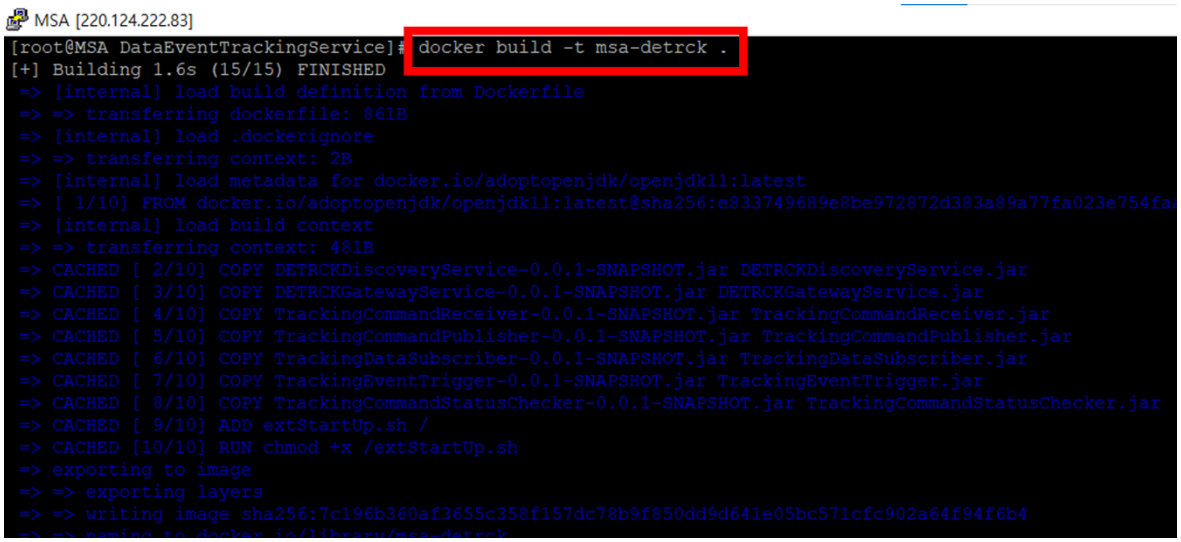
> ※ 서버에 이미 `172.20.0.0/16` 대역을 사용하는 다른 네트워크가 있으면 `Pool overlaps with other one on this address space` 오류가 발생합니다. 이 경우 기존 네트워크를 확인(`docker network ls`)하여 재사용하거나, `network-dash-create.yml` 의 대역을 다른 값으로 변경하십시오.

사. 백엔드 이미지 생성

백엔드 서비스를 기동하려면 실행 파일(jar)을 담은 이미지(`msa-detrck`)를 먼저 만들어야 합니다. `docker-compose.yml` 에 이미지를 만드는 과정이 포함되어 있지 않으므로 아래 명령어를 직접 실행합니다.

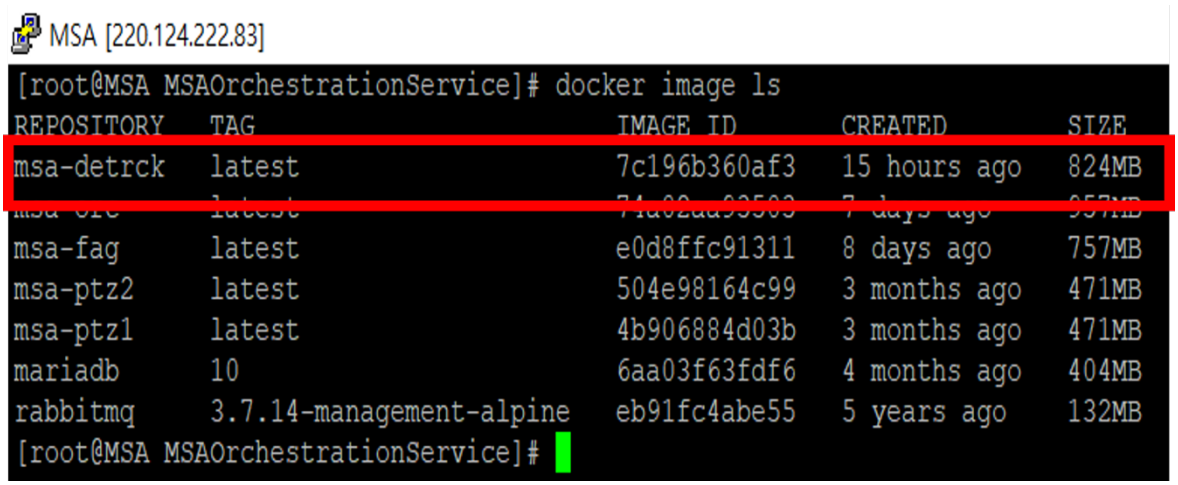
```
cd /usr/local/dataeventtrackingservice/DataEventTrackingService
docker build -t msa-detrck .
```

- 명령어 마지막의 점(.)은 현재 폴더를 의미하므로 빠뜨리지 않도록 주의합니다.
- `Successfully tagged msa-detrck` 메시지가 출력되면 정상적으로 생성된 것입니다.



[그림 23] 백엔드 이미지 생성(docker build)

```
docker images | grep msa-detrck
```



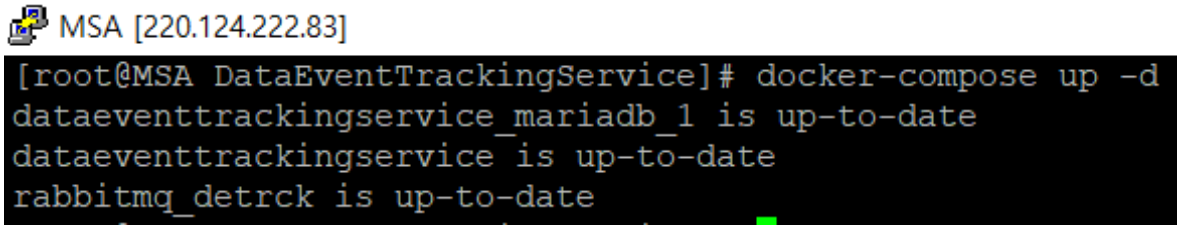
[그림 24] 생성된 이미지 확인(docker images)

> ※ **백엔드 소스를 수정한 경우 반드시 이 단계를 다시 수행해야 합니다.** jar 파일만 교체하고 이미지를 다시 만들지 않으면 이전 버전이 계속 구동됩니다.
>
> ※ 이미지를 다시 만든 후에는 컨테이너도 재생성해야 반영됩니다. (`docker compose up -d --force-recreate`)

아. 백엔드 서비스 기동

```
cd /usr/local/dataeventtrackingservice/DataEventTrackingService
docker compose up -d
```

- `-d` 는 백그라운드로 기동하는 옵션입니다. 이 옵션이 없으면 터미널이 로그 출력에 점유됩니다.
- 최초 실행 시에는 이미지를 내려받으므로 수 분이 소요될 수 있습니다.



[그림 25] 백엔드 서비스 기동(docker compose up -d)

○ 기동되는 컨테이너와 포트 (서버 포트 : 컨테이너 포트)

컨테이너	포트	설명
dataeventtrackingservice	8763:8761	Eureka 서비스 등록
	8002:8000	API 게이트웨이
	22419:22419	명령 수신 (대시보드가 호출)
	22429:22429	명령 발행
	22439:22439	데이터 수신
	22449:22449	이벤트 트리거
	22459:22459	(현재 미사용)
rabbitmq_detrck	5674:5672	메시지 통신 포트(AMQP)
	15674:15672	관리 콘솔(웹 화면)
	1885:1883	MQTT
	61617:61613	STOMP

컨테이너	포트	설명
mariadb	21119:3306	데이터베이스 접속 포트

- 대시보드가 호출하는 트래킹 서버 포트는 **22419** 이며, 5 장「라」에서 설정한 트래킹 서버 주소와 일치해야 합니다.

> ※ 7 개 서비스는 컨테이너 내부의 `extStartUp.sh` 에 의해 **5 초 간격으로 순차 기동**되므로, 컨테이너를 기동한 직후에는 아직 준비되지 않은 서비스가 있을 수 있습니다. 전체 기동까지 약 30 초 이상 소요됩니다.
>
> ※ MariaDB 는 데이터 폴더(`/hdd/dataTest/mariadb3`)가 비어 있는 **최초 기동 시에만** `db/initdb.d` 하위의 SQL(테이블 생성 등)을 실행합니다. 이미 데이터가 있는 경우에는 실행되지 않으므로, 테이블 변경이 필요한 경우 데이터베이스에서 직접 반영해야 합니다.

자. 웹 서버(Nginx) 기동

대시보드 화면 파일(dist)은 6 장「다」의 운용 폴더 이관 시 /usr/local/dataeventtrackingservice/frontend/dist 에 이미 배치되어 있습니다. 본 절에서는 웹 서버만 기동합니다.

1) 화면 파일 확인

```
ls -al /usr/local/dataeventtrackingservice/frontend/dist
```

- index.html 과 assets 폴더가 있으면 정상입니다.

> ※ 파일이 없거나 이전 빌드본인 경우, 5 장「마」○ 빌드 산출물을 운용 폴더에 반영 을 수행하지 않은 것입니다. 작업 PC 에서 해당 단계를 수행한 뒤 7 장「다」의 dist 업로드 절차로 교체하십시오.
>
> ※ 이미 운용 중인 서버에서 화면만 갱신하는 경우는 7 장「다. 화면 갱신 배포 순서 요약」을 참고하십시오.

2) 웹 서버(Nginx) 기동

```
cd /usr/local/dataeventtrackingservice
docker compose -f docker-compose_front.yml up -d
```

- 설정 파일이 기본 이름(`docker-compose.yml`)이 아니므로 `-f` 옵션으로 파일명을 지정해야 합니다.

구분	내용
컨테이너 명	`nginx-proxy`
포트	**8088:80** (서버의 8088 포트로 접속)

구분	내용
설정 파일	`./nginx/nginx.conf` → 컨테이너의 Nginx 설정으로 연결
화면 파일	`./frontend/dist` → 컨테이너의 화면 폴더로 연결
네트워크	`monitoring`

`frontend/dist` 폴더가 컨테이너에 직접 연결되므로, 이미 웹 서버가 구동 중인 상태에서 `dist` **내용만** 교체하는 경우에는 재기동 없이 화면에 반영됩니다.

3) 요청 전달(프록시) 설정

현재 적용 중인 `nginx.conf` 는 아래와 같이 요청을 전달합니다.

요청 경로	전달 대상	용도
/	/usr/share/nginx/html	대시보드 화면 파일 전달 (화면 이동 처리 포함)
/dataeventtrackingservice/	http://dataeventtrackingservice:22419/	데이터/이벤트 트래킹 조회
/meta/	메타 정보 서버 주소	연합트윈 메타 정보 조회

> ※ 전달 대상이 컨테이너 이름(`dataeventtrackingservice`)으로 지정되어 있으므로, **백엔드 컨테이너가 먼저 기동되어 있어야 합니다.** 기동되어 있지 않으면 Nginx 가 `host not found in upstream` 오류를 출력하며 곧바로 종료됩니다.

차. 구동 상태 확인

백엔드와 웹 서버는 서로 다른 compose 파일로 기동되므로, 확인 명령도 각각 실행해야 합니다.

○ 백엔드 컨테이너 확인

```
cd /usr/local/dataeventtrackingservice/DataEventTrackingService
docker compose ps            # 구동 중인 컨테이너 목록
docker compose ps -a         # 종료된 컨테이너까지 포함한 전체 목록
```

- `STATUS` 항목이 `Up` 으로 표시되면 정상 기동된 상태입니다.
- 아래 5 개 컨테이너가 모두 표시되어야 합니다.

```
dataeventtrackingservice   rabbitmq_detrck   mariadb   rabbitmq_exporter   mariadb_exporter
```

○ 웹 서버 컨테이너 확인

```
cd /usr/local/dataeventtrackingservice
docker compose -f docker-compose_front.yml ps
```

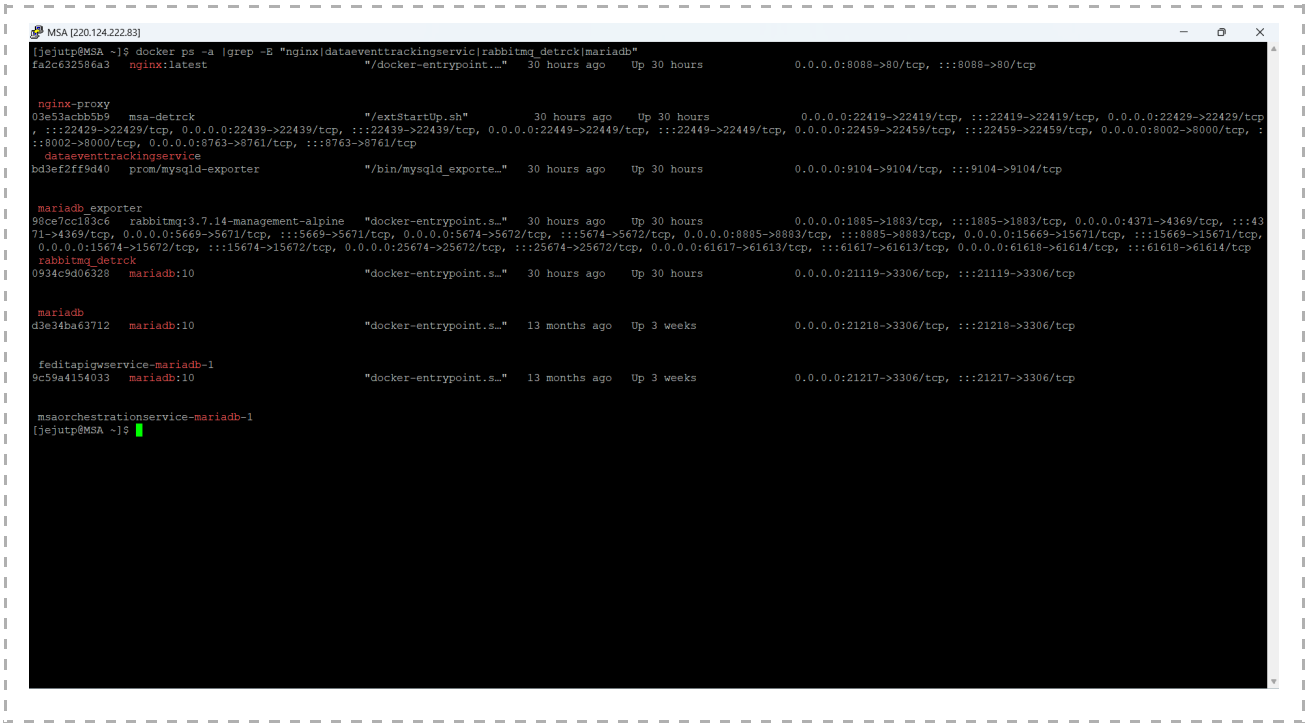
- `nginx-proxy` 가 `Up` 으로 표시되면 정상입니다.

> ※ 웹 서버는 파일명이 기본값(docker-compose.yml)이 아닌 docker-compose_front.yml 이므로 반드시 -f 옵션을 붙여야 합니다. 옵션 없이 docker compose ps 를 실행하면 이 폴더에는 docker-compose.yml 이 없어 no configuration file provided 오류가 나고, 백엔드 폴더에서 실행하면 웹 서버가 목록에 나오지 않습니다.

○ 백엔드·웹 서버를 한 번에 확인

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}" \
  | grep -E "nginx-proxy|dataeventtrackingservice|rabbitmq_detrck|mariadb"
```

- `docker ps` 는 `compose` 파일과 무관하게 서버에서 구동 중인 모든 컨테이너를 보여주므로, 백엔드와 웹 서버를 한 번에 확인할 때 사용합니다.



[그림 26] 구동 컨테이너 확인

○ 로그 확인

```
docker compose logs -f dataeventtrackingservice   # 백엔드 기동 로그
docker logs -f nginx-proxy                        # 웹 서버 로그 (compose 파일과 무관)
docker compose logs --tail 100 mariadb            # 최근 100 줄만 확인
```

백엔드 로그에 아래와 같이 각 서비스의 기동 메시지가 순서대로 출력되면 정상입니다.

```
Starting DETRCKGatewayService-0.0.1-SNAPSHOT Server...
Starting TrackingCommandReceiver-0.0.1-SNAPSHOT Server...
  ( … 이하 생략 … )
```

- 로그 확인을 종료하려면 `Ctrl + C` 를 입력합니다. (컨테이너는 계속 구동됩니다)

○ 서버 내부에서 응답 여부 확인

```
curl -I http://localhost:8088/                                  # 대시보드 화면
curl -I http://localhost:8088/dataeventtrackingservice/ # 트래킹 조회 전달
```

- 첫 줄에 `HTTP/1.1 200 OK` 가 출력되면 정상입니다.

○ 웹 브라우저 확인 주소

구분	접속 주소	접근 조건
대시보드	`http://{serverIp}:8088/`	누구나 (개방)
데이터/이벤트 트래킹 화면	`http://{serverIp}:8088/data-tracking`	누구나 (개방)
Eureka (서비스 등록 현황)	`http://{serverIp}:8763`	**운영자 확인용 — 아래 SSH 터널 이용 권장**
RabbitMQ 관리 콘솔	`http://{serverIp}:15674`	운영자 확인용

- 화면이 이전 버전으로 표시되는 경우 브라우저 캐시를 삭제한 후 다시 확인합니다. (강력 새로 고침: Windows / Linux `Ctrl + Shift + R`, macOS `Cmd + Shift + R`)

> ※ Eureka(8763)와 RabbitMQ 관리 콘솔(15674)은 이용자용 화면이 아니라 운영자 확인용입니다. 확인이 필요할 때는 아래 두 가지 방법 중 하나를 사용합니다.

방법 1 — 서버 안에서 확인 (간단)

```
curl -s http://localhost:8763/eureka/apps | grep -o '<app>.*</app>' | head
```

방법 2 — SSH 터널로 브라우저 확인 (화면이 필요한 경우)

작업 PC 에서 아래 명령으로 터널을 연결한 뒤, **작업 PC 의 브라우저**에서 `http://localhost:8763` 로 접속합니다.

```
ssh -L 8763:localhost:8763 -L 15674:localhost:15674 {계정}@{serverIp} -p {sshPort}
```

- 터널이 연결된 동안에는 `http://localhost:15674` 로 RabbitMQ 관리 콘솔도 함께 확인할 수 있습니다.
- 터미널 창을 닫으면 터널도 함께 끊어집니다.

○ 배포 확인 체크리스트

No	확인 항목	확인 내용
1	백엔드 빌드	jar 7 개 생성 및 `initResource/DataEventTrackingService/` 반영 여부
2	이미지 재생성	소스 수정 후 `docker build -t msa-detrck .` 재수행 여부
3	서버 URL 설정	소스 4 개 파일의 트래킹 서버 주소 수정 완료 여부

No	확인 항목	확인 내용
4	프론트 빌드	URL 수정 후 npm run build 재수행, dist 생성 및 initResource/frontend/ 반영 여부
5	숨김 파일 전송	서버의 DataEventTrackingService/.env, DataEventTrackingService/.env.prod 존재 여부
6	기존 배포본 백업	기존 dist 폴더 이름 변경(백업) 완료 여부 (화면 갱신 배포 시)
7	컨테이너 기동	백엔드 5 개 + 웹 서버 1 개 컨테이너 모두 Up 표시 여부 (docker ps)
8	화면 표시	운용 서버 주소 접속 시 대시보드 초기 화면 정상 표시 여부
9	데이터 연동	대시보드 내 조회 기능 및 데이터 정상 표시 여부
10	화면 이동	메뉴 이동 및 새로 고침 시 화면 정상 표시 여부

카. 컨테이너 중지 및 재기동

구분	명령어	설명
기동	docker compose up -d	컨테이너를 만들고 기동합니다.
중지	docker compose stop	컨테이너만 중지합니다. (설정 / 데이터 유지)
재개	docker compose start	중지된 컨테이너를 다시 기동합니다.
재기동	docker compose restart	중지 후 다시 기동합니다. (설정 변경 반영)
삭제	docker compose down	컨테이너를 중지하고 삭제합니다. (저장된 데이터와 네트워크는 유지)
강제 재생성	docker compose up -d --force-recreate	이미지를 다시 만든 후 반영할 때 사용합니다.
개별 재기동	docker compose restart {컨테이너명}	특정 컨테이너만 재기동합니다.

○ 대상별 실행 위치

설정 파일이 두 개로 나뉘어 있으므로, 명령을 실행하는 위치와 옵션에 유의합니다.

```
# 대시보드(Nginx)를 재기동하는 경우
cd /usr/local/dataeventtrackingservice
docker compose -f docker-compose_front.yml restart


# 백엔드 / RabbitMQ / MariaDB 를 재기동하는 경우
cd /usr/local/dataeventtrackingservice/DataEventTrackingService
docker compose restart
```

- 모든 컨테이너는 자동 재기동(restart: unless-stopped)으로 설정되어 있어, 서버를 재부팅하면 자동으로 다시 기동됩니다.

- 단, `docker compose stop` 으로 직접 중지한 컨테이너는 자동으로 기동되지 않습니다.

타. 문제 해결

증상	원인	조치
컨테이너 기동 시 `network monitoring not found` 오류	네트워크가 생성되지 않음	6 장「바」의 네트워크 생성을 먼저 수행
네트워크 생성 시 `Pool overlaps` 오류	동일 IP 대역(172.20.0.0/16)을 쓰는 네트워크가 이미 존재	기존 네트워크 재사용 또는 대역 변경
`nginx-proxy` 가 기동 직후 종료됨	`host not found in upstream` (백엔드 컨테이너 미기동)	백엔드 컨테이너를 먼저 기동한 후 `nginx-proxy` 재기동
백엔드 컨테이너 기동 실패	`msa-detrck` 이미지가 생성되어 있지 않음	`Tracking` 폴더에서 `docker build -t msa-detrck .`
소스를 고쳤는데 반영되지 않음	이미지를 다시 만들지 않음	jar 교체 → `docker build` → `docker compose up -d --force-recreate`
백엔드가 DB 접속 실패	`.env` 와 `.env.prod` 의 계정 정보 불일치	양쪽 설정 값 확인 및 데이터베이스 계정 확인
백엔드 기동 시 설정값이 비어 있음	`.env`, `.env.prod` 가 서버로 전송되지 않음 (숨김 파일 누락)	6 장「다」의 숨김 파일 존재 확인 수행
화면이 백지로 표시됨	`frontend/dist` 에 화면 파일이 없음	5 장「마」의 운용 폴더 반영 누락 여부 확인. 이미 운용 중인 서버는 7 장「다」의 dist 업로드 수행 후 nginx-proxy 재기동
화면은 표시되나 데이터가 조회되지 않음	트래킹 서버 주소 불일치 또는 백엔드 미기동	5 장「라」의 주소 설정 확인 및 `docker compose ps` 로 기동 상태 확인
포트 사용 중 오류 (`port is already allocated`)	동일 포트를 사용하는 다른 프로그램이 구동 중	`sudo lsof -i :{포트}` 로 확인 후 해당 프로그램 종료 또는 포트 변경
서버에서는 접속되나 외부 PC 에서 접속 불가	외부에서 서버의 8088 포트에 접근할 수 없음	네트워크 담당자에게 8088 포트 접근 허용 요청 (공유기 / 보안 그룹 포함)
Windows 에서 Clone 실패 (`Filename too long`)	경로 길이 제한(260 자) 초과	`git config --global core.longpaths true` 설정 후 짧은 경로에서 재시도
`docker` 명령이 `permission`	계정이 `docker` 그룹에 없음	6 장「나」5) 수행 후 **재접속**

증상	원인	조치
`denied … docker.sock` 오류		
`docker: command not found`	Docker 가 설치되지 않음	6 장「나」4) 수행
`rabbitmq_detrck` · `mariadb` 가 `Restarting` 반복	SELinux 가 `/hdd` 쓰기를 차단	6 장「라」의 `restorecon` 수행. `sudo ausearch -m avc -ts recent` 로 차단 기록 확인
재부팅 후 컨테이너가 하나도 없음	Docker 서비스가 자동 시작되지 않음	`sudo systemctl enable --now docker` (6 장「나」4) (5))
`docker pull` / `dnf install` 이 응답 없음	인터넷 차단 또는 프록시 미설정	7 장「바」(폐쇄망) ·「사」(프록시) 참고
트래킹 이력의 시각이 실제와 다름	서버 시각 미동기화 또는 시간대 불일치	6 장「나」3) 수행 (`timedatectl`)

7. 부록

가. 신규 서버 구축 순서 요약

순서	작업	명령어 / 참조	위치
1	소스 코드 내려받기	`git clone https://github.com/etri/FeDiT.git`	작업 PC
2	백엔드 빌드	`./mvnw clean install`	작업 PC
3	jar 배포 폴더 반영	4 장「바」	작업 PC
4	패키지 설치	`npm install`	작업 PC
5	서버 URL 설정	5 장「라」— `src/config/trackingApi.ts` 운용 프리셋	작업 PC
6	프론트 빌드	`npm run build`	작업 PC
7	Docker 설치	6 장「나」4) `dnf install docker-ce …`	서버
8	docker 그룹 등록	`sudo usermod -aG docker $USER` → 재접속	서버
9	SELinux 규칙 등록	`sudo semanage fcontext -a -t container_file_t '/hdd(/.*)?'`	서버
10	운용 폴더 이관	`scp -r ./initResource … + sudo cp -r`	작업 PC / 서버
11	데이터 폴더 생성	`./create_directories.sh`	서버
12	SELinux 컨텍스트 적용	`sudo restorecon -Rv /hdd`	서버
13	설정 파일 수정	6 장「마」— `.env`, `.env.prod` (`cors_allowed_origin_patterns` 를 서버	서버

		주소로)	
14	네트워크 생성	`docker compose -f network-dash-create.yml up -d`	서버
15	백엔드 이미지 생성	`docker build -t msa-detrck .`	서버
16	백엔드 기동	`docker compose up -d`	서버
17	화면 파일 확인	6 장「자」1) — 운용 폴더 이관 시 함께 반영됨	서버
18	웹 서버 기동	`docker compose -f docker-compose_front.yml up -d`	서버
19	화면 확인	`http://{serverIp}:8088/`	브라우저

> ※ 7~9 번은 **아무것도 설치되지 않은 신규 서버에서만** 수행합니다. 이미 Docker 가 구동 중인 서버라면 10 번부터 시작합니다.

나. 백엔드 갱신 배포 순서 요약

순서	작업	명령어 / 참조	위치
1	최신 소스 반영	`git pull origin main`	작업 PC
2	설정 파일 확인	6 장「마」— `.env.prod` 의 `cors_allowed_origin_patterns` 가 현재 서버 주소인지 확인 (다르면 등록·취소가 403)	서버
3	빌드	`./mvnw clean install`	작업 PC
4	jar 업로드	`scp ./initResource/DataEventTrackingService/*.jar …`	작업 PC
5	이미지 재생성	`docker build -t msa-detrck .`	서버
6	컨테이너 재생성	`docker compose up -d --force-recreate`	서버
7	기동 확인	`docker compose logs -f dataeventtrackingservice`	서버

다. 화면 갱신 배포 순서 요약

순서	작업	명령어 / 참조	위치
1	최신 소스 반영	`git pull origin main`	작업 PC
2	패키지 설치(변경 시)	`npm install`	작업 PC
3	서버 URL 확인	5 장「라」— `src/config/trackingApi.ts` 의 `TARGET` 이	작업 PC

		운용 서버 프리셋인지 확인 (로컬이면 데이터가 조회되지 않음)	
4	빌드	`npm run build`	작업 PC
5	기존 dist 백업	폴더 이름 변경(99.backup)	서버 / FTP
6	dist 업로드	본 절 아래 ○ dist 업로드 절차	작업 PC
7	화면 확인 (캐시 삭제 후)	`http://{serverIp}:8088/`	브라우저

○ dist 업로드 절차

5 장「마」에서 빌드한 dist 폴더를 서버의 배포 경로에 직접 업로드합니다. 이미 운용 중인 서버의 화면만 교체하는 절차이며, 신규 서버 구축 시에는 6 장「다」의 운용 폴더 이관으로 이미 반영됩니다.

```
/usr/local/dataeventtrackingservice/frontend
```

① 기존 배포본을 백업합니다. (서버에서 입력)

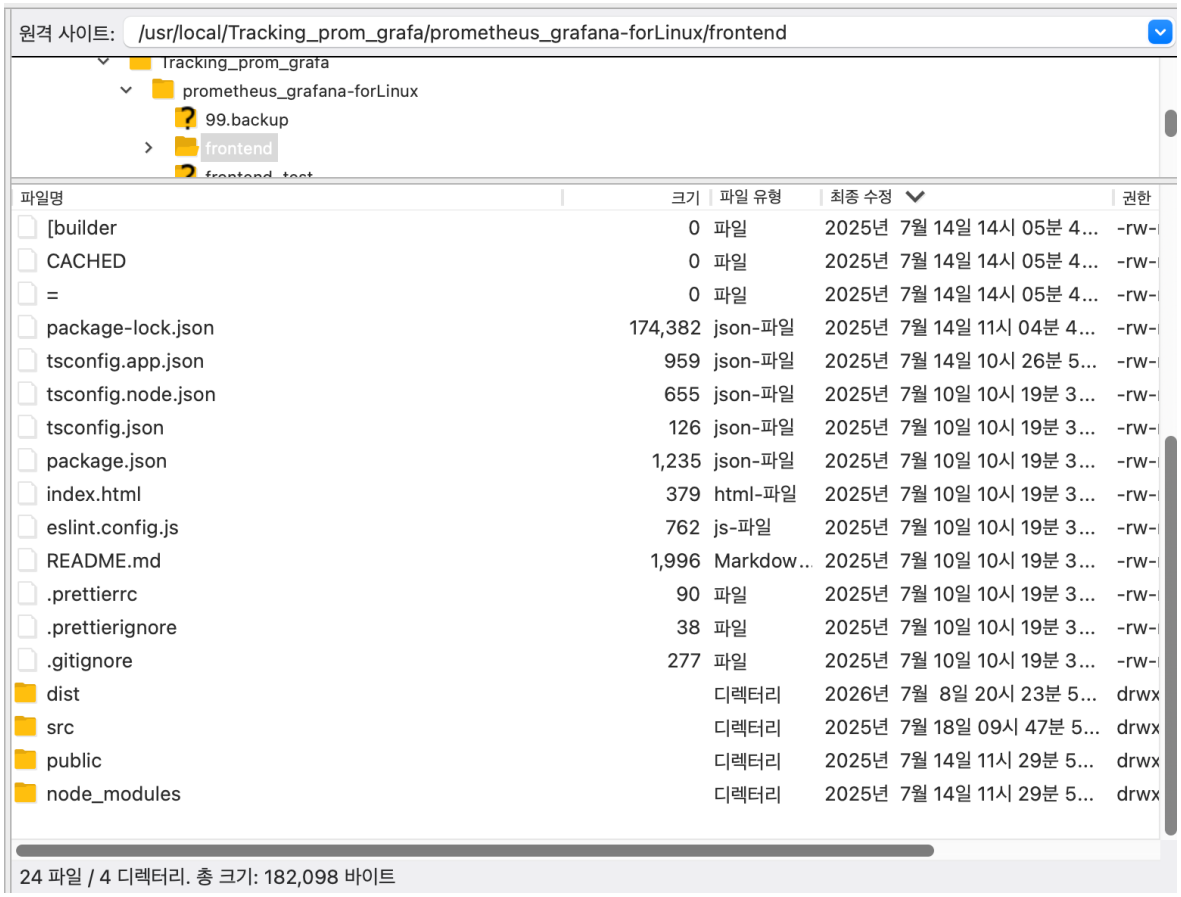
```
cd /usr/local/dataeventtrackingservice/frontend
mv dist 99.backup
```

② `dist` 폴더를 업로드합니다. (작업 PC 에서 입력)

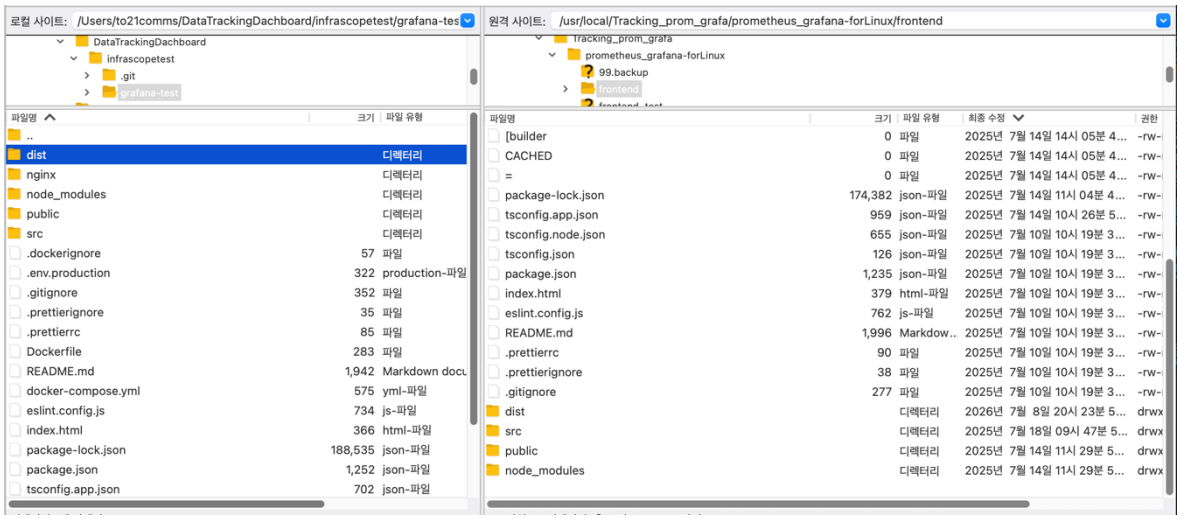
```
cd {작업폴더}/OPFW/msa_dynamic_event_tracker/FrontEnd/src/DataEventTrackingDashboard


scp -r -P {sshPort} ./dist \
    {계정}@{serverIp}:/usr/local/dataeventtrackingservice/frontend/
```

- FileZilla 를 사용하는 경우: 좌측 [로컬 사이트] 에서 `FrontEnd/src/DataEventTrackingDashboard/dist` 폴더를, 우측 [원격 사이트] 의 `frontend` 폴더로 드래그 앤 드롭합니다.
- 화면 하단의 전송 큐에 [실패한 전송] 항목이 없는지 확인합니다.



[그림 27] FTP 접속 및 배포 경로 확인



[그림 28] 빌드 산출물(dist) 업로드

③ 업로드 결과를 확인합니다. (서버에서 입력)

```
ls -al /usr/local/dataeventtrackingservice/frontend/dist
```

- `index.html` 과 `assets` 폴더가 있으면 정상입니다.

> ※ `dist` 폴더를 **삭제하지 않고 이름을 변경(백업)** 하는 방식을 권장합니다. 폴더를 삭제한 후 새로 만들면 웹 서버 컨테이너와의 연결이 끊어져 화면이 표시되지 않을 수 있으며, 이 경우 `nginx-proxy` 를 재기동해야 합니다.
>
> ※ 배포 후 문제가 발생하면 `99.backup` 폴더를 다시 `dist` 로 되돌려 이전 상태로 복구합니다.

라. 자주 사용하는 명령어

구분	명령어	설명
상태 확인	docker compose ps	해당 compose 파일이 관리하는 컨테이너 상태 확인
상태 확인	docker compose ps -a	종료된 컨테이너까지 확인
로그 확인	docker compose logs -f {서비스명}	실시간 로그 확인 (종료: `Ctrl + C`)
로그 확인	docker compose logs --tail 100 {서비스명}	최근 100 줄만 확인
재기동	docker compose restart {서비스명}	특정 서비스만 재기동
내부 접속	docker compose exec {서비스명} bash	컨테이너 내부 접속 (종료: `exit`)
이미지 확인	docker images	생성된 이미지 목록 확인
네트워크 확인	docker network ls	생성된 네트워크 목록 확인
자원 사용량	docker stats --no-stream	컨테이너별 CPU / 메모리 사용량
포트 확인	sudo ss -lntp | grep {포트}	해당 포트를 사용 중인 프로그램 확인

마. 설정 항목 일람

운용 환경 변경 시 아래 항목을 일괄 확인하십시오.

구분	설정 위치	비고
대시보드 접속 포트	`docker-compose_front.yml` (8088:80)	이용자 PC 에서 접근 가능해야 함
트래킹 서버 주소 (프론트)	`FrontEnd/.../src/config/trackingApi.ts`	5 장「라」· 이 파일 한 곳만 수정
CORS 허용 출처 (백엔드)	`DataEventTrackingService/.env.prod` (cors_allowed_origin_patterns)	서버 IP 변경 시 필수. 누락 시 등록·취소가 403
메타 정보 주소 (프론트)	`DataTrackingFormItem.tsx` 100/106/130 행	소스 하드코딩

메타 정보 주소 (프록시)	`nginx/nginx.conf /meta/`	
DB 접속 정보	`DataEventTrackingService/.env`, `DataEventTrackingService/.env.prod`	양쪽 일치 필요
RabbitMQ 접속 정보	`DataEventTrackingService/.env.prod`	컨테이너명 기준
로그 / 데이터 저장 경로	`create_directories.sh`, `DataEventTrackingService/docker-compose.yml`	/hdd 하위

> ※ 위 표의 주소가 서로 다른 서버를 가리키고 있으면 화면은 표시되나 데이터가 조회되지 않습니다. 운용 서버를 변경할 때는 **표의 모든 항목을 함께** 확인하십시오.

바. 인터넷이 차단된 환경에서의 설치 (폐쇄망)

운용 서버가 인터넷에 접속할 수 없는 경우, 필요한 패키지와 컨테이너 이미지를 **인터넷이 가능한 장비에서 미리 내려받아 반입**해야 합니다.

○ 1) Docker 설치 패키지 반입

인터넷이 가능한 **동일 버전의 Rocky 9 장비**에서 수행합니다.

```
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo


mkdir -p ~/docker-rpms && cd ~/docker-rpms
sudo dnf download --resolve --alldeps \
  docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin \
  policycoreutils-python-utils chrony


tar czf docker-rpms.tar.gz *.rpm
```

운용 서버로 전송한 뒤 설치합니다.

```
tar xzf docker-rpms.tar.gz
sudo dnf install -y ./*.rpm
sudo systemctl enable --now docker
```

○ 2) 컨테이너 이미지 반입

이 시스템이 사용하는 이미지는 아래 5 종입니다.

이미지	사용처
`nginx:latest`	대시보드 웹 서버 (6 장「자」)
`rabbitmq:3.7.14-management-alpine`	메시지 큐 (6 장「아」)
`mariadb:10`	데이터베이스 (6 장「아」)

이미지	사용처
adoptopenjdk/openjdk11:latest	백엔드 이미지(msa-detrck)의 기반 (6 장「사」)
busybox	Docker 네트워크 생성 (6 장「바」)

인터넷이 가능한 장비에서 내려받아 하나의 파일로 묶습니다.

```
docker pull nginx:latest
docker pull rabbitmq:3.7.14-management-alpine
docker pull mariadb:10
docker pull adoptopenjdk/openjdk11:latest
docker pull busybox


docker save -o detrck-images.tar \
  nginx:latest rabbitmq:3.7.14-management-alpine mariadb:10 \
  adoptopenjdk/openjdk11:latest busybox
```

운용 서버로 전송합니다. **(작업 PC 에서 입력)**

```
scp -P {sshPort} detrck-images.tar {계정}@{serverIp}:/usr/local/dataeventtrackingservice/
```

서버에서 적재합니다.

```
cd /usr/local/dataeventtrackingservice
docker load -i detrck-images.tar
docker images
```

> ※ adoptopenjdk/openjdk11 이 없으면 6 장「사」의 docker build 가 실패합니다. 백엔드 이미지를 만들 때 Dockerfile 의 기반 이미지로 사용되기 때문입니다.
>
> ※ 반입 후에는 6 장「나」8)의 docker run --rm hello-world 가 여전히 실패합니다(해당 이미지는 반입 목록에 없음). 폐쇄망에서는 docker images 로 5 종이 모두 적재되었는지 확인하는 것으로 대신합니다.

사. 프록시 환경에서의 Docker 설정

사내 프록시를 거쳐야 외부에 접속할 수 있는 환경에서는 Docker 데몬에 프록시를 별도로 지정해야 합니다. 사용자 계정의 환경변수(export HTTP_PROXY=…)만 설정하면 docker pull 은 여전히 실패합니다.

```
sudo mkdir -p /etc/systemd/system/docker.service.d


sudo tee /etc/systemd/system/docker.service.d/http-proxy.conf > /dev/null <<'EOF'
[Service]
Environment="HTTP_PROXY=http://{프록시주소}:{프록시포트}"
Environment="HTTPS_PROXY=http://{프록시주소}:{프록시포트}"
Environment="NO_PROXY=localhost,127.0.0.1,::1"
EOF


sudo systemctl daemon-reload
sudo systemctl restart docker


sudo systemctl show docker --property=Environment
docker pull busybox                          # 정상적으로 내려받아지면 설정 완료
```

> ※ {프록시주소} · {프록시포트} 는 네트워크 담당자에게 확인합니다.

> ※ 프록시에 인증이 필요한 경우 `http://{계정}:{비밀번호}@{프록시주소}:{프록시포트}` 형식으로 입력합니다. 이 파일에는 비밀번호가 평문으로 저장되므로 `sudo chmod 600 /etc/systemd/system/docker.service.d/http-proxy.conf` 로 권한을 제한하십시오.

아. Docker 미사용 시 로컬 환경 직접 구성

작업 PC 에 Docker 를 설치할 수 없는 경우, 데이터베이스와 메시지 큐를 직접 설치하여 2 장「다」를 대신할 수 있습니다.

○ 1) 데이터베이스 설치 및 계정 생성

MariaDB 를 설치한 후 root 계정으로 접속하여 서비스 계정을 생성합니다.

```
CREATE USER 'fdtTwin'@'%' IDENTIFIED BY 'Fdt1234!@#$';
GRANT ALL PRIVILEGES ON *.* TO 'fdtTwin'@'%';
FLUSH PRIVILEGES;
```

※ MariaDB 설치 시 root 비밀번호 설정 화면이 표시되지 않아 접속이 거부되는 경우가 있습니다. 이때는 서비스를 중지한 뒤 `mysqld --skip-grant-tables` 로 임시 기동하고 `ALTER USER` 로 비밀번호를 재설정한 후 서비스를 다시 시작합니다.

○ 2) 스키마 및 테이블 생성

DB 접속 도구(HeidiSQL · MySQL Workbench 등)에서 아래 파일을 실행합니다. 스키마 `dataEventTrackings` 와 테이블 11 개가 생성됩니다.

```
{작업폴더}/OPFW/msa_dynamic_event_tracker/initResource/DataEventTrackingService/db/initdb.d/create_table.sql
```

○ 3) 메시지 큐 설치

RabbitMQ 를 설치하고 기동한 뒤, 계정 `guest` 으로 접속되는지 확인합니다.

※ 직접 설치한 경우 포트가 기본값(MariaDB 3306, RabbitMQ 5672)이므로, 2 장「다」5) 표의 포트를 설치 환경에 맞게 바꾸어 `application.properties` 에 입력하십시오.

자. 로컬 빌드·구동 검증 결과

본 절은 3 장 ~ 5 장의 빌드 절차와 2 장「다. 로컬 실행 환경 준비」의 인프라 기동 절차를 저장소의 소스로 실제 수행하여, 백엔드와 프론트엔드가 로컬 환경에서 정상적으로 빌드되고 구동되는지 확인한 결과입니다.

아래 내용은 2026-08-06 기준으로 확인된 실행 결과이며, 처음 수행하는 담당자가 자신의 결과와 비교할 수 있도록 실제 출력값을 그대로 수록하였습니다.

○ 1) 검증 환경

구분	검증에 사용한 값
운영체제	macOS 26.5 (Apple Silicon, arm64)
JDK	Java 11.0.21 LTS
빌드 도구	Maven 3.8.7 (프로젝트에 포함된 `mvnw` 래퍼가 자동 내려받음)
Node.js / npm	v20.19.5 / 10.8.2
Docker / Compose	28.1.1 / v2.36.0
데이터베이스	MariaDB 10 (컨테이너 `detrck-mariadb-local`, 포트 30068)
메시지 큐	RabbitMQ 3.7.14-management-alpine (컨테이너 `detrck-rabbitmq-local`, 포트 30071 / 30072)

> ※ 백엔드 빌드는 반드시 JDK 11 로 수행합니다. 작업 PC 에 여러 JDK 가 설치되어 있는 경우 `JAVA_HOME` 을 JDK 11 로 지정한 뒤 빌드하십시오. (2 장「가. 개발 환경 정보」참고)
>
> ※ 본 절의 결과는 macOS(arm64) 기준입니다. Windows 에서도 동일한 절차로 수행하며, 명령어는 2 장「마. 표기 규칙」에 따라 각 운영체제에 맞는 형태를 사용합니다.

○ 2) 백엔드 빌드 결과

4 장「마. 빌드」의 명령행 빌드를 수행한 결과입니다.

```
cd {작업폴더}/OPFW/msa_dynamic_event_tracker/BackEnd/src/DataEventTrackingService


# macOS / Linux
export JAVA_HOME=$(/usr/libexec/java_home -v 11)   # macOS
./mvnw clean package -DskipTests


# Windows
mvnw.cmd clean package -DskipTests
```

상위 프로젝트를 포함한 9 개 프로젝트가 모두 성공하고, 마지막에 `BUILD SUCCESS` 가 출력됩니다.

```
[INFO] Reactor Summary for DataEventTrackingService 0.0.1-SNAPSHOT:
[INFO]
[INFO] DataEventTrackingService ........................... SUCCESS
[INFO] common ............................................. SUCCESS
[INFO] TrackingEventTrigger ............................... SUCCESS
[INFO] TrackingDataSubscriber ............................. SUCCESS
[INFO] TrackingCommandPublisher ........................... SUCCESS
[INFO] TrackingCommandReceiver ............................ SUCCESS
[INFO] TrackingCommandStatusChecker ....................... SUCCESS
[INFO] DETRCKDiscoveryService ............................. SUCCESS
[INFO] DETRCKGatewayService ............................... SUCCESS
[INFO] ------------------------------------------------------------
```

```
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

빌드가 끝나면 각 모듈의 `target/` 폴더에 아래와 같이 jar 파일이 생성됩니다. 이 중 `common` 을 제외한 7 개가 4 장「사」에서 배포 폴더로 복사할 대상입니다.

모듈	산출물	크기(참고)
common	`common-0.0.1-SNAPSHOT.jar`	48 KB (공통 라이브러리, 배포 대상 아님)
DETRCKDiscoveryService	`DETRCKDiscoveryService-0.0.1-SNAPSHOT.jar`	54 MB
DETRCKGatewayService	`DETRCKGatewayService-0.0.1-SNAPSHOT.jar`	43 MB
TrackingCommandReceiver	`TrackingCommandReceiver-0.0.1-SNAPSHOT.jar`	67 MB
TrackingCommandPublisher	`TrackingCommandPublisher-0.0.1-SNAPSHOT.jar`	44 MB
TrackingDataSubscriber	`TrackingDataSubscriber-0.0.1-SNAPSHOT.jar`	44 MB
TrackingEventTrigger	`TrackingEventTrigger-0.0.1-SNAPSHOT.jar`	29 MB
TrackingCommandStatusChecker	`TrackingCommandStatusChecker-0.0.1-SNAPSHOT.jar`	44 MB

```
[INFO] No tests to run.
[INFO]
[INFO] --- jar:3.2.2:jar (default-jar) @ DETRCKGatewayService ---
[INFO] Building jar: /Users/to21comms/Downloads/msa_dynamic_event_tracker/BackEnd/src/DataEventTrackingService/DETRCKGatewayServic
[INFO]
[INFO] --- spring-boot:2.7.13:repackage (repackage) @ DETRCKGatewayService ---
[INFO] Replacing main artifact with repackaged archive
[INFO]
[INFO] --- install:2.5.2:install (default-install) @ DETRCKGatewayService ---
[INFO] Installing /Users/to21comms/Downloads/msa_dynamic_event_tracker/BackEnd/src/DataEventTrackingService/DETRCKGatewayService/t
[INFO] Installing /Users/to21comms/Downloads/msa_dynamic_event_tracker/BackEnd/src/DataEventTrackingService/DETRCKGatewayService/p
[INFO] ------------------------------------------------------------------------
[INFO] Reactor Summary for DataEventTrackingService 0.0.1-SNAPSHOT:
[INFO]
[INFO] DataEventTrackingService ........................... SUCCESS [  0.130 s]
[INFO] common ............................................. SUCCESS [  1.111 s]
[INFO] TrackingEventTrigger ............................... SUCCESS [  0.593 s]
[INFO] TrackingDataSubscriber ............................. SUCCESS [  0.542 s]
[INFO] TrackingCommandPublisher ........................... SUCCESS [  0.388 s]
[INFO] TrackingCommandReceiver ............................ SUCCESS [  0.612 s]
[INFO] TrackingCommandStatusChecker ....................... SUCCESS [  0.296 s]
[INFO] DETRCKDiscoveryService ............................. SUCCESS [  0.327 s]
[INFO] DETRCKGatewayService ............................... SUCCESS [  0.277 s]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.413 s
[INFO] Finished at: 2026-08-06T15:10:40+09:00
[INFO] ------------------------------------------------------------------------
```

[그림 29] 백엔드 빌드 결과 – BUILD SUCCESS

> ※ 의존 라이브러리가 이미 내려받아져 있는 상태에서는 빌드가 수 초 만에 끝납니다. 최초 빌드에서는 라이브러리를 모두 내려받으므로 수 분이 소요되는 것이 정상입니다.
>
> ※ macOS / Linux 에서 `./mvnw` 실행 시 `permission denied` 가 출력되는 경우, 래퍼 파일에 실행 권한이 없는 상태입니다. `chmod +x mvnw` 로 권한을 부여하거나 `sh ./mvnw` 형태로 실행하십시오.
>
> Windows(`mvnw.cmd`)에서는 해당하지 않습니다.

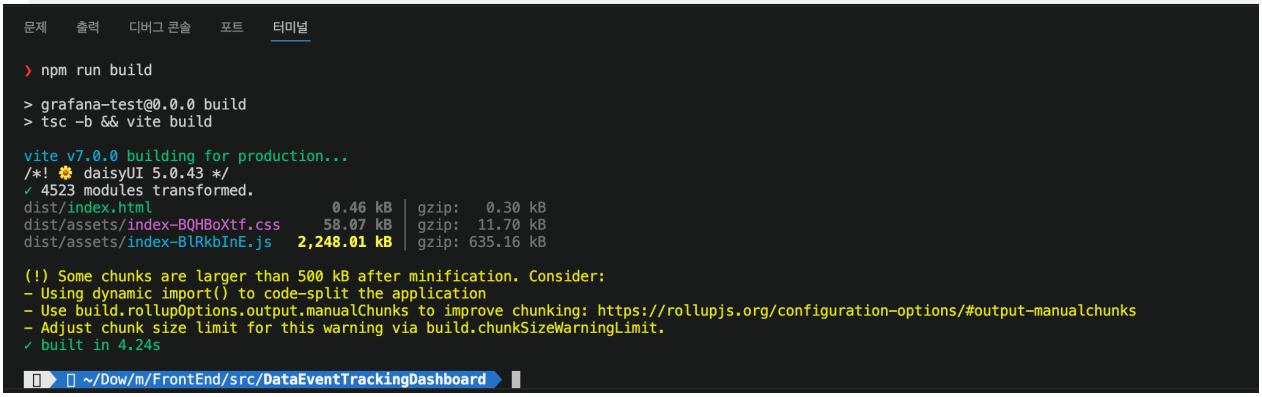
○ 3) 프론트엔드 빌드 결과

5 장「나. 패키지 설치」와「마. 배포 빌드」를 수행한 결과입니다.

```
cd {작업폴더}/OPFW/msa_dynamic_event_tracker/FrontEnd/src/DataEventTrackingDashboard
npm install
npm run build
added 319 packages, and audited 320 packages in 14s


> grafana-test@0.0.0 build
> tsc -b && vite build


vite v7.0.0 building for production...
✓ 4522 modules transformed.
dist/index.html                     0.46 kB │ gzip:   0.30 kB
dist/assets/index-{hash}.css       58.07 kB │ gzip:  11.70 kB
dist/assets/index-{hash}.js     2,248.26 kB │ gzip: 635.19 kB
✓ built in 4.23s
```



[그림 30] 프론트엔드 빌드 결과 – dist 생성

- 의존 패키지 319 개가 설치되며 프로젝트 하위에 `node_modules` 폴더가 생성됩니다.
- 타입 검사(`tsc -b`)가 오류 없이 통과한 뒤 `vite build` 가 수행됩니다.
- `dist` 폴더에 `index.html`, `assets/index-{hash}.js`, `assets/index-{hash}.css`, `vite.svg` 가 생성됩니다. (5 장「바. 빌드 산출물 확인」의 구성과 동일)

> ※ `npm install` 시 출력되는 취약점(vulnerabilities) 경고와 빌드 시 출력되는 `Some chunks are larger than 500 kB` 경고는 모두 빌드 결과에 영향을 주지 않습니다. (5 장「나」·「마」 참고)

○ 4) 로컬 인프라 기동 결과

2 장「다. 로컬 실행 환경 준비」의 절차로 데이터베이스와 메시지 큐를 기동한 결과입니다.

```
cd {작업폴더}/OPFW/msa_dynamic_event_tracker/initResource/DataEventTrackingService
docker compose -f docker-compose.local.yml up -d
docker compose -f docker-compose.local.yml ps
NAME                    STATUS          PORTS
detrck-mariadb-local    Up              0.0.0.0:30068->3306/tcp
detrck-rabbitmq-local   Up              0.0.0.0:30071->5672/tcp, 0.0.0.0:30072->15672/tcp
```

```
❯ cd ~/Downloads/msa_dynamic_event_tracker/initResource/DataEventTrackingServic
e
❯ docker compose -f docker-compose.local.yml ps
WARN[0000] /Users/to21comms/Downloads/msa_dynamic_event_tracker/initResource/Dat
aEventTrackingService/docker-compose.local.yml: the attribute `version` is obsol
ete, it will be ignored, please remove it to avoid potential confusion
NAME                    IMAGE                                COMMAND
      SERVICE    CREATED             STATUS          PORTS
detrck-mariadb-local    mariadb:10                           "docker-entrypoint.s
…"   mariadb    About an hour ago   Up 44 minutes   0.0.0.0:30068->3306/tcp
detrck-rabbitmq-local   rabbitmq:3.7.14-management-alpine   "docker-entrypoint.s
…"   rabbitmq   About an hour ago   Up 44 minutes   4369/tcp, 5671/tcp, 15671/tc
p, 25672/tcp, 0.0.0.0:30071->5672/tcp, 0.0.0.0:30072->15672/tcp
❯
 ~/Dow/m/in/DataEventTrackingService                ✓ base  15:12:30
```

[그림 31] 로컬 인프라 기동 확인 (docker compose ps)

최초 기동 시 스키마 · 서비스 계정 · 테이블이 자동으로 생성되며, 아래 명령어로 테이블 11 개가 확인됩니다.

```
docker exec detrck-mariadb-local mariadb -ufdtTwin -pFdt1234!@#$ \
  -e "show tables from dataEventTrackings;"
```

```
❯ docker exec detrck-mariadb-local mariadb -ufdtTwin -pFdt1234'!@#$' \
    -e "show tables from dataEventTrackings;"
Tables_in_dataEventTrackings
data_event_tracking_request_info
event_occured_info
event_transmission_result_info
tracking_detector_proc_info
tracking_process_info
tracking_process_status_change_record
tracking_response_data
tracking_response_record
tracking_result_data_transmission_info
tracking_status_change_record
tracking_topic_info
❯
 ~/Dow/m/in/DataEventTrackingService                ✓ base  15:13:00
```

[그림 32] 스키마 자동 생성 확인 – 테이블 11 개

> ※ 2 장「다」4)에 기술된 기대 결과(테이블 11 개)와 동일하게 출력되었습니다. 별도로 SQL 을 실행하거나 계정을 만들 필요가 없습니다.

○ 5) 로컬 로그 설정 파일 생성 (백엔드 구동 전 필수)

트래킹 서비스 5 종은 로그 설정 파일(`src/main/resources/logback/logback.xml`)에 로그 경로가 `\hdd\DtEvtTracking\{서비스}` 로 지정되어 있습니다. 운용 서버에서는 6 장「라」의 `create_directories.sh` 로 해당 경로가 생성되지만, 작업 PC 에서는 폴더를 만들어도 이 경로가 그대로 사용되지 않습니다. logback 이 값의 역슬래시를 이스케이프 문자로 처리하여, 앞의 역슬래시는 사라지고 `\t` 는 탭 문자로 치환되기 때문입니다.

트래킹 서비스 5 종은 마지막 폴더명이 모두 t 로 시작하므로 5 종 전부 같은 영향을 받습니다. 그대로 기동하면 아래 오류로 종료됩니다.

```
ERROR in ch.qos.logback.core.rolling.RollingFileAppender[ROLLING-FILE]
  - Failed to create parent directories for
    [...\DataEventTrackingService\hddDtEvtTracking→rackEntTrigger\TrackingEventTrigger.log]
**************************
APPLICATION FAILED TO START
**************************
```

C:\hdd\DtEvtTracking\... 폴더를 미리 만들어 두어도 참조되지 않습니다. 소스를 수정하지 않고 회피하려면, 콘솔로만 출력하는 로컬 전용 로그 설정 파일을 만들어 기동 시 --logging.config 옵션으로 지정합니다.

```
# Windows (PowerShell) — Set-Content 는 한 줄입니다.
$Work = "C:\FeDiT-work\FeDiT\OPFW\msa_dynamic_event_tracker"
Set-Content -Path "$Work\logback-local.xml" -Encoding utf8 -Value '<configuration><appender
name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender"><encoder><pattern>%-
5level %d{HH:mm:ss.SSS} [%thread] %logger{36} - %msg%n</pattern></encoder></appender><root
level="INFO"><appender-ref ref="CONSOLE"/></root></configuration>'


# macOS / Linux — echo 는 한 줄입니다.
WORK=~/FeDiT-work/FeDiT/OPFW/msa_dynamic_event_tracker
echo '<configuration><appender name="CONSOLE"
class="ch.qos.logback.core.ConsoleAppender"><encoder><pattern>%-5level %d{HH:mm:ss.SSS}
[%thread] %logger{36} - %msg%n</pattern></encoder></appender><root level="INFO"><appender-
ref ref="CONSOLE"/></root></configuration>' > $WORK/logback-local.xml


# 기동 시 지정 (7 종 공통, 4 장 「아」 ㅇ 6) 참고)
java -jar {서비스}\target\{서비스}-0.0.1-SNAPSHOT.jar --logging.config=file:///C:/FeDiT-
work/FeDiT/OPFW/msa_dynamic_event_tracker/logback-local.xml
```

서비스	logback.xml 의 LOG_DIR 표기	실제 해석 결과
TrackingCommandReceiver	\hdd\DtEvtTracking\trackCmdReceiver	hddDtEvtTracking→rackCmdReceiver
TrackingCommandPublisher	\hdd\DtEvtTracking\trackCmdPublisher	hddDtEvtTracking→rackCmdPublisher
TrackingDataSubscriber	\hdd\DtEvtTracking\trackDataSub	hddDtEvtTracking→rackDataSub
TrackingEventTrigger	\hdd\DtEvtTracking\trackEntTrigger	hddDtEvtTracking→rackEntTrigger
TrackingCommandStatusChecker	\hdd\DtEvtTracking\trackCmdStsChecker	hddDtEvtTracking→rackCmdStsChecker

※ 표의「실제 해석 결과」는 현재 폴더 기준의 상대 경로이며, 탭(→)이 포함되어 Windows 에서는 생성할 수 없는 이름입니다. 이 때문에 폴더를 만드는 방식으로는 해결되지 않습니다.

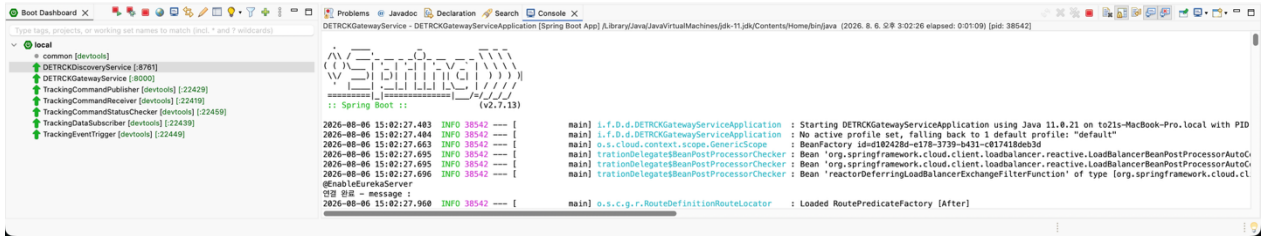
※ 소스의 logback.xml 에서 경로를 /hdd/DtEvtTracking/{서비스} 형태의 슬래시로 바꾸면 폴더를 만드는 방식으로도 동작합니다. 다만 수정 내용이 서버 배포용 jar 에 그대로 반영되므로 로컬 확인 목적으로는 권장하지 않습니다.

※ DETRCKDiscoveryService 와 DETRCKGatewayService 는 별도의 로그 설정 파일이 없어 이 항목의 영향을 받지 않습니다. 7 종에 같은 옵션을 함께 적용해도 문제되지 않습니다.

○ 6) 백엔드 로컬 구동 결과

빌드된 jar 7 개를 4 장 「바」의 기동 순서(Discovery → Gateway → 나머지)에 따라 기동한 결과입니다. 모든 서비스에서 `Started {서비스명}Application in {n} seconds` 메시지가 출력되어 정상 기동이 확인되었습니다.

순서	서비스	포트	기동 확인 로그
1	DETRCKDiscoveryService	8761	`Started DETRCKDiscoveryServiceApplication in 2.348 seconds`
2	DETRCKGatewayService	8000	`Started DETRCKGatewayServiceApplication in 1.809 seconds`
3	TrackingCommandReceiver	22419	`Started TrackingCommandReceiverApplication in 2.656 seconds`
4	TrackingCommandPublisher	22429	`Started TrackingCommandPublisherApplication in 1.886 seconds`
5	TrackingDataSubscriber	22439	`Started TrackingDataSubscriberApplication in 2.023 seconds`
6	TrackingEventTrigger	22449	`Started TrackingEventTriggerApplication in 1.548 seconds`
7	TrackingCommandStatusChecker	22459	`Started TrackingCommandStatusCheckerApplication in 1.566 seconds`



[그림 33] STS Boot Dashboard – 백엔드 7 종 기동

기동 이후 각 서비스의 상태 점검 지점(`/actuator/health`)으로 데이터베이스 및 메시지 큐 연결을 확인합니다.

```
curl -s http://localhost:22419/actuator/health

{
  "status": "UP",
  "components": {
    "db":                 { "status": "UP", "details": { "database": "MariaDB" } },
    "rabbit":             { "status": "UP" },
    "discoveryComposite": { "status": "UP" },
    "diskSpace":          { "status": "UP" },
    "ping":               { "status": "UP" }
  }
}
```

- `status` 가 `UP` 이면 정상입니다. 트래킹 서비스 5 종(22419 · 22429 · 22439 · 22449 · 22459) 모두 `UP` 으로 확인되었습니다.
- `db` 가 `UP` 이면 MariaDB 연결이, `rabbit` 이 `UP` 이면 RabbitMQ 연결이 정상인 상태입니다.

- RabbitMQ 관리 콘솔(http://localhost:30072)에서 federate.tracking 익스체인지와 tracking.receiver.rec · tracking.publish.pub · tracking.trigger.trn · topic.publish.che 등의 큐가 서비스 기동과 함께 자동 생성된 것을 확인할 수 있습니다.

Eureka 등록 상태는 아래 명령어로 확인합니다.

```
curl -s -u dtEtTrck:fdtTwns21! http://localhost:8761/eureka/apps | grep -o "<name>.*</name>"

<name>EUREKA-GATEWAY-SERVICE</name>
<name>TRACKINGCOMMANDRECEIVER</name>
```

> ※ Eureka 에는 Gateway 와 TrackingCommandReceiver 두 개만 등록됩니다. 나머지 서비스 4 종(Publisher · DataSubscriber · EventTrigger · StatusChecker)은 application.properties 에 eureka.client 설정이 선언되어 있지 않아 등록되지 않으며, 이는 현재 구성상 정상입니다. 해당 서비스들은 Eureka 가 아니라 RabbitMQ 를 통해 연계됩니다.

백엔드 자체의 REST 경로는 아래와 같이 확인합니다.

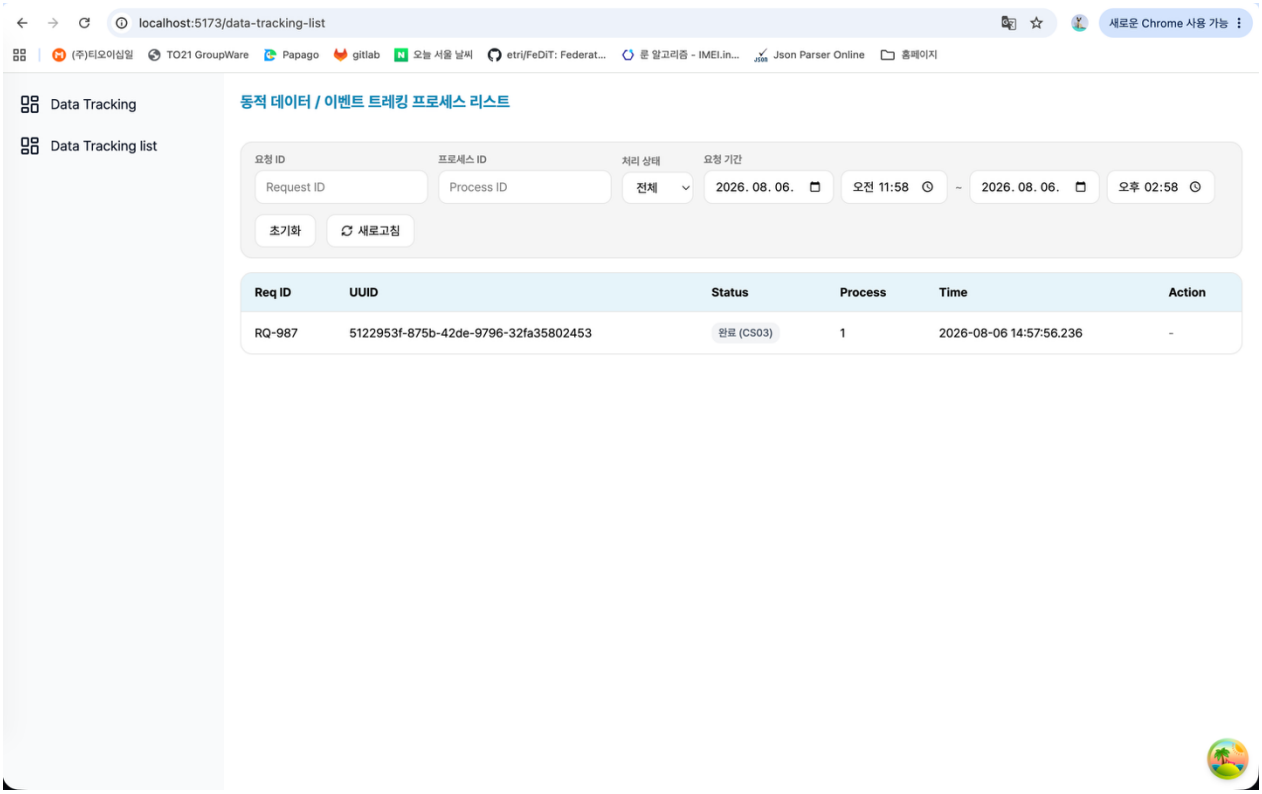
```
curl -i http://localhost:22419/fdt/detrck/rcv/tracking-request-list
```

> ※ 트래킹 이력이 하나도 없는 최초 상태에서는 이 호출이 HTTP 500 과 함께 ***** dtEtTrckReqList is null or Empty ***** 로그를 출력합니다. 이는 조회 결과가 없을 때의 동작이며, 요청이 서비스와 데이터베이스까지 정상적으로 전달되었음을 의미합니다.

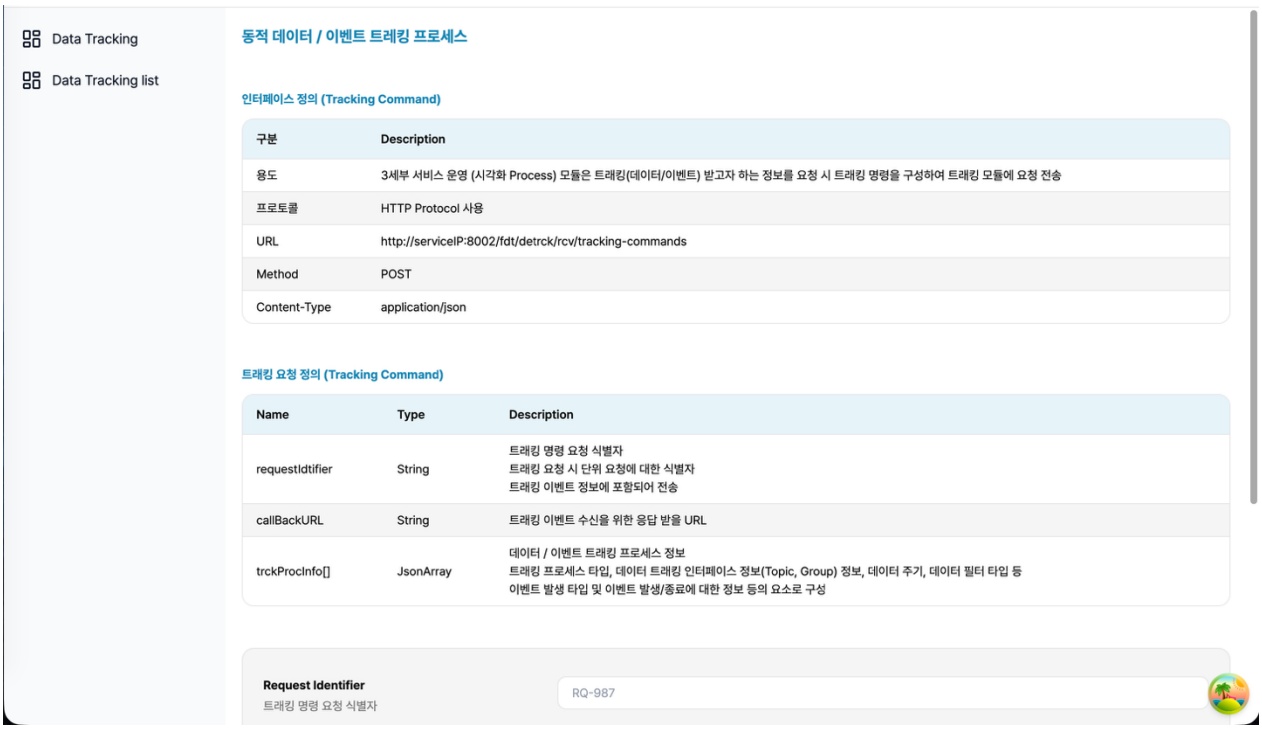
○ 7) 프론트엔드 로컬 구동 결과

5 장 「다. 로컬 구동 확인」의 개발 서버와, 「마. 배포 빌드」로 생성한 산출물(dist)을 각각 구동하여 확인하였습니다.

구분	명령어	접속 주소	결과
개발 서버	npm run dev	http://localhost:5173/	VITE v7.0.0 ready in 334 ms / HTTP 200
빌드 산출물 확인	npm run preview	http://localhost:4173/	dist 의 정적 파일 응답 / HTTP 200



[그림 34] 트래킹 목록 화면 (http://localhost:5173/data-tracking-list)



[그림 35] 트래킹 등록 화면 (http://localhost:5173/data-tracking)

- 두 경우 모두 화면이 정상적으로 표시되며, 좌측 메뉴에서 [Data Tracking] 을 선택하면 트래킹 화면으로 이동합니다.

- `npm run preview` 는 5 장「마」에서 만든 `dist` 를 그대로 서비스하므로, 서버에 올리기 전에 빌드 산출물 자체를 미리 확인하는 용도로 사용할 수 있습니다.

> ※ 화면이 표시되는 것과 데이터가 조회되는 것은 별개입니다. 로컬에서 트래킹 데이터까지 조회하려면 아래「○ 8)」의 서버 주소 설정을 반드시 함께 수행해야 합니다.

○ 8) 로컬 구동 시 서버 주소(URL) 설정

5 장「라. 서버 URL 설정」은 배포 빌드를 기준으로 기술되어 있어, 로컬 구동에는 그대로 적용되지 않습니다. 로컬에서 화면과 백엔드를 연결하려면 아래 내용을 확인해야 합니다.

① `.env.production` 은 로컬 구동에 사용되지 않습니다.

Vite 는 실행 모드에 따라 환경 파일을 선택적으로 읽습니다. `npm run dev` 는 `development` 모드로 동작하므로 `.env.production` 을 읽지 않으며, 저장소에는 `.env.production` 만 있고 `.env` · `.env.development` 는 없습니다. 실제로 개발 서버가 주입한 값을 확인하면 `VITE_` 변수가 하나도 포함되어 있지 않습니다.

```
import.meta.env = {"BASE_URL": "/", "DEV": true, "MODE": "development",
                   "PROD": false, "SSR": false}
```

- 따라서 5 장「라」1)의 `.env.production` 수정은 배포 빌드(`npm run build`)에만 해당하며, 로컬 구동과는 무관합니다.
- 로컬 구동 시에는 모든 주소가 각 소스에 적힌 기본값으로 동작합니다. 트래킹 서버 주소의 경우 `src/config/trackingApi.ts` 의 `DEFAULT_BASE_URL` 이 사용됩니다.

② 화면이 호출하는 주소는 트래킹 서버 한 곳이 전부가 아닙니다.

소스 전체에서 외부 주소를 사용하는 지점은 아래와 같습니다. 로컬 구동 시 실제로 사용되는 값을 함께 정리하였습니다.

구분	소스 위치	로컬 구동 시 사용되는 값
트래킹 서버 주소	`src/config/trackingApi.ts` (트래킹 화면 4 개가 공통 사용)	`http://220.124.222.83:8088`
연합트윈 메타 조회	`DataTrackingFormItem.tsx`:100, 106, 130	`http://220.124.222.84:1221/fedit/v1/...`
Grafana 대시보드	`Dashboard.tsx`:2	`http://221.133.48.18`
Grafana 상세 화면	`RelayPage.tsx`:6	`http://localhost:3000/grafana/...` (소스에 직접 입력)
Neo4j 그래프	`GraphDB.tsx`:52 ~ 55	`bolt://localhost:7687`
쿠버네티스 제어	`KubernetesControl.tsx`,	`/api/v1/...` → 개발 프록시

	DeploymentDetail.tsx	localhost:9505
온톨로지 조회	useDigitalTwins.ts:6	/ngsi-api/... → 개발 프록시 localhost:8083

> ※ 위 표에서 본 문서의 대상은「트래킹 서버 주소」와「연합트윈 메타 조회」두 가지입니다. 나머지 항목은 Grafana · Neo4j · 쿠버네티스 · 온톨로지 등 별도 시스템의 주소이므로 트래킹 화면만 사용하는 경우에는 변경하지 않아도 됩니다. 해당 화면에서 데이터가 조회되지 않는 것은 정상입니다.
>
> ※ .env.production 의 VITE_API_BASE_URL 은 소스 어디에서도 사용되지 않습니다.

③ 트래킹 서버 주소를 백엔드 주소로 바꾸는 것만으로는 동작하지 않습니다.

로컬에서 트래킹 서버 주소를 백엔드 주소(http://localhost:22419)로 지정하여 호출해 본 결과는 아래와 같습니다.

호출 방식	응답	원인
http://localhost:22419/dataeventtrackingservice/fdt/detrck/rcv/...	HTTP 404	/dataeventtrackingservice 는 Nginx 가 백엔드로 전달할 때 제거하는 접두어입니다. 백엔드 자체 경로는 /fdt/detrck/rcv/... 입니다.
http://localhost:22419/fdt/detrck/rcv/... (브라우저에서 등록·취소)	HTTP 403	허용 출처로 등록되지 않은 곳에서의 요청이기 때문입니다. 아래「CORS」 설명을 참고하십시오.

여기서 주의할 점이 있습니다. 브라우저는 등록·취소(POST · DELETE)에는 같은 출처라도 Origin 헤더를 붙이고, 조회(GET)에는 붙이지 않습니다. 그래서 허용 출처 설정이 잘못되어 있으면 아래처럼 동작합니다.

화면 동작	HTTP 메서드	Origin 헤더	결과
목록 · 상세 조회	GET	없음	정상
트래킹 등록 · 취소	POST · DELETE	있음	403 Forbidden

> ※「조회는 되는데 등록만 안 되는」증상이 나타나면 대부분 이 문제입니다. 브라우저 개발자 도구 [Network] 탭에서 해당 요청이 403 인지 확인하십시오.

허용 출처는 아래 설정으로 지정합니다. 소스에 고정되어 있지 않으므로 배포 환경에 맞게 바꾸면 됩니다.

구분	설정 위치	값 예시
로컬	TrackingCommandReceiver/src/main/resources/application.properties	http://localhost:[*]

	(cors.allowed-origin-patterns)	
운용 서버	initResource/DataEventTrackingService/.env.prod (cors_allowed_origin_patterns)	http://{serverIp}:8088

```
# initResource/DataEventTrackingService/.env.prod
cors_allowed_origin_patterns=http://{serverIp}:8088
```

※ 여러 주소를 허용할 때는 쉼표로 구분합니다. 포트 전체를 허용하려면 대괄호로 감쌉니다. (예: `http://localhost:[*],http://127.0.0.1:[*]`)

※ 운용 서버의 IP 가 바뀌면 이 값도 반드시 함께 바꾸어야 합니다. 빠뜨리면 화면은 뜨고 조회도 되지만 트래킹 등록·취소만 실패합니다. (6 장「마」·7 장「마」 참고)

④ 로컬 구동 방법 (확인된 절차)

로컬에서도 화면과 API 의 출처를 같게 맞추면 됩니다. 개발 서버(5173)로 들어온 `/dataeventtrackingservice/` 요청을 백엔드(22419)로 전달하는 프록시가 `vite.config.ts` 에 이미 포함되어 있으므로, `src/config/trackingApi.ts` 의 주소를 개발 서버 주소로 바꾸기만 하면 됩니다.

```
// FrontEnd/src/DataEventTrackingDashboard/src/config/trackingApi.ts


// const TARGET: Target = { baseUrl: 'http://220.124.222.83:8088', contextPath:
'/dataeventtrackingservice' }
const TARGET: Target = { baseUrl: 'http://localhost:5173', contextPath:
'/dataeventtrackingservice' }
```

이 파일에는 아래 네 가지 설정이 주석으로 준비되어 있습니다. 사용할 줄 하나만 주석을 해제하면 됩니다.

용도	baseUrl	contextPath	비고
제주 TP 운용 서버	http://220.124.222.83:8088	/dataeventtrackingservice	기본값
그 외 운용 서버	http://{serverIp}:8088	/dataeventtrackingservice	Nginx 경유
로컬 (개발 서버 경유)	http://localhost:5173	/dataeventtrackingservice	브라우저 확인은 이 설정을 사용
로컬 (백엔드 직접 호출)	http://localhost:22419	빈 값	브라우저에서는 CORS 차단. curl · Postman 전용

※「로컬 (백엔드 직접 호출)」설정은 이전 소스에서 각 호출부에 `// 내부 테스트용` 으로 주석 처리되어 있던 것과 동일한 주소를 만듭니다. `http://localhost:22419/fdt/detrck/rcv/...` 와 같이 접두어 없이 호출되며, 브라우저가 아닌 `curl` · Postman 등으로 백엔드만 단독 점검할 때 사용합니다.

참고로 `vite.config.ts` 에 포함된 프록시 설정은 아래와 같습니다. (개발 서버에서만 동작)

```
'/dataeventtrackingservice': {
    target: 'http://localhost:22419',
    changeOrigin: true,
```

```
    rewrite: (path) => path.replace(/^\/dataeventtrackingservice/, ''),
},
```

TS trackingApi.ts | vite.config.ts ×

vite.config.ts > default > server > proxy

```ts
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

// https://vite.dev/config/
export default defineConfig({
    plugins: [react(), tailwindcss()],
    server: {
        proxy: {
            // 데이터/이벤트 트래킹 백엔드(TrackingCommandReceiver).
            // 운용 환경에서는 Nginx가 같은 역할을 하며, 이 설정은 개발 서버에서만 동작한다.
            // src/config/trackingApi.ts 의 주소를 개발 서버(localhost:5173)로 두면
            // 화면과 API가 같은 출처가 되어 CORS 없이 로컬 백엔드를 호출할 수 있다.
            '/dataeventtrackingservice': {
                target: 'http://localhost:22419',
                changeOrigin: true,
                rewrite: (path) =>
                    path.replace(/^\/dataeventtrackingservice/, ''),
            },
            '/api': {
                target: 'http://localhost:9505',
                changeOrigin: true,
                rewrite: (path) => path.replace(/^\/api/, '/api'),
            },
```

[그림 36] vite.config.ts – 개발 서버 프록시 설정

`npm run dev` 로 기동한 뒤 아래 명령어로 확인합니다.

```
curl -o /dev/null -w "%{http_code}\n" \
  http://localhost:5173/dataeventtrackingservice/fdt/detrck/rcv/tracking-request-list
```

설정 전후의 응답을 비교하면 아래와 같습니다.

호출 경로	응답	판정
`http://localhost:22419/dataeventtrackingservice/fdt/...` (직접)	`404`	실패 — 경로 없음
`http://localhost:22419/fdt/...` (직접, 브라우저 출처)	`403`	실패 — CORS 차단
`http://localhost:5173/dataeventtrackingservice/fdt/...` (프록시)	`500`	정상 도달

> ※ 위의 `500` 은 트래킹 이력이 없을 때의 응답입니다. 백엔드 로그에 `getTrackingRequestList` 호출 기록이 남는 것으로 요청이 컨트롤러까지 전달된 것을 확인하였습니다. 트래킹을 1 건 등록하면 정상적으로 목록이 반환됩니다.
>
> ※ 서버에 배포하기 전에는 `DEFAULT_BASE_URL` 을 반드시 운용 서버 주소로 되돌린 뒤 빌드해야 합니다. 로컬 주소가 남은 채로 빌드하면 화면은 표시되나 데이터가 조회되지 않습니다.

※ 소스를 수정하지 않고 빌드 시점에만 주소를 바꾸려면 환경 변수 `VITE_DATA_TRACKING_URL` 을 사용할 수도 있습니다. 이 값이 지정되면 `DEFAULT_BASE_URL` 보다 우선합니다.

※ 연합트윈 메타 조회 주소(`DataTrackingFormItem.tsx`)는 외부 서버를 직접 호출하므로 위 절차로 해결되지 않습니다. 해당 서버에 접근할 수 없는 환경에서는 트래킹 등록 화면의 연합트윈 · 디지털객체 목록이 비어 있게 되며, 이 경우에도 목록 조회 · 상세 조회 등 나머지 기능은 정상적으로 확인할 수 있습니다.

○ 9) 검증 결과 요약

검증 항목	수행 내용	결과
백엔드 빌드	`./mvnw clean package -DskipTests`	BUILD SUCCESS (9 개 프로젝트) · jar 8 개 생성
프론트엔드 패키지 설치	`npm install`	패키지 319 개 설치 완료
프론트엔드 빌드	`npm run build`	타입 검사 통과 · `dist` 생성
로컬 인프라 기동	`docker compose -f docker-compose.local.yml up -d`	컨테이너 2 개 Up
스키마 자동 생성	`show tables from dataEventTrackings`	테이블 11 개 생성 확인
백엔드 구동	jar 7 개 기동	7 종 모두 정상 기동
DB · 메시지 큐 연결	`/actuator/health`	`db` · `rabbit` 모두 `UP`
서비스 등록	Eureka 레지스트리 조회	Gateway · CommandReceiver 등록 확인
프론트엔드 구동	`npm run dev` / `npm run preview`	`HTTP 200` 응답
화면 ~ 백엔드 연결	개발 프록시 경유 조회 API 호출	백엔드 컨트롤러 도달 확인

※ 본 절의 검증 범위는 작업 PC 에서의 빌드와 구동까지입니다. 백엔드 Docker 이미지 생성(6 장「사」)과 운용 서버 배포(6 장 전체)는 포함되지 않습니다.

※ 백엔드는 빌드 산출물(jar)을 `java -jar` 로 직접 기동하여 확인하였습니다. STS 의 Boot Dashboard 로 기동하는 경우에도 동일한 기동 로그가 출력됩니다. (4 장「바」참고)

○ 10) 검증 중 확인된 유의 사항

로컬에서 처음 구동할 때 실제로 마주치게 되는 사항입니다. 서버 배포 시에는 해당하지 않거나 자동으로 해결되는 항목이므로, 로컬 구동 단계에서만 참고하시기 바랍니다.

구분	증상	조치

빌드	macOS / Linux 에서 `./mvnw` 실행 시 `permission denied`	`chmod +x mvnw` 를 수행하거나 `sh ./mvnw` 형태로 실행합니다.
구동	트래킹 서비스 5 종이 `APPLICATION FAILED TO START` 로 기동 실패하고 `Failed to create parent directories for [/hdd/DtEvtTracking/...]` 가 출력됨	로그 저장 경로가 없기 때문입니다. 「○ 5)」의 절차로 로그 폴더를 생성합니다.
구동	`rabbit` 항목이 `DOWN` 이거나 데이터베이스 연결에 실패함	`application.properties` 의 접속 정보가 운용 서버 주소로 되어 있는 상태입니다. 4 장「라」에 따라 2 장「다」5)의 로컬 값으로 변경합니다.
화면	화면은 뜨지만 트래킹 목록이 조회되지 않고, 브라우저 개발자 도구에 CORS 오류 또는 `404` 가 표시됨	`src/config/trackingApi.ts` 의 주소를 개발 서버(`http://localhost:5173`)로 바꿉니다. 「○ 8)」 참고. `.env.production` 을 수정해도 로컬에는 반영되지 않습니다.
화면	목록 조회는 되는데 트래킹 등록·취소만 `403 Forbidden` 으로 실패함	허용 출처(CORS)에 대시보드 주소가 없기 때문입니다. `.env.prod` 의 `cors_allowed_origin_patterns` (로컬은 `application.properties` 의 `cors.allowed-origin-patterns`)에 해당 주소를 추가합니다.
화면	트래킹 등록 화면의 연합트윈 · 디지털객체 목록이 비어 있음	연합트윈 메타 조회 서버(`220.124.222.84:1221`)에 접근할 수 없는 상태입니다. 접근 가능한 주소로 변경하거나, 해당 목록 없이 나머지 기능을 확인합니다.
조회	데이터가 없는 상태에서 조회 API 가 `HTTP 500` 을 반환함	조회 결과가 없을 때의 정상 동작입니다. 트래킹을 1 건 등록한 뒤 다시 확인합니다.